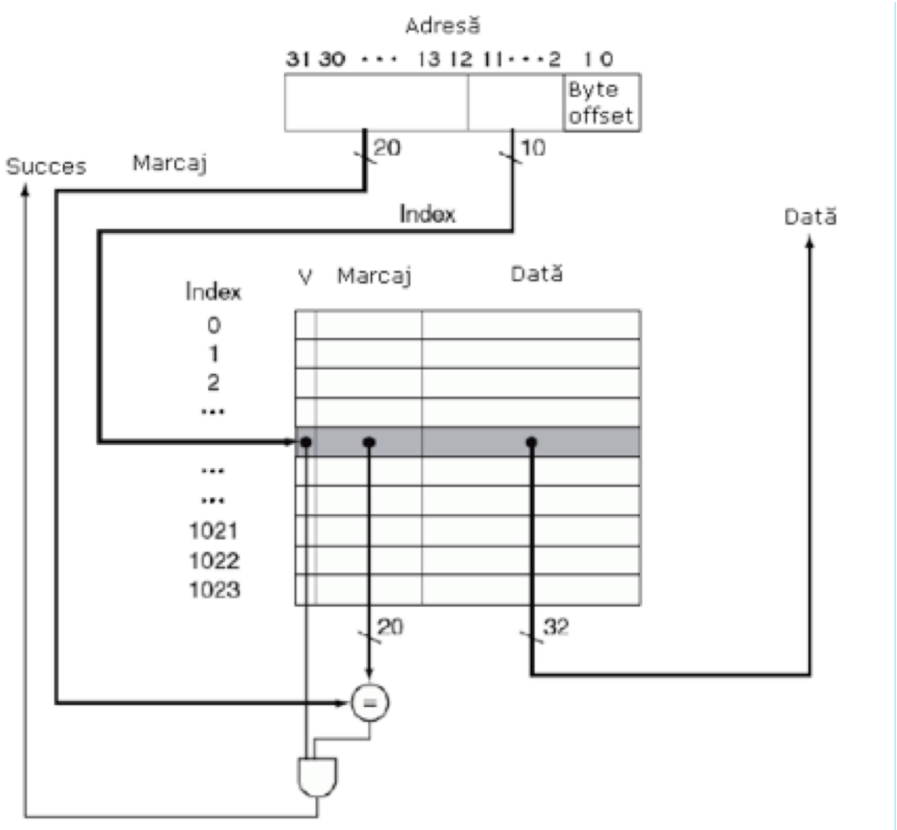
Grile Calculatoare Numerice 2

- Parametrul "d" al unei adrese de memorie generată de ALU reprezintă
  - cei mai putin semnificativi biti din adresa (curs)
  - log(2)L biti(curs)

- Localizarea temporară presupune:
  - dacă o locație este referită, atunci ea va fi referită din nou în viitorul apropiat (curs)
  - daca se face referire la un anumit obiect din memorie, este posibil ca acesta sa fie referit din nou cat de curand (+1)

- Se consideră următorul program:
  instrucțiunea 1 – 096 ADD
  instrucțiunea 2 – 100 J 304
  instrucțiunea 3 – 104 ADD
  instrucțiunea 4 – 304 ADD
  - **instrucțiunea 3 nu trebuie executată (curs)**

- Se consideră memoria cache din figura următoare



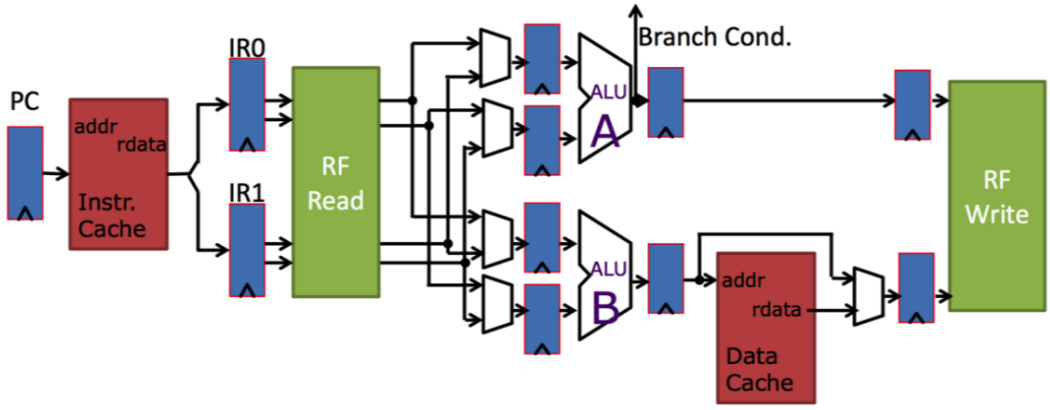
  Fiecare cuvânt se găsește:
  - în propriul său bloc
  Indexul memoriei cache este folosit la:
  - selectarea blocului de memorie cache (+2)
  - compararea cu valoarea din câmpul marcaj al memoriei cache
  Câmpul marcaj din adresă este folosit la:
  - compararea cu valoarea din câmpul marcaj al memoriei cache

  Marcajul din memoria cache este comparat cu porțiunea superioara a adresei pentru:
  - a determina dacă locația din memoria cache este diferită sau nu de adresa cerută

- Avem o memorie cache având 4 blocuri de câte 1 cuvânt. Memoria cache corespondență directă. Să se găsească numărul de MISS-uri pentru următoarea secvență de adrese de bloc: 0,8, 0, 6, 8
  - **5 (corect, curs)**

  Să se găsească numărul de MISS-uri pentru următoarea secvență de adrese de bloc: 8,8, 2, 0, 8
  - 4

- Se consideră procesorul din figura de mai jos:



Execuția programului de mai jos conduce la un hazard ?

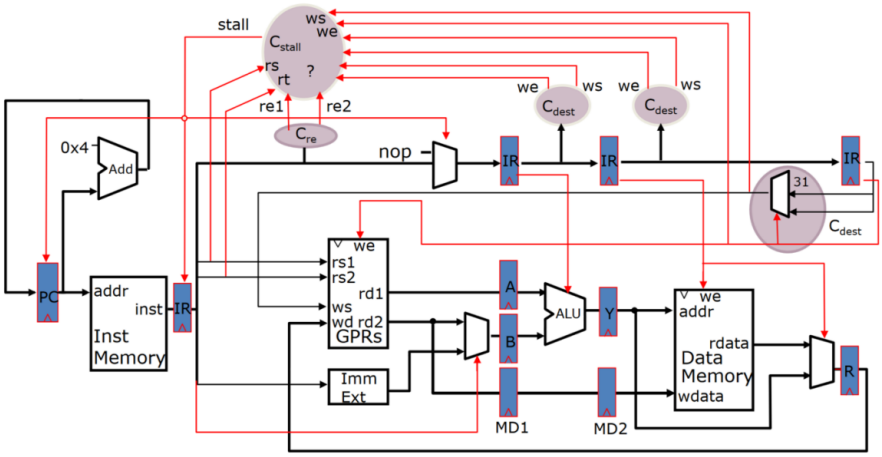
Instrucțiunea	T0	T1	T2	T3	T4	T5
ADDIU	IF	ID	EX_A	W		
LW	IF	ID	EX_B	W		
LW		IF	ID	EX_B	W	
ADDIU		IF	ID	EX_A	W	
LW			IF	ID	EX_A	W
LW			IF	ID	EX_B	W

  - da, un hazard de date (2 voturi)

Executia programului de mai jos in conditiile ideale (nu exista hazard de nici un fel) este: incorecta, incorecta deoarece chiar daca se executa 2 instructiuni in paralel caile pe care ele se executa nu sunt corecte

Instrucțiunea	T0	T1	T2	T3	T4	T5
ADDIU	IF	ID	EX_A	W		
LW	IF	ID	EX_B	W		
LW		IF	ID	EX_B	W	
ADDIU		IF	ID	EX_A	W	
LW			IF	ID	EX_A	W
LW			IF	ID	EX_B	W

- În cazul implementării de mai jos, semnalul WE este necesar deoarece:



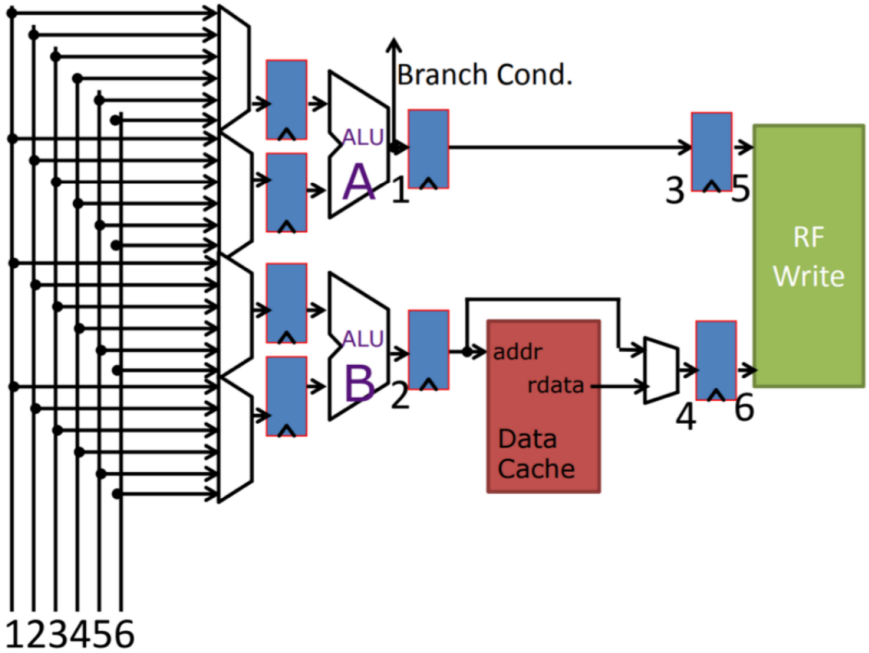
- există instrucțiuni în MIPS care nu scriu nici un registru
- este posibil să avem spre execuție instrucțiunea STORE

- Tratarea unei întreruperi presupune cel puțin execuția următorilor pași: (astea par a fi in curs)
  - salvarea conținutului registrului EPC pentru a permite întreruperi în întreruperi
  - conținutul instrucțiunii curente se salvează într-un registrul special EPC
  - utilizarea instrucțiunii RFE (o instrucțiune de salt indirect) pentru resume la cod
  - citirea unui registru status care ne indică cauza întreruperii

- În cazul lui RAID 3:
  - **vom avea 1 extra disk pentru menținerea informației de verificare (corect, curs)**
  - **adăugăm informație redundantă pentru restaurarea informației pierdute în caz de crash (corect, curs)**
  - **schema folosită este determinarea parității informației (corect, curs)**
- Fiecare stagiu trebuie să:
  - posede resurse hardware care sunt necesare execuției task-ului

- Scrierea în memoria cache prin intermediul tehnicii WRITEBACK/COPYBACK presupune:
  - **doar scriere în memoria cache (curs)**
  - **dirty bit = 1 cand avem HIT (pe scriere) - curs**

- În cazul unui procesor superscalar IO implementarea tehnicii de bypass presupune introducerea unor multiplexoare ca în figura de mai jos:

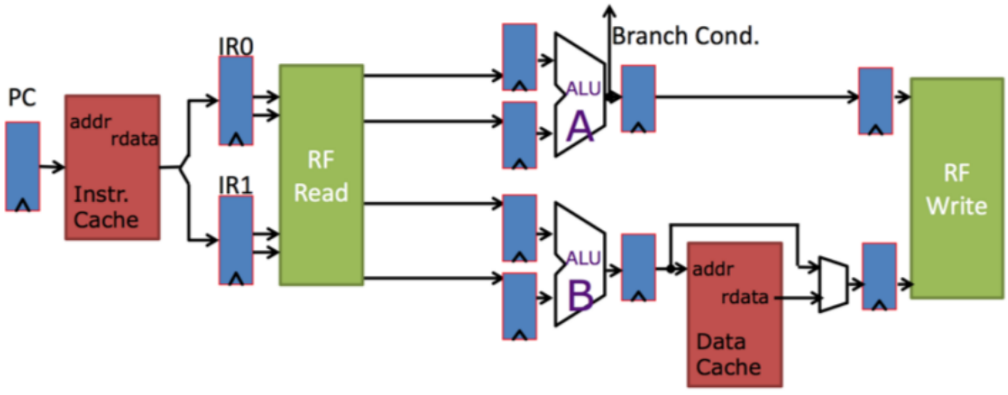


Un mare dezavantaj al acestei implementări este creșterea dimensiunii multiplexoarelor odată cu creșterea dimensiunii BA-ului. Pentru eliminarea acestui dezavantaj putem: Select one or more: O soluție ar fi să împărțim stagiile DECODE și ISSUE(curs)
    - să folosim clustere de procesoare superscalare
    - să facem bypass doar la anumite locații
    - să nu implementăm un bypass complet
    - să folosim procesare out-of-order

- Bandwidth-Delay Product se definește ca și:
    - cantitatea de date ce pot fi transferate în același moment de timp
    - cantitatea de date ce poate fi accesata * unitatea de timp (+1)
    - niciun raspuns corect (+2)

- Orice memorie cache poate fi definită prin:
    - 3 parametri (curs):
        - Numărul de linii cache – C
        - Dimensiunea liniei de cache - L
        - Gradul de asociativitate - m
    - în cazul în care este posibilă definirea parametrizată a unei memorii cache, atunci unul dintre parametri este "dimensiunea liniei cache"
    - în cazul în care este posibilă definirea parametrizată a unei memorii cache, atunci unul dintre parametri este C
    - în cazul în care este posibilă definirea parametrizată a unei memorii cache, atunci unul dintre parametri este "m"
    - în cazul în care este posibilă definirea parametrizată a unei memorii cache, atunci unul dintre parametri este "gradul de asociativitate"
    - în cazul în care este posibilă definirea parametrizată a unei memorii cache, atunci unul dintre parametri este "numărul de linii cache"

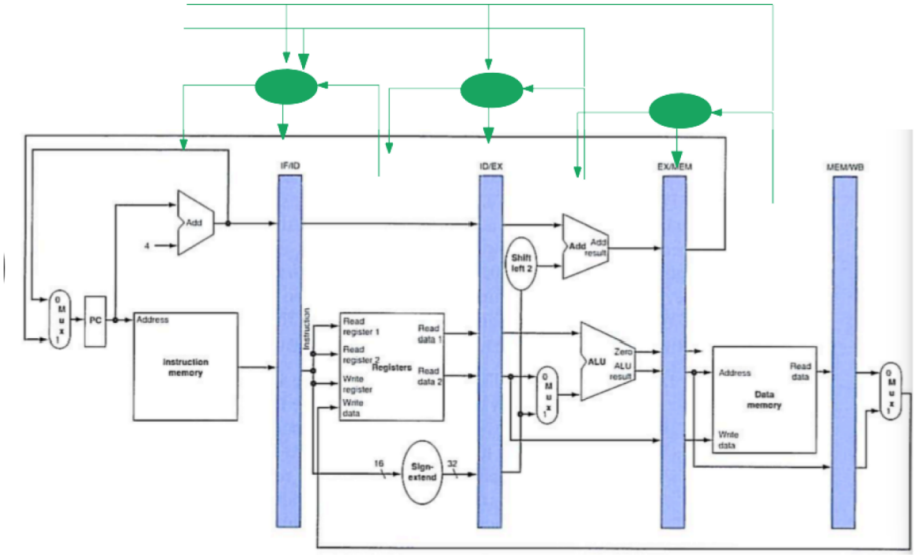
- În cazul procesorului prezentat mai jos, ce se întâmplă dacă în faza de ID prima instrucțiune este un LOAD:



    - comutarea execuției pe BA-ul B

- STANDBY SPARES presupune:
    - **cuplarea unor resurse hardware noi imediat ce o resursă hardware este defectă (corect, curs)**
    - cuplarea unor resurse hardware noi imediat ce o resursă hardware este defectă fără intervenția administratorului de sistem

- Se consideră următoarea implementare a unui procesor în BA cu reliefarea controlului (cu culoarea verde). Să presupunem că avem o instrucțiune mai devreme care este dependentă de o instrucțiune mai târzie și viceversa. În cazul unui ciclu mare de ceas :

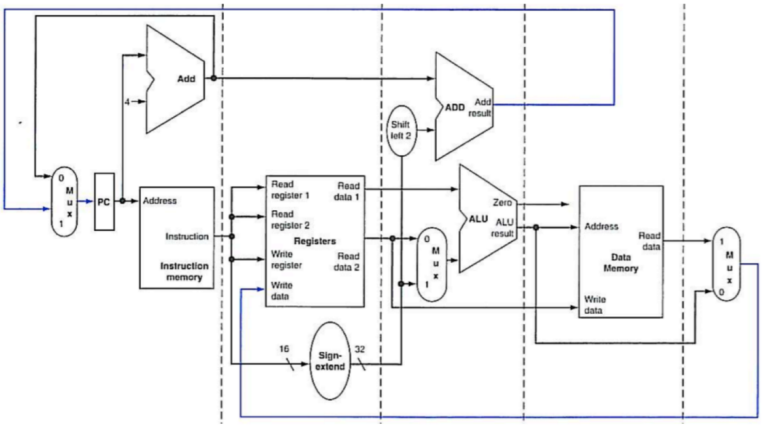


  - toate instrucțiunile vor fi dependente între ele și mașina se va opri
  - toate instructiunile vor fi dependente intre ele si masina se va opri

- În cazul unei memorii cache se folosește localizarea spațială deoarece se dorește:
  - **ca blocul memoriei cache să fie mai mare decât lungimea unui cuvânt (curs)**
  - nici un raspuns corect

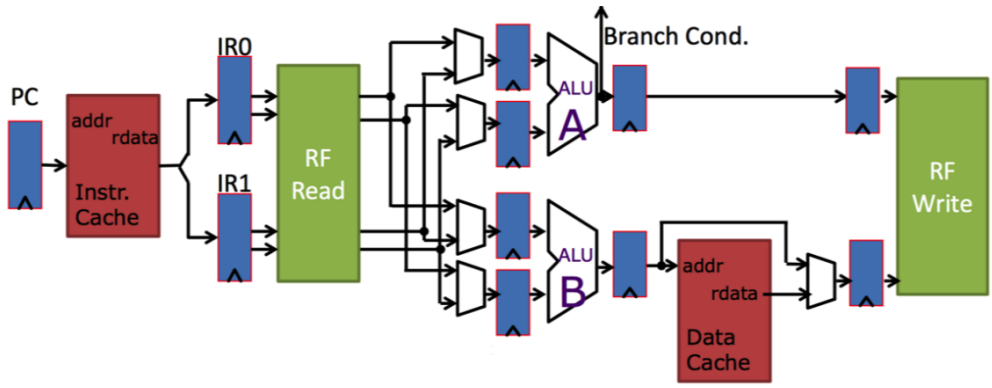
- Procesoarele superscalare permit execuția mai multor instrucțiuni în paralel. În acest caz CPI este:
  - mai mic ca 1 (curs)

- În figura de mai jos liniile punctate specifică stagiile de pipe în următoarea ordine:



  - IF, ID, EX, MEM, WB

- Se consideră procesorul din figura de mai jos:



Execuția programului de mai jos este:

Instrucțiunea	T0	T1	T2	T3	T4	T5
ADDIU	IF	ID	EX_A	W		
LW	IF	ID	EX_B	W		
LW		IF	ID	EX_A	W	
ADDIU		IF	ID	EX_B	W	
LW			IF	ID	EX_A	W
LW			IF	ID	EX_B	W

- **incorectă deoarece chiar dacă se execută 2 instrucțiuni în paralel calea pe care ele se execută nu este corectă**
- incorecta
- dumnezeu stie daca e corecta sau incorecta
- sau corecta corecta ?

- Definiție - Hazard de date - instrucțiunea planificată spre execuție
  - nu poate fi executată în ciclul de ceas deoarece datele necesare execuției nu sunt încă disponibile (curs)

- Cum rezolvăm situația în care avem întreruperi simultane multiple în diferite stagii de pipe în cazul unui procesor superscalar IO ?
  - instrucțiunea cea mai nouă care a generat o excepție va face KILL pentru toate instrucțiunile din spate aflate în banda de asamblare
  - instrucțiunea cea mai veche care a generat o excepție va face KILL pentru prima instrucțiune din spate aflate în banda de asamblare
  - instructiunea determinate prin algoritmul LRU care a generat o exceptie va face KILL pentru toate instructiunile din spate aflate in banda de asamblare

- Unul din MISS-urile posibile într-o memorie cache este cel legat de capacitatea:
  - mică a memoriei cache

- Se consideră ecuația de control următoare:

$$stall = (\cancel{((rs_D = ws_E).we_E} + \cancel{(rs_D = ws_M).we_M} + (rs_D = ws_W).we_W).re1_D$$
$$\cancel{+((rt_D = ws_E).we_E} + (rt_D = ws_M).we_M + (rt_D = ws_W).we_W).re2_D)$$

din care primii trei termeni au fost eliminați.

De ce nu mai avem nevoie de ei ?
  - deoarece avem metoda de forwarding (spune o grila) (+1)
  - deoarece avem metoda de bypass (spune o grila) (+1)
  - pot fi ignorați fără a influența implementarea finală a controlului
  - niciun raspuns corect

- Issue este:
  - stagiul în care toți operanzii există așa că putem executa instrucțiunea
  - stagiul în care nu toți operanzii există așa că trebuie să golim BA-ul (+1)
  - un alt stagiu in BA

- Se consideră următoarea execuție:

	t0	t1	t2	t3	t4	t5	t6
r1 = (r0)+10	IF_1	ID_1	EX_1	MEM_1	WB_1		
r3 = (r2)+17		IF_2	nop	ID_2	EX_2	MEM_2	WB_2

Execuția instrucțiunilor nu prezintă hazard și totuși în t2 există un NOP. Dacă am elimina NOP-ul există posibilitatea apariției unui hazard și dacă da ce tip are ?
  - structural dacă avem BRANCH
  - structural daca avem JUMP(asa spune alta grila)
  - de date dacă avem STORE
  - de date dacă avem LOAD

Execuția instrucțiunilor nu prezintă hazard și totuși în t2 există un NOP. De ce?
  - În t2 nu se poate face diferențierea dintre o instrucțiune JUMP sau BRANCH
  - În t2 nu se știe dacă trebuie executată instrucțiunea 2
  - In t2 nu se poate face diferentierea dintre o instructiune STORE sau LOAD(spune alta grila)

- Într-o BA perioada ceasului va fi
  - maximul timpilor de execuție necesari pentru fiecare stagiu

- Banda de asamblare poate fi întreruptă
  - de anumite evenimente interne sau de evenimente externe

- Se consideră ecuația de control următoare:

$$stall = (\cancel{((rs_D = ws_E).we_E} + (rs_D = ws_M).we_M + (rs_D = ws_W).we_W).re1_D$$
$$+((rt_D = ws_E).we_E + (rt_D = ws_M).we_M + (rt_D = ws_W).we_W).re2_D)$$

din care primul termen a fost eliminat.
De ce nu mai avem nevoie de primul termen ?
  - poate fi ignorat fără a influența implementarea finală a controlului (cele mai multe)
  - deoarece avem metoda de bypass
  - deoarece avem metoda de forwarding(+1)

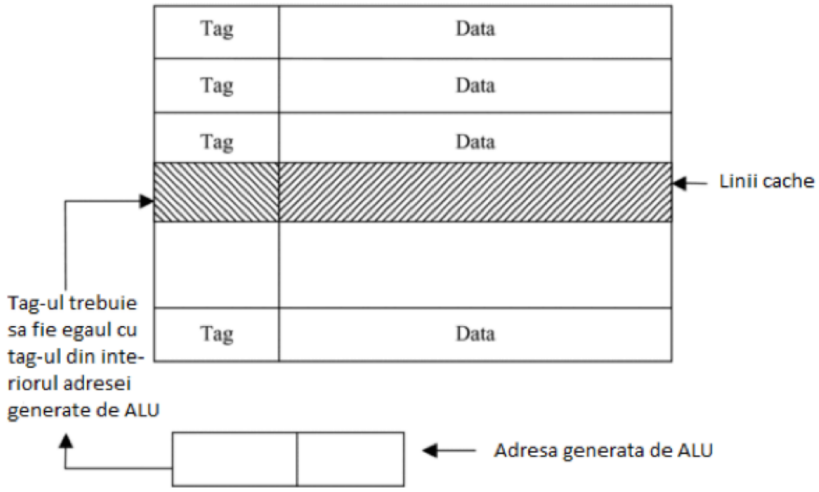
- Cause este:
  - registrul care ne spune de ce am luat în considerare excepția
  - registrul care ne spune cate exceptii avem de tratat
  - priority encoder

- În cazul rulării unui program dacă apare o dependență de ieșire atunci

- ordinea instrucțiunilor trebuie păstrată

va

- Se consideră memoria cache din figură.



În cazul unei memorii cu mapare directă câmpul TAG:

- va trebui să memoreze doar o parte din adresă |||||
- va trebui să memoreze întreaga adresă |||

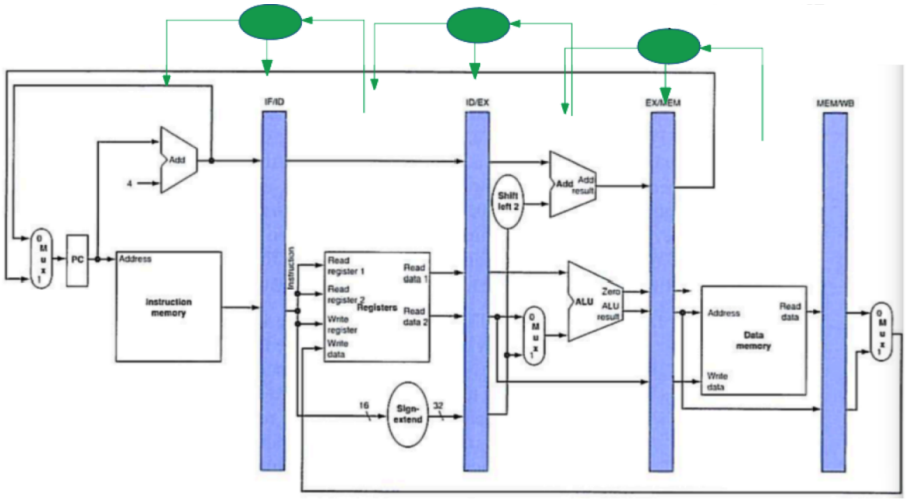
Campul TAG se foloseste pentru:

- a face o diferențiere între diferitele lucrurile pe care le vom memora în aceeași locație |||||

In cazul unei memorii cache complet asociativa(datele se pun in orice locatie de memorie cache) campul tag:

- va trebui sa memoreze urmatoarea adresa

- Se consideră următoarea implementare a unui procesor în BA cu reliefarea controlului (cu culoarea verde). Este această implementare optimă?



- Implementarea este optimă
- Implementarea nu este optima
- Implementarea nu este optima deoarece poate aparea deadlock

- O întrerupere se definește astfel:
  - oprirea execuției curente a programului și executarea unei subrutine

- Se consideră execuția programului de mai jos în cazul unui procesor superscalar IO care implementează tehnica de bypass.

BEQZ	F	D	I	Ex_A	W				
I1	F	D	I	Ex_B	-				
I2		F	D	I	-	-			
I3		F	D	I	-	-			
I4			F	D	-	-	-		
I5			F	D	-	-	-		
I6				F	-	-	-	-	
I7				F	-	-	-	-	
I8					F	D	I	Ex_A	W
I9					F	D	I	Ex_B	W

Când știm că avem o instrucțiune BEQZ?

- după stagiul EX_A

Câți ciclii de ceas vom avea penalitate?

- 4
- 3
- nu vom avea penalitati

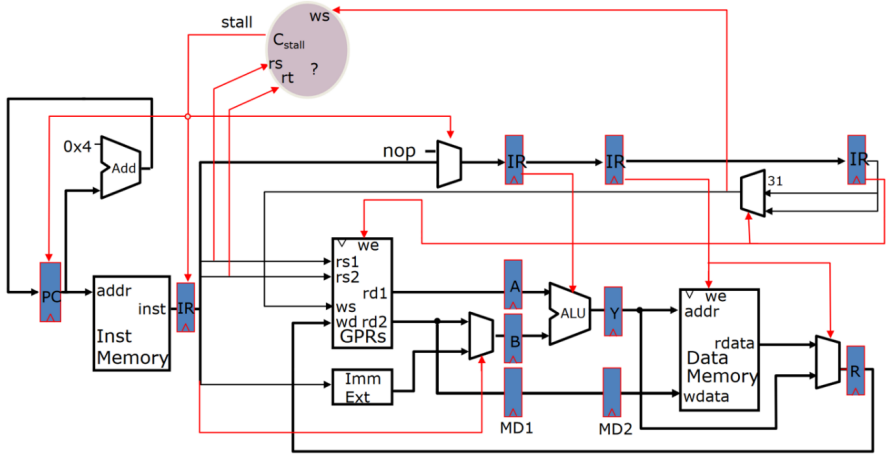
Câte instrucțiuni trebuiesc eliminate din BA în cazul execuției BEQZ și cum putem determina dacă instructiunea se va executa sau nu?

- 5 și vom folosi predictori pentru a determina dacă instrucțiunea se va executa sau nu
- 7 și vom folosi predictori pentru a determina dacă instrucțiunea se va executa sau nu
- 4 si vom folosi predictori pentru a determina dacă instrucțiunea se va executa sau nu
-

Executia programului este:

- corecta
- corecta deoarece stagiul ID poate fi spart in stagiile D si I

- Conform implementării de mai jos a controlului se poate:



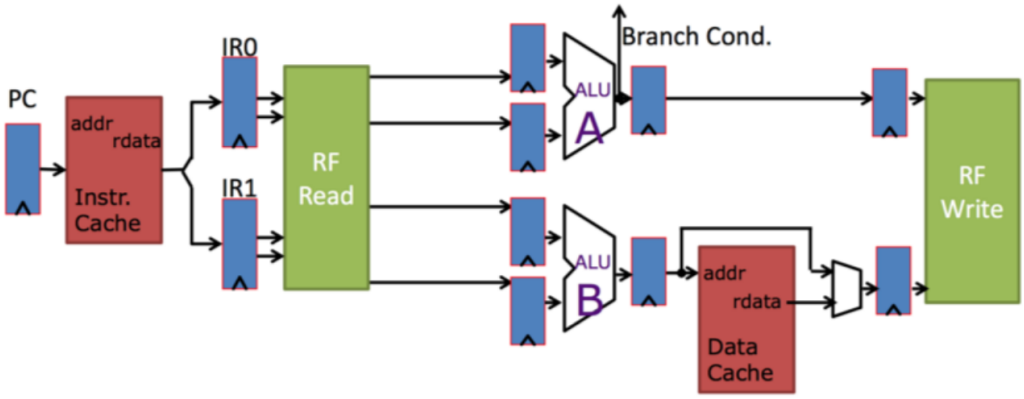
-> nu putem face stall la prima instructiune => deadlock(curs)

- **putem spune STALL oricărei instrucțiuni ce este faza IF**
- **insera NOPuri**
- nu putem insera NOP-uri în stagiile următoare(spune o grila)
- **nici un răspuns corect**
- nu insera NOP-uri (spune o grila)
- **putem insera NOP-uri in stagiile urmatoare**

- Dacă există un hazard de date acesta se poate rezolva prin:
  - stall

- forwarding
- interlock
- bypass

- În cazul procesorului prezentat mai jos se observă că BA-ul este:

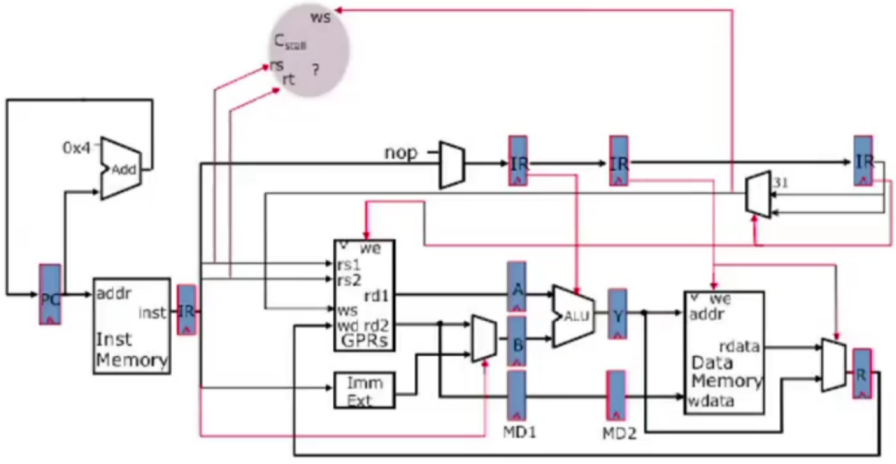


  - nesimetric
  - simetric partial(intrebarea se gaseste din nou in grile cu acest raspuns)(+1)

- Dacă 2 instrucțiuni sunt dependete:
  - ele nu pot fi paralele
  - ele trebuiesc executate în ordine
  - ele nu pot fi parțial suprapuse

- De ce avem nevoie de o memorie virtuală?
  - Permite folosirea în comun, eficientă şi sigură a memoriei de către mai multe programe
  - Înlăturarea problemelor de programare cauzate de o memorie principală mică

- Prin introducerea benzii de asamblare perioada ceasului
  - se reduce
- Al doilea pas în determinarea semnalului de control pentru o BA este:



  - se compară doar cele două surse de intrare
  - se compară destinația operandului cu cele două surse de intrare(+1)

- Prin introducerea controlului într-o BA putem (cele mai multe voturi)
  - la diferite momente de timp să executăm anumiți pași dintr-o instrucțiune
  - executa instrucțiuni paralel

- Ierarhiile de memorie folosesc avantajul localizării spațiale:

- **duplicarea blocurilor conținând cuvinte învecinate din memorie în nivelurile superioare ale ierarhiei**
- **mutarea blocurilor conținând cuvinte învecinate din memorie în nivelurile superioare ale ierarhiei** (+4)
- niciun raspuns corect(spune alta grila, dar eu nu cred 😀)

- Pentru scrierea în memorie se folosesc metodele WRITEBACK și WRITE-THROUGH. Având în vedere că ambele metode generează trafic de memorie care din ele generează un trafic de memorie mai mare?
  - WRITE-THROUGH (curs)
- Scrierea în memoria cache prin intermediul tehnicii WRITE-THROUGH presupune:
  - scriere în memoria cache dar și în nivelul următor al ierarhiei de memorie (+1 + curs)
  - nu avem nevoie de dirty bit (curs)

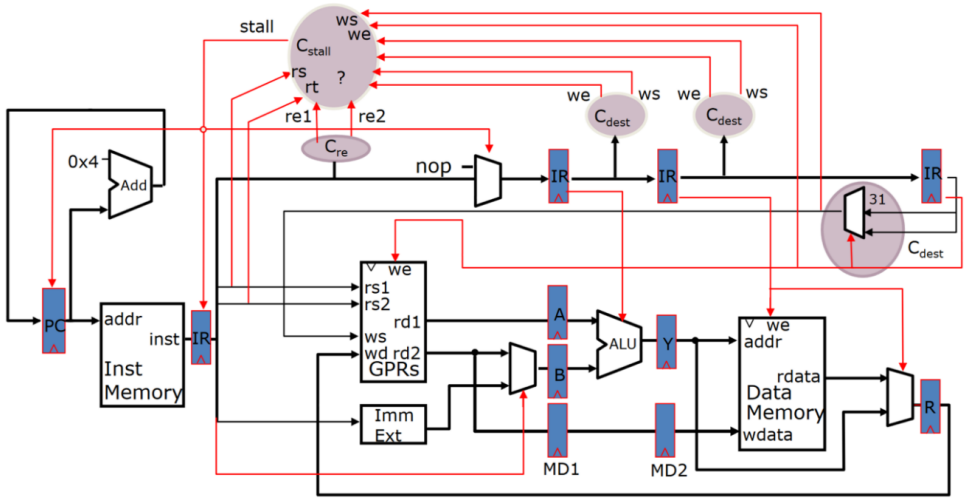
- În cazul lui RAID 4:
  - **paritatea este memorată ca blocuri și asociată cu un set de blocuri de date (corect, curs)**
  - **dezavantajul este faptul ca paritatea disk-ului trebuie recalculata la fiecare scriere (corect, curs)**
- În cazul lui RAID 5:
  - **anumite scrieri mici pot fi executate în paralel (corect, curs)**
  - **se va distribui informatia de paritate pe toate discurile (corect, curs**)

- Se consideră ecuația de control următoare:

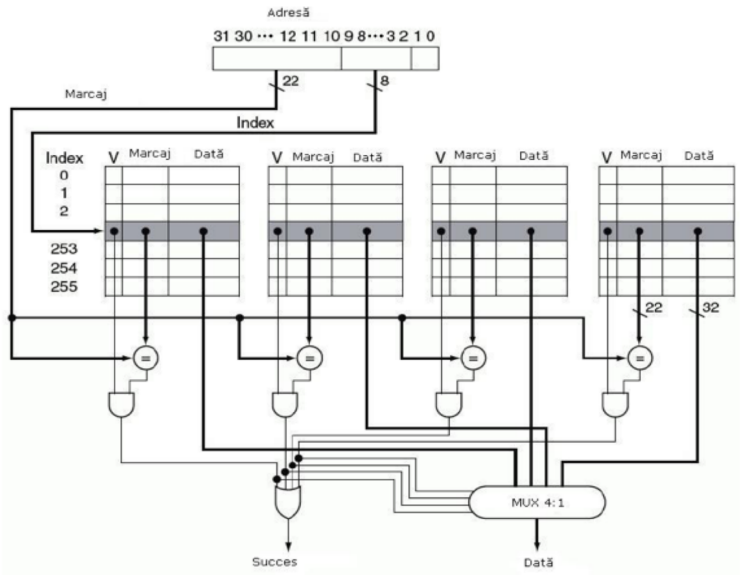
$$stall = (\cancel{((rs_D = ws_E).we_E} + \cancel{(rs_D = ws_M).we_M} + (rs_D = ws_W).we_W).re1_D$$
$$+((rt_D = ws_E).we_E + (rt_D = ws_M).we_M + (rt_D = ws_W).we_W).re2_D)$$

din care primii doi termeni au fost eliminați.
De ce nu mai avem nevoie de ei?
  - **deoarece avem metoda de forwarding**
  - **deoarece avem metoda de bypass**

- Având în vedere implementarea de mai jos, de ce MUX-ul din stagiul WB este încercuit ?



  - MUX-ul este foarte necesar deoarece instrucțiunile JUMP și LINK doar citesc nu și scriu
  - Identificatorul de registru se calculeaza pe baza unui IR
- Se consideră memoria cache prezentată în figura de mai jos.



Se folosește pentru selecția datei ce va fi transmisă la procesor un multiplexor cu semnal:
- de selectare conform câmpului "Index"
- de selectare decodificat

Marcajul fiecărui bloc este folosit la:
- comparație(+1)
- pentru selectarea blocului conținând adresa de interes

Valoarea câmpului index este folosită:
- pentru selectarea setului conținând adresa de interes

- Bandwidth se definește ca:
  - numărul de accesări pe unitatea de timp (curs)
- Întreruperile sunt:
  - prioritizate folosind LRU(+1)
  - prioritizate folosind daisy chain

- Se consideră programul de mai jos. În cazul în care există un hazard specificați tipul său:
  ADDI R1, R0, #10;
  ADDI R4, R1, #17;
  - există hazard și este de tipul RAW(curs)

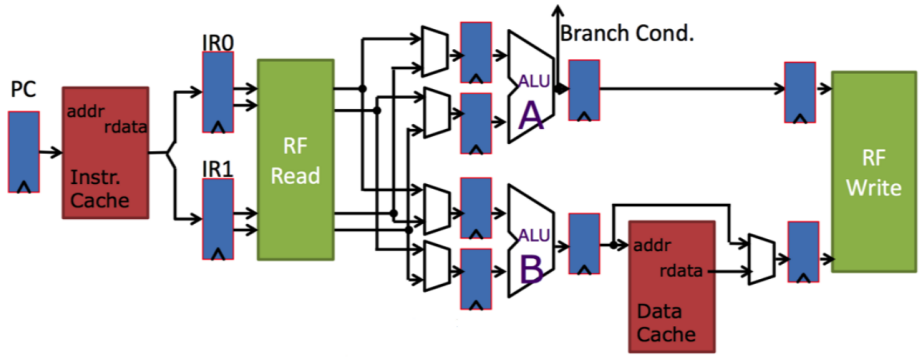
- Ierarhiile de memorie folosesc avantajul localizării temporale:
  - păstrând datele accesate recent cât mai aproape de procesor
  - niciun rasp corect(spune o grila)

- Punctul de comitere se poate pune doar în stagiul de BA:
  - în care este imposibil să mai apară vreo excepție (cele mai multe voturi)
  - in care este posibil sa mai apara un hazard de control(am gasit in alta grila)

- Localizarea spațială presupune:
  - dacă o locație este referită, atunci locațiile învecinate ei vor fi referite în viitorul apropiat (curs)
  - dacă se face referire la un anumit obiect din memorie, obiectele ale căror adrese sunt învecinate cu acesta, tind să fie adresate cât de curând (+1)

- Desfacerea unui ciclu poate fi făcută
  - **dinamic de către hardware (corect, curs)**
  - **static de către compilator (corect, curs)**

- În cadrul procesorului prezentat mai jos se observă:



  - apariția unei logici de direcționare a instrucțiunilor compusă din niste multiplexoare (+2)
  - apariția a două decodificatoare (+1)
  - apariția a două decodificatoare deoarece se decodifică 2 instrucțiuni în paralel (+1)
  - apariția a două decodificatoare deoarce ele vor verifica depindențele dintre instrucțiuni (+1)
  - aparitia registrelor RF Read si Rf Write
  - aparitia unei logici de directionare a instructiunilor compusa din niste registre
- Un task trebuie
  - să folosească toate stagiile
  - ordinea task-urilor este aceeași
- Unul din MISS-urile posibile într-o memorie cache este cel compulsory(obligatorii) - prima referință a unui bloc. Acest tip de MISS apare și în cazul:
  - unei memorii cache asociativitate parțială(+1)
  - unei memorii cache cu mapare directă(+1)
  - unei memorii cache asociativitate totală(+1)
  - unei memorii cache infinita

- Presupunem că instrucțiunea i precede instrucțiunea j.
  O dependență de ieșire apare când instrucțiunile i și j:
  - **scriu același registru (corect, curs)**
  - **scriu aceeași locație de memorie (corect, curs)**

  O antidependență între i și j apare când:
  - **instrucțiunea j scrie un registru sau o locație de memorie citită de instrucțiunea i (corect, curs)**

- În cazul scrierii în memoria cache (MISS) se folosește tehnica "No Write Allocate" care presupune:
  - doar scriere din memoria principală (CORECT - curs)

- O dependență de dată converge către:
  - Posibilitatea unui hazard
  - ordinea în care rezultatele trebuiesc calculate (+1)
  - cum un paralelism poate fi exploatat

- Paralelismul la nivel de thread presupune:
  - Multiprocessor (+1)
  - Multithreading
  - Multicore (+1)
  - Manycore (+1)

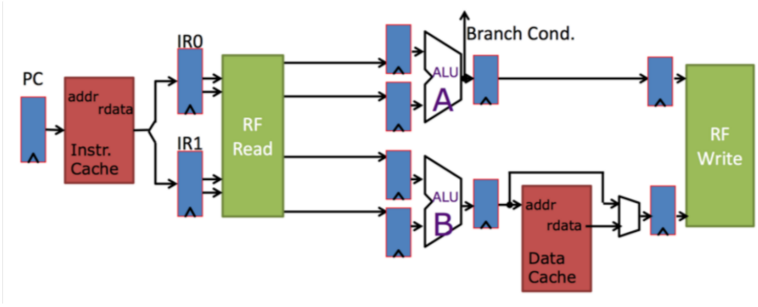
- Îmbunătățirea performanțelor unui procesor se poate face prin:
  - Paralelism la nivel de instrucțiune (+2)
  - Paralelism la nivelul datelor (+1)

- Paralelismul benzii de asamblare (+2)
- **Paralelism la nivel de thread (corect, curs)**

- Arhitectura calculatoarelor este o proiectare a straturilor de abstractizare / implementare care ne permit
  - **prelucrarea de aplicații folosind eficient tehnologiile de fabricație (curs)**
  - **executarea de informații(curs)**

- Existența compilatoarelor și runtime systems cât mai inteligente este necesară deoarece
  - se elimină instrucțiunile redundante (intrebarea se repeta, raspuns comun)
  - eliminarea instrucțiunilor care nu ne sunt utile (intrebarea se repeta, raspuns comun)
  - evitarea hazardurilor structurale ale BA

- În cazul procesorului prezentat mai jos



în faza de EX ne trebuie o logică ca să putem procesa valoarea lui
  - PC+4

în faza de IF ne trebuie o logică ca să putem procesa valoarea lui
  - PC+4
  - PC

in faza de WB ne trebuie o logica ca sa putem procesa valoarea lui
  - PC + 8

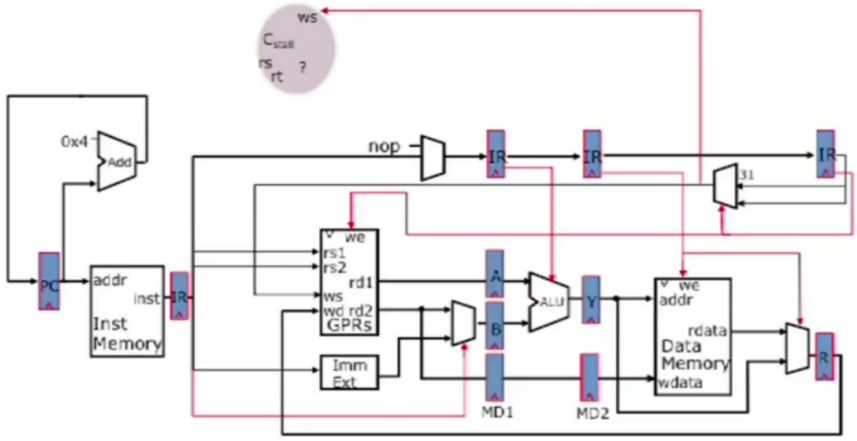
in faza de ID ne trebuie o logica ca sa putem procesa valoarea lui
  - nici un răspuns corect

Se observa:
  - Aparitia a 2 decodificatoare
  - Aparitia a 2 decodificatoare deoarece ele vor verifica dependentele dintre instr
  - \- // dependentele doar dintre anumite instructiuni

- Memoria principală trebuie să conțină:
  - **un mecanism de protecție a programelor între ele (corect, curs)**
  - **doar porțiunile active ale programelor (corect, curs)**
  - doar porțiunile ce se vor activa ale programelor ||

- O bandă de asamblare cu mai multe stagii este
  - de preferat

- Primul pas în determinarea semnalului de control îl constituie:



- verificarea câmpului RD din cadrul unei instrucțiuni MIPS

- Întreruperile/excepțiile sunt:
  - sincrone
  - asincrone

- Se consideră execuția programului de mai jos. Execuția rezolvă hazardul de date care apare între ultimele două instrucțiuni pe R5

	T0	T1	T2	T3	T4	T5	T6	T7
ADDIU R_1, R_1, 1	F	D	EX_A	W				
ADDIU R_3, R_4, 1	F	D	EX_B	W				
ADDIU R_5, R_6, 1		F	D	EX_A	W			
ADDIU R_7, R_5, 1		F	D	D	D	D	EX_A	W

În cazul folosirii tehnicii de bypass pentru ultima instrucțiune, stagiul EX_A se va executa în ciclul de ceas:
  - T4(+1)
  - T6(+1)

În cazul folosirii tehnicii de bypass pentru ultima instrucțiune, stagiul W se va executa în ciclul de ceas:
  - T5
  - T3(spune alta grila)

Pentru eliminarea acestui hazard și o execuție la viteză maximă în banda de asamblare putem:
- aplica tehnica de bypass și interschimbăm instrucțiunile 2 și 3 instrucțiuni între ele.
- interschimbam instructiunile 2 si 3 instructiuni intre ele

- Noțiunea de HIT se refră la:
  - **Regăsirea informației necesare procesorului într-un bloc de memorie superior** (curs)
  - Mutarea informației necesare procesorului într-un bloc de memorie inferior
  - Neregăsirea informației necesare procesorului într-un bloc de memorie superior
  - **nici un răspuns corect** (+3)
  - Ștergerea informației necesare procesorului într-un bloc de memorie superior
  - Duplicarea informației necesare procesorului într-un bloc de memorie superior

- Se consideră programul de mai jos:

```
DADDU    R2, R3, R4
BEQZ     R2, L1
LW       R1, 0(R2)
L1:
```

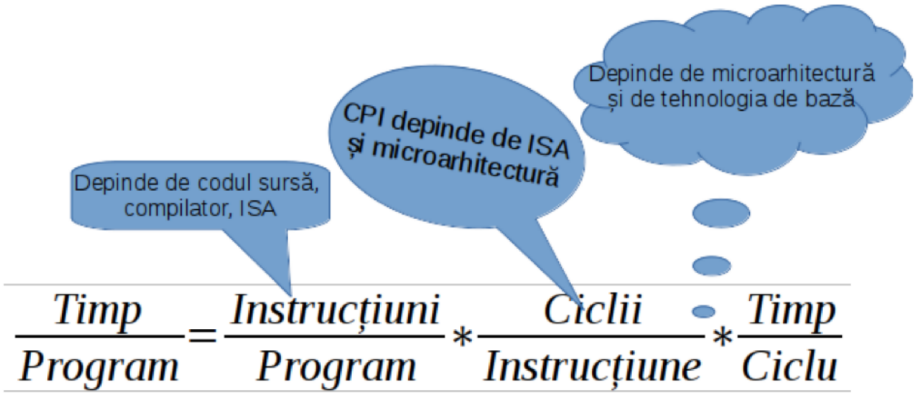
Dacă se mută instrucțiunea de LOAD înainte de branch

- **se ignoră dependența de control (corect, curs)**
- **instr LOAD poate cauza o exceptie de memorie (corect, curs)**

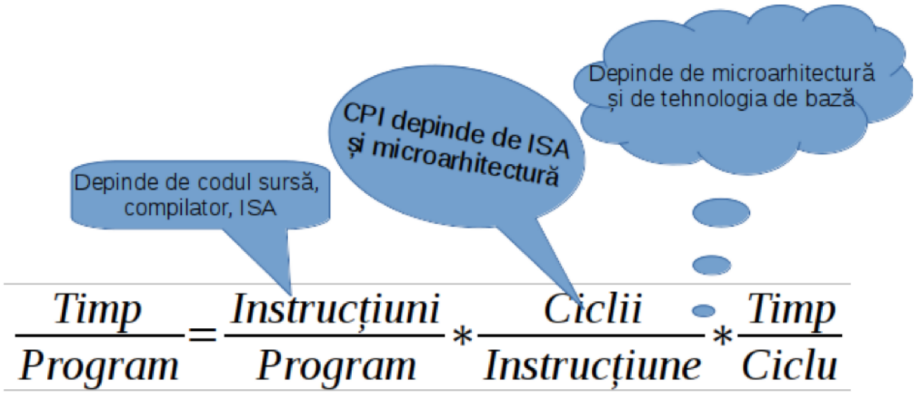
Dacă nu se menține dependența de date implicată de R2:

- se poate modifica rezultatul programului
- programul nu se poate executa(spune alta grila)

- Performanța unui procesor este dată de legea "Iron" (specificată mai jos). Pentru a crește performanța scopul este:



  - de minimizare al celor 3 factori
- Legea "Iron" (specificată mai jos) presupune existența a 3 factori. Fiecare factor este influențat de:



  -
  - compilatorul; microarhitectura și ISA dar nu foarte mult; tehnologia de implementare și microarhitectura folosită (+1)
  - niciun raspuns corect

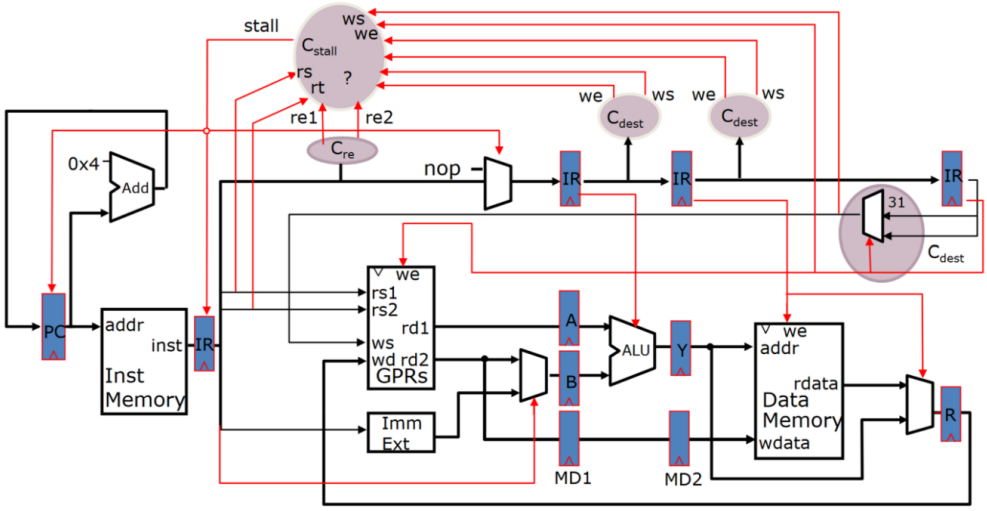
- O modalitate de reprezentare a execuției instruncțiunilor într-o BA este cea prezentată mai jos. Această reprezentare este:

Timp	t1	t2	t3	t4	t5	t6	t7	t8	.....
Instr_1	IF_1	ID_1	EX_1	MEM_1	WB_1				
Instr_2		IF_2	ID_2	EX_2	MEM_2	WB_2			
Instr_3			IF_3	ID_3	EX_3	MEM_3	WB_3		
Instr_4				IF_4	ID_4	EX_4	MEM_4	WB_4	

  - Foarte utilă ajutând la determinarea perioadei de ceas necesară funcționării unui BA non-ideală

- Foarte utilă ajutând la determinarea hazardurilor care pot apărea în funcționarea non-ideală a unei BA

- În cazul implementării de mai jos când se calculează biții WE și RE?



  - Se face în stagiul EX
  - Se face în stagiul 4
  - Se face în stagiul MEM
  - Se face în stagiul 3(spune o grila)
  - Se face in stagiul ID
  - Se face in stagiul 2
  - niciun rasp corect(spune o grila)

- Se consideră următorul program:

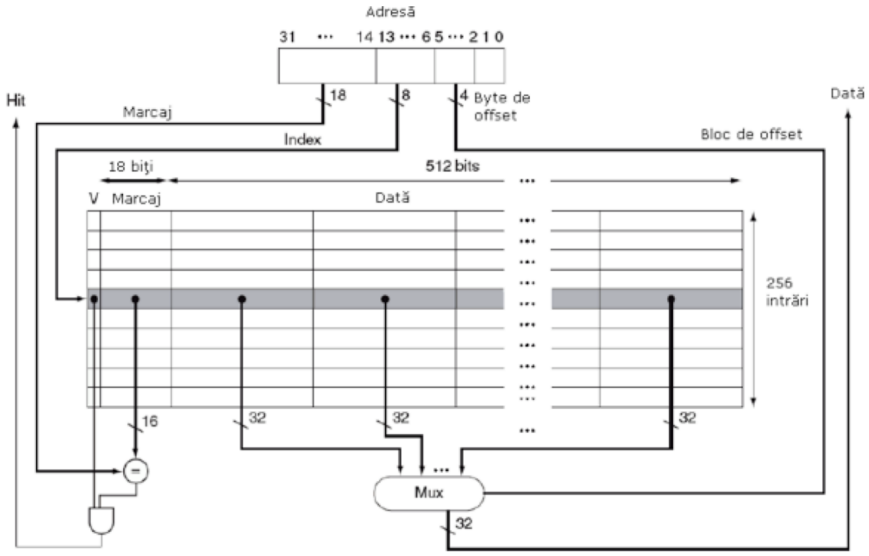
  ADDI R1, R0, #10;
  ADDI R4, R1, #17;
  ...
  În cazul existenței unui hazard execuția sa ar fi:

  -

T0	T1	T2	T3	T4	T5	T6	T7	T8	T9	T10
IF_1	ID_1	EX_1	MA_1	WB_1						
	IF_2	ID_2	ID_2	ID_2	ID_2	EX_2	MA_2	WB_2		
		IF_3	IF_3	IF_3	IF_3	ID_3	EX_3	MA_3	WB_3	
						IF_4	ID_4	EX_4	MA_4	WB_4

- În cazul lui RAID 6:
  - se folosesc mai multe blocuri secundare pentru recuperarea datelor în caz de eșec multiplu(+1)
  - **overhead-ul este dublu față de RAID 5 (corect, curs)**
  - **bloc secundar este folosit pentru recuperarea datelor in caz de esec multiplu (corect, curs)**

- Se consideră memoria cache din figura de mai jos doar ca exemplu fără ca vreun calcul să fie influențat de ea.



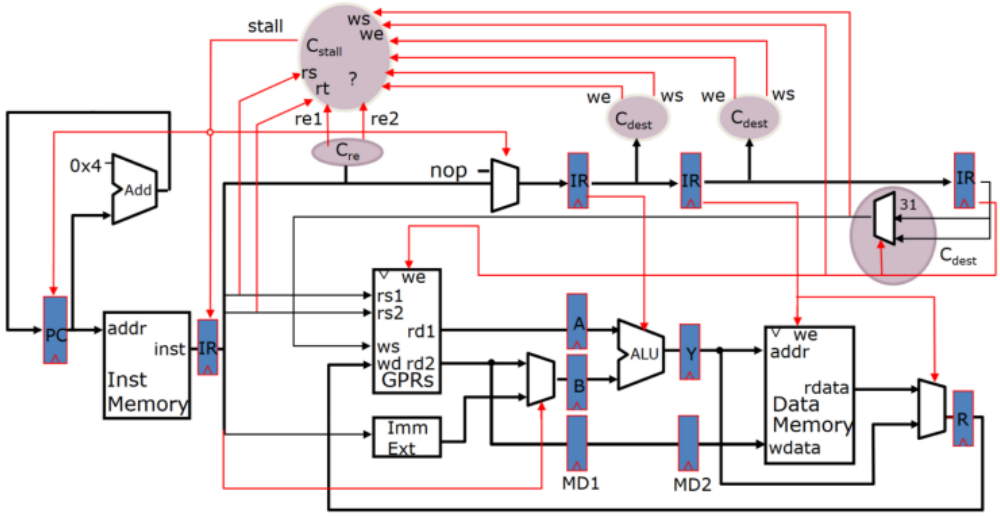
Blocul de memorie cache trebuie să fie:

- ca blocul memoriei cache să fie mai mare decât lungimea unui cuvânt

La aparitia unui esec: se vor extrage mai multe cuvinte adiacente

cuvintele extrase vor avea o probabilitate mai mare de a fi accesate in viitor(asa spune alta grila)

- În cazul implementării de mai jos, în cazul în care biții WE și RE trebuiesc determinați, aceștia se vor transmite către : Calcularea acestor biți se face în stagiul 2 (ID) al benzii de asamblare și vor fi transferați apoi prin stagiile următoare mai exact stagiile 3 și 4 (curs)
  - biții trebuiesc determinați și transmiși stagiilor 3 și 4
  - bitii trb determinati si transmisi stagiilor urmatoare(spune alta grila)



- Se considera executia urmatorului program:

```
Loop: L.D       F0, 0(R1)
      //F0 este un șir de elem
      ADD.D     F4, F0, F2
      //adăugare scalar în F2
      S.D       F4, 0(R1)
      //memorare rezultat
      DADDUI  R1, R1, #-8
      //decrementare pointer 8 bytes
      BNE       R1, R2, LOOP
      //salt R1 != R2
```

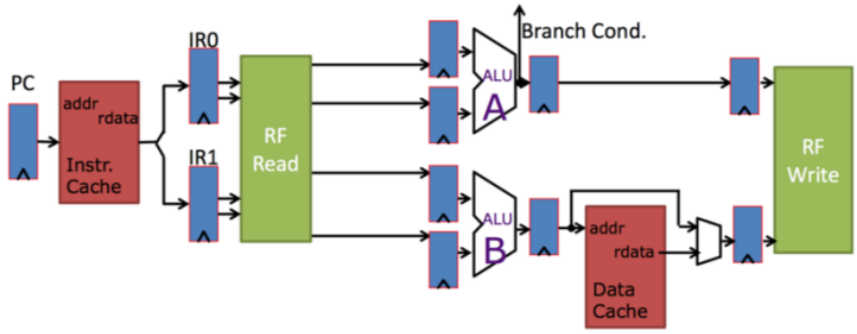
In cazul executiei acestui program:
  - ordinea instrucțiunilor trebuie păstrată pentru o execuție corectă a programului (+5)
  - F0 prezintă dependențe între L.D și ADD.D (+5)
  - R1 reprezinta dependente intre DADDIU si BNE (+1)
  - F4 reprezinta dependente intre ADD.D si S.D

- Exploatarea localizarii temporare se face prin:
  - memorarea conținutului adreselor de locație folosite
  - memorarea adreselor de locație folosite
  - **nici un răspuns corect** (+2)
  - **ștergerea conținutului adreselor de locație folosite**
  - ștergerea adreselor de locație folosite

- Definiție - Hazardul de control - Este necesar să luăm o decizie bazată pe rezultatului unei reuniuni de instrucțiuni cât timp acestea sunt în așteptare
  - **1. nici un răspuns corect**
  - 2. definiția este corectă
  - 3. definiția este incorectă - instrucțiunile trebuie să fie în faza de ID
  - 4. definiția este incorectă - instrucțiunile trebuie să fie eliminate
  - 5. **definiția este incorectă - instrucțiunile trebuie să fie în execuție (+1)**
  - 6. definiția este incorectă - instrucțiunile trebuie să fie în faza de IF

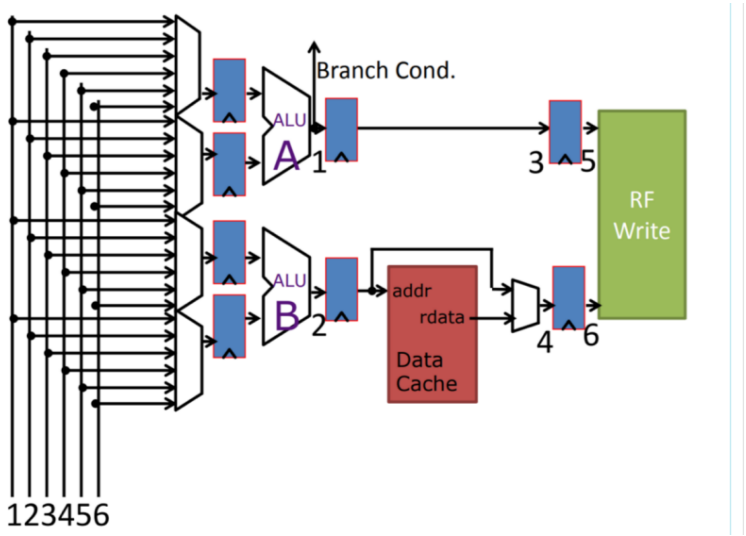
- Într-o bandă de asamblare
  - 3. nu ne putem întoarce în timp
- Un bloc de memorie virtuală este denumit
  - 4. pagină (curs)

- În cazul procesorului de mai jos:



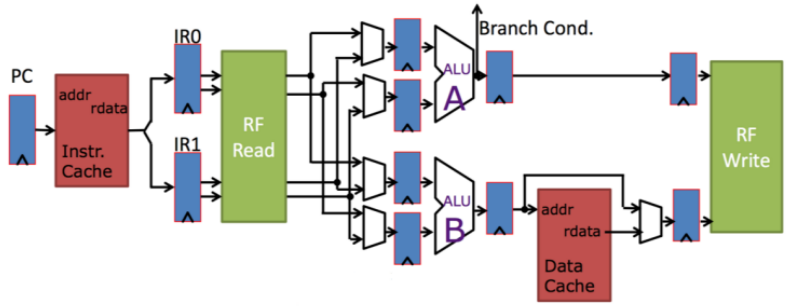
  - putem executa 2 operații pe întregi la un moment dat
  - banda de asamblare A poate executa operații pe întregi și salturi
  - banda de asamblare B poate executa operații pe întregi și accese la memorie(+1)
  - Va trebui să fim capabili să putem citi 2 instrucțiuni în același ciclu de ceas(+1)
  - nu putem executa salturi in BA-ul B(+1)
  - nu putem executa accese la memorie in BA-ul A(+1)
  - superscalar pe 2 cai OOO
  - superscalar pe 2 cai IO(spune alta grila)
  - Va trebui sa reducem numarul de porturi la FileRegister
  - Trebuie marita dimensiunea memoriei
  - Nu mai este necesar un FileRegister(spune alta grila)

- În cazul unui procesor superscalar IO implementarea tehnicii de bypass presupune introducerea unor multiplexoare ca în figura de mai jos:



  Un mare dezavantaj al acestei implementări este creșterea dimensiunii multiplexoarelor odată cu creșterea dimensiunii BA-ului. Dacă ar fi utilizat un procesor OOO atunci:
    - în banda de asamblare A nu executăm nimic
    - vom comite informația mai devreme în fișierul de registre (+3)

- Procesorul prezentat mai jos este:
  - un procesor superscalar pe 2 cai IO (CORECT - curs)



- Memoria virtuala implementeaza
  - translatarea spatiului de adrese al programului in adrese fizice (corect, curs)

- O metodă de îmbunătățire a performanței memoriilor cache presupune reducerea ratei de MISS prin folosirea în cadrul memoriei cache a unor blocuri de date de dimensiune
  - cat mai mare
- Se considera urmatorul program:
  DADDU R1, R2, R3
  BEQZ R4, L
  DSUBU R1, R5, R6
  L: …..
  OR R7, R1, R8
  Instrucțiunea OR prezintă dependență
  - **de date pentru DADDU si DSUBU (corect, curs)**

  Instructiunea DSUBU poate fi mutata deasupra branch-ului?
  - nu deoarece ar fi modificare ilegala a fluxului de control
  - nu deoarece ar fi modificare ilegala a antidependentei existenta in program
  - da deoarece ar fi modificare legala a fluxului de control(spune o grila)

- Se consideră secvența de cod următoare:

r1 «- Mem [r0 + 10]
r4 «- r1 + 17
În cazul existenței unui hazard cum se soluționează respectivul hazard ?
    - **exista hazard de date si se rezolva prin stall (curs https://drive.google.com/drive/folders/0ByVYedhOP_PlTi04akFkVnNxelk?resourcekey=0-TB92JAX-OA-9OO75uXRdwA)**
    - exista hazard de date si se rezolva prin bypass (intrebarea se repeta, dar are alt raspuns)
    - niciun raspuns nu este corect (intrebarea se repeta, dar are alt raspuns)
- Unul din MISS-urile posibile într-o memorie cache este cel legat de conflict care presupune:
    - apariția de coliziuni în memoriile cache care nu au asociativitate totală
- RAID 0 presupune:
    - 1. răspândirea datelor pe mai multe disk-uri (corect, curs)
    - 2. existența unui singur disk la nivelul utilizatorului (corect, curs)
    - 3. Performanță mare pentru sistemele de editare video (corect, curs)
    - 4. Performanță mare pentru accese la informație de dimensiune mare (corect, curs)

- Unul din MISS-urile posibile într-o memorie cache este cel compulsory(obligatorii) - prima referință a unui bloc. O posibilitate de eliminare a acestui MISS este:
    - introducerea unei unități de prefetch
- Avem o memorie cache având 4 blocuri de câte 1 cuvânt. Memoria cache este cu asociativitate pe 2 căi și pentru replasare se folosește algoritmul LRU.
  Să se găsească numărul de MISS-uri pentru următoarea secvență de adrese de bloc: 0,8, 0, 6, 8
    - 4
  Să se găsească numărul de MISS-uri pentru următoarea secvență de adrese de bloc: 8, 8, 2, 0, 2
    - 3
  Să se găsească numărul de MISS-uri pentru următoarea secvență de adrese de bloc: 8, 8, 2, 0, 9
    - 4

- Hazardul de date daca exista se poate solutiona
    - prin forwarding
    - prin interlock
    - **prin folosirea instructiunilor speculative** (curs)
    - **prin bypass (curs)**
    - **prin stall (curs)**
    - *planificare - curs, dar nu e printre variante aici*
- În cazul scrierii în memoria cache (MISS) se folosește tehnica "Write Allocate" care presupune:
    - citim blocul în memoria cache apoi scriem (CORECT - curs)

- Ce se intampla daca avem un STALL si un JUMP in stagiul de decode la acelasi moment de timp?
    - **se va produce un stall general(+1)**
    - trebuie crescuta frecventa ceasului in cazul in care executia programului nu este corecta
    - **se va genera un conflict structural major mai tarziu in BA**
    - nu stiu exact ce se intampla dar nu vor fi probleme in executia instructiunilor in BA
    - **nici un raspuns corect(+1)**
- Definitie - un hazard reprezinta situatia in care:
    - intr-o banda de asamblare urmatoarea instructiune nu poate fi executata in urmatorul ciclu de ceas
- Banda de asamblare este o tehnica
    - in care mai multe instructiuni sunt executate simultan
- Toate stagiile unei benzi de asamblare ideale trebuie sa fie
    - balansate

- toate stagiile trebuie sa dureze aproximativ acelasi timp
- complet independente din punct de vedere al executiei(o alta grila nu ia varianta asta, dar nu e validata)

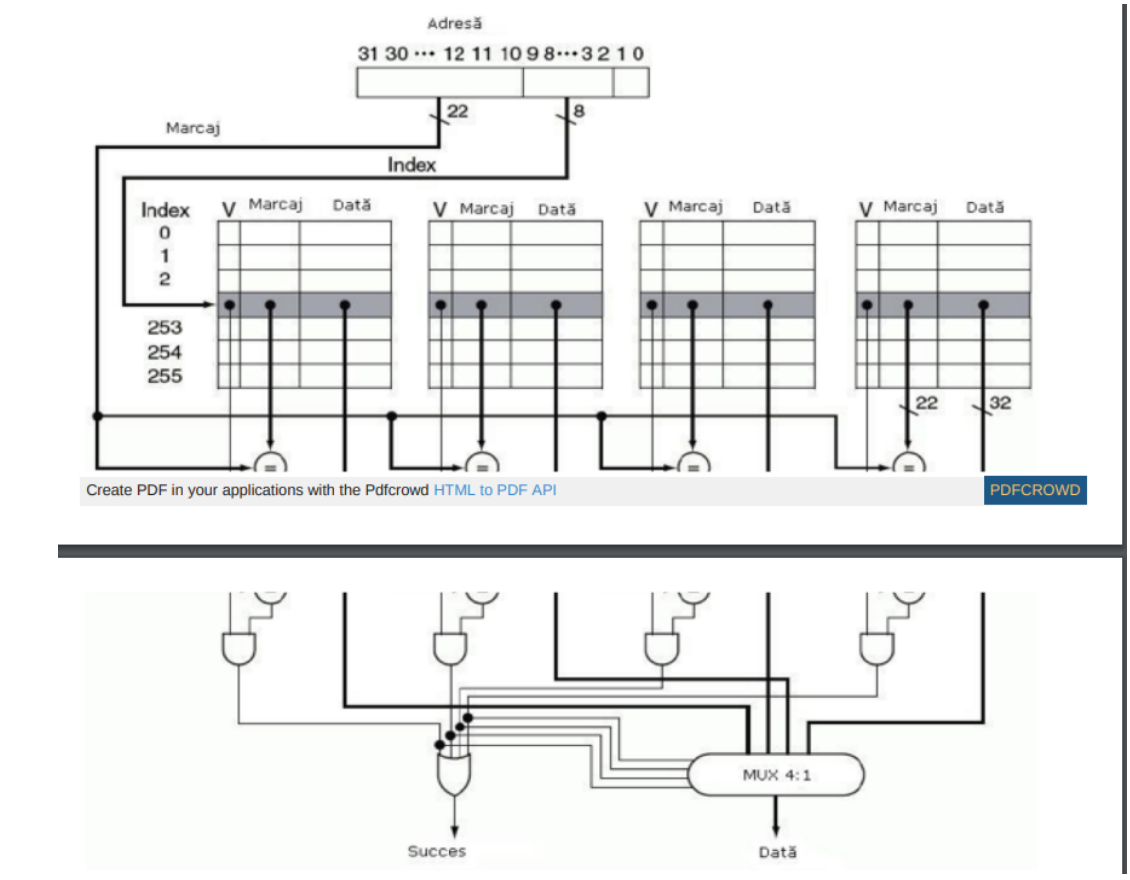
- Se considera programul de mai jos:

```
DADDU R1, R2, R3
BEQZ R12, skip
DSUBU R4, R5, R6
DADDU R5, R4
```

Presupunem ca R4 al instructiunii DSUBU nu a fost utilizar dupa instructiunea OR. Putem muta DSUBU inainte de branch?

- da (+1)
- **da, deoarece R4 este dead deci DSUBU nu poate genera o exceptie (corect, curs)**
- **da, deoarece modificarea valorii R4 inainte de branch nu poate afecta fluxul de date deci DSUBU nu poate genera o exceptie (corect, curs)**

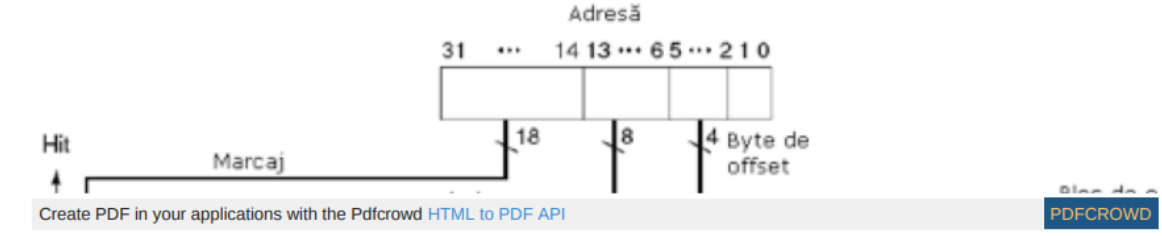
- Detectarea dependentelor in cazul registrelor este
  - usoara deoarece numele registrelor este fix in cadrul instructiunilor
- Arhitectura calculatoarelor este o proiectare a straturilor de abstractizare / implementare care ne permit -> executarea de informatii, prelucrarea de aplicații folosind eficient tehnologiile de fabricație (curs)
- Instrucțiunile implicate într-o antidependență sau o dependență de ieșire sau o dependență de nume -> pot fi reordonate (banuiala)
  -> pot fi doar executate simultan(spune alta grila)
  -> pot fi executate simultan(spune alta gila)
- Banda de asamblare poate fi întreruptă -> de anumite evenimente interne sau de evenimente externe
- Se consideră memoria cache prezentată în figura de mai jos:



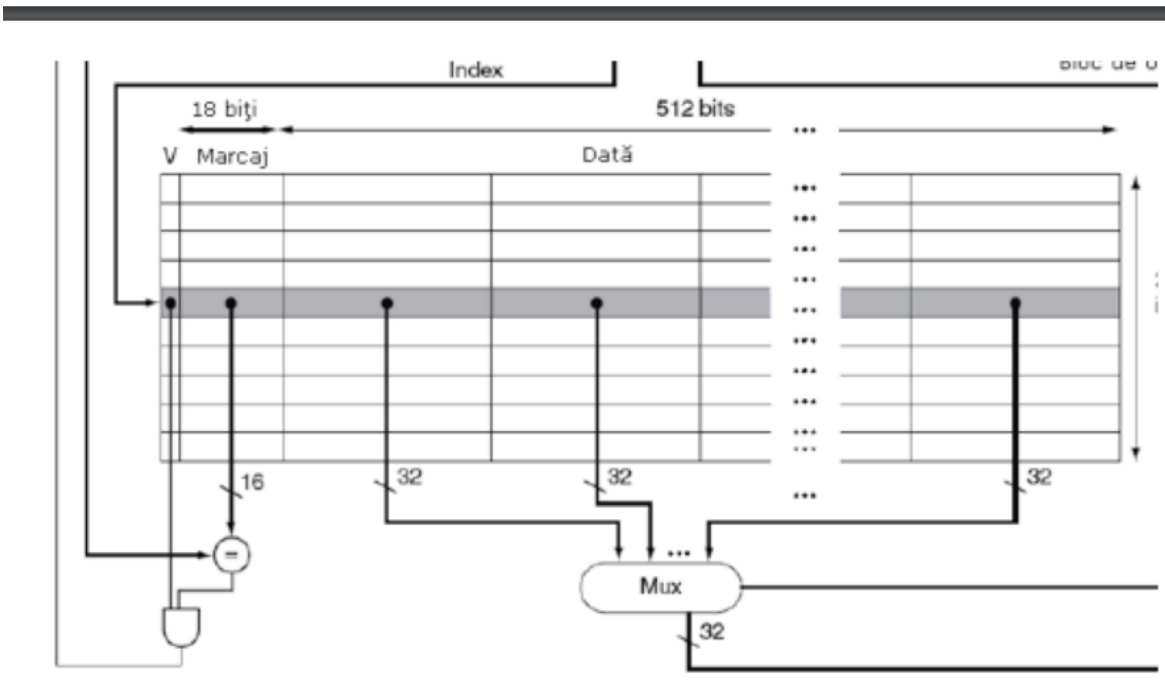
- Marcajul fiecărui bloc este folosit la:
  - comparatie (+1)

- Se folosește pentru selecția datei ce va fi transmisă la procesor un multiplexor cu semnal ->
  - de selectare conform câmpului "Index"
  - de selectare codificat (spune o grila)
  - de selectare conform câmpului "Dată"
  - de selectare decodificat

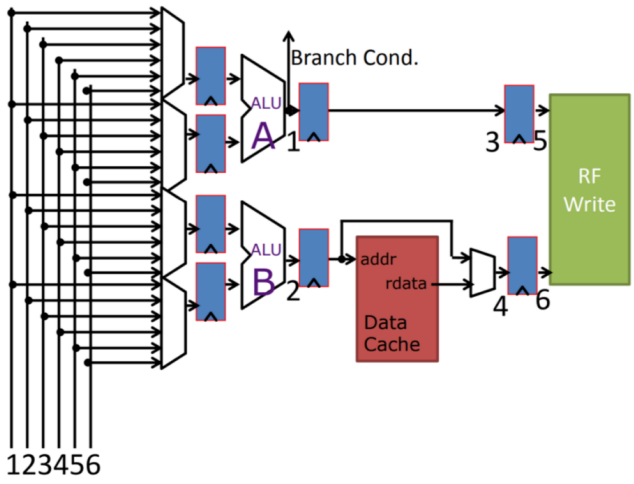
- O întrerupere se definește astfel: oprirea execuției curente a programului și executarea unei subrutine
- STANDBY SPARES presupune: -> Cuplarea unor resurse hardware noi imediat ce o resursă hardware este defectă (curs https://andrei.clubcisco.ro/3cn2/2012/curs10.pdf)
- Se consideră memoria cache din figura de mai jos doar ca exemplu fără ca vreun calcul să fie influențat de ea. Blocul de memorie cache trebuie să fie ->
  - ca blocul memoriei cache să fie egal decât lungimea unui cuvânt
  - **ca blocul memoriei cache să fie mai mare decât lungimea unui cuvânt**
  - ca blocul memoriei cache să fie mai mic decât lungimea unui cuvânt
  - nici un răspuns corect
  - ca blocul memoriei cache să fie diferit de lungimea unui cuvânt



Create PDF in your applications with the Pdfcrowd HTML to PDF API
PDFCROWD



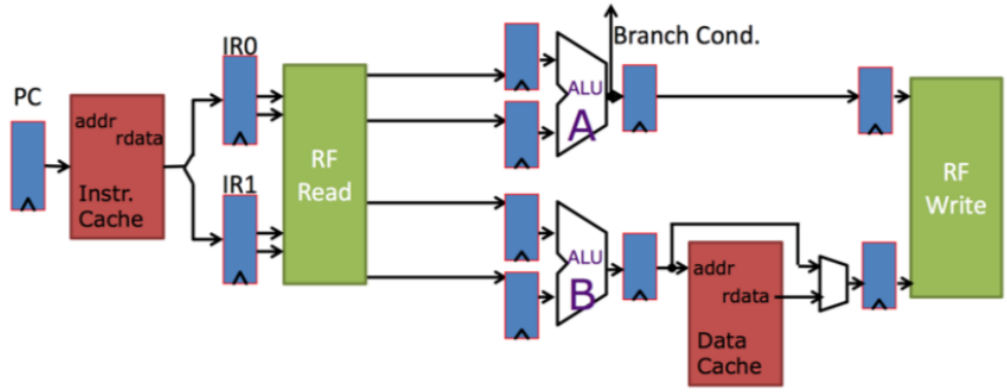
  - La apariția unui eșec: se vor extrage mai multe cuvinte adiacente
- In cazul unui procesor superscalar IO implementarea tehnicii de bypass presupune introducerea unor multiplexoare ca în figura de mai jos: Un mare dezavantaj al acestei implementări este creșterea dimensiunii multiplexoarelor odată cu creșterea dimensiunii BA-ului. Dacă ar fi utilizat un procesor OOO atunci



-
  - în banda de asamblare A nu executăm nimic
  - vom comite informația mai târziu în fișierul de registre
  - în banda de asamblare A se vor executa toate instrucțiunile programate spre execuție
  - nu se poate folosi un procesor OOO
  - vom comite informația mai devreme în fișierul de registre
- Coada Issue are rolul de:
  - de a reordona dependințele RAW, WAW, WAR (chatGPT)(+1)
  - de a mentine toate dependintele RAW, WAW, WAR(spune alta grila)
  - de a mentine intreruperile si exceptiile (spune alta grila)
- O modalitate de reprezentare a execuției instruncțiunilor într-o BA este cea prezentată mai jos. Această reprezentare este: Foarte utilă ajutând la determinarea perioadei de ceas necesară funcționării unui BA non-ideală (banuiala)

Timp	t1	t2	t3	t4	t5	t6	t7	t8	.....
Instr_1	IF_1	ID_1	EX_1	MEM_1	WB_1				
Instr_2		IF_2	ID_2	EX_2	MEM_2	WB_2			
Instr_3			IF_3	ID_3	EX_3	MEM_3	WB_3		
Instr_4				IF_4	ID_4	EX_4	MEM_4	WB_4	

-
- Cause este: registrul care ne spune de ce am luat în considerare excepția (chatGPT)(+1)
- Prin introducerea benzii de asamblare perioada ceasului : se reduce (chatGPT)
- Se considera procesorul din figura de mai jos:

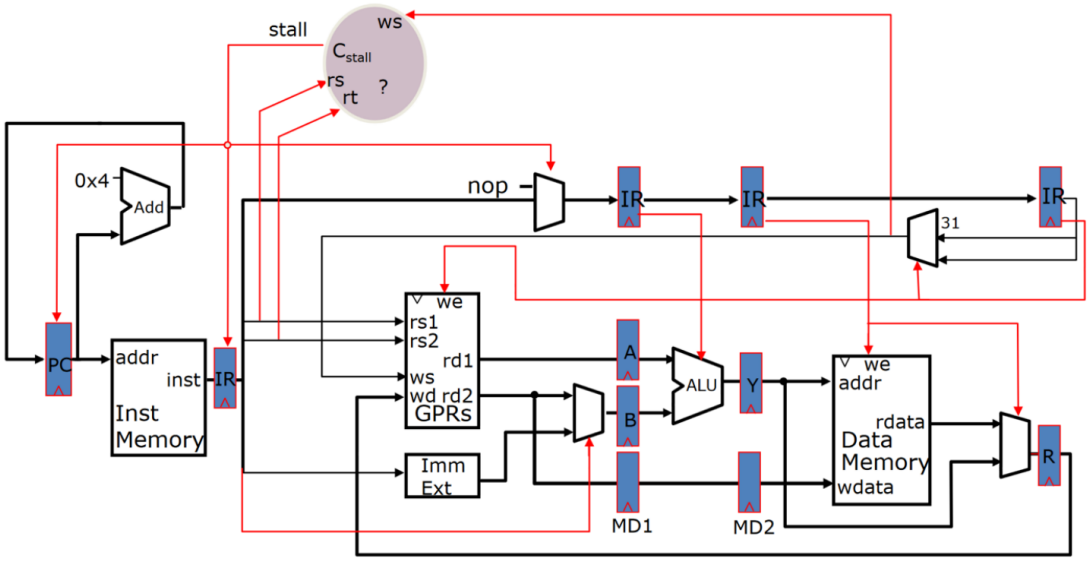


Se presupune ca executia programului de mai jos conduce la un hazard. Ce tip de hazard este si care este executia corecta a programului?

Instrucțiunea	T0	T1	T2	T3	T4	T5
ADDIU	IF	ID	EX_A	W		
LW	IF	ID	EX_B	W		
LW		IF	ID	EX_B	W	
ADDIU		IF	ID	EX_A	W	
LW			IF	ID	EX_A	W
LW			IF	ID	EX_B	W

→ este imposibila aparitia oricarui tip de hazard ca atare executia este corecta

- Conform implementării de mai jos a controlului se poate:

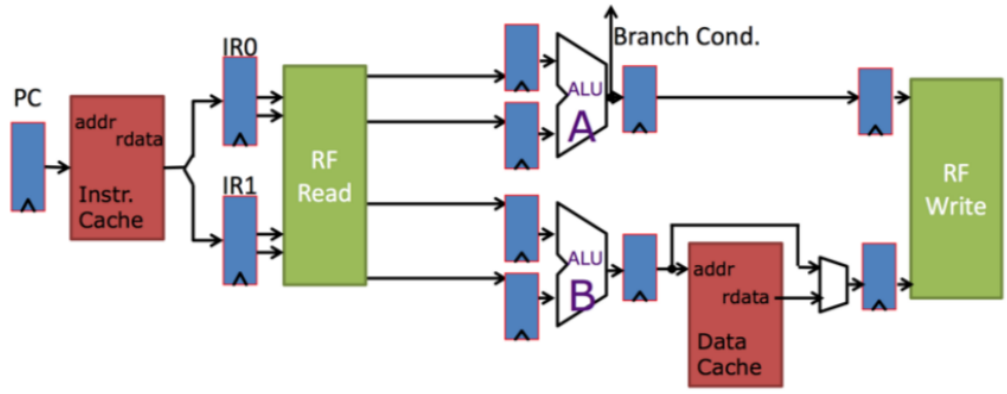


-
  - nu insera NOP-uri
  - nu putem insera NOP-uri în stagiile următoare (+1)
  - nici un răspuns corect (+1)
  - putem spune STALL oricărei instrucțiuni ce este faza IF
- Execuția programului de mai jos conduce la un hazard -> da, un hazard stuctural

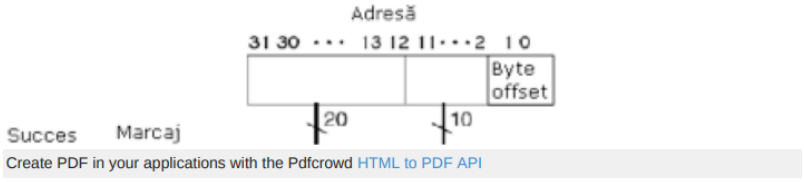
Instrucțiunea	T0	T1	T2	T3	T4	T5
ADDIU	IF	ID	EX_A	W		
LW	IF	ID	EX_B	W		
LW		IF	ID	EX_B	W	
ADDIU		IF	ID	EX_A	W	
LW			IF	ID	EX_A	W
LW			IF	ID	EX_B	W

-
- Definiție - Hazard de date - instrucțiunea planificată spre execuție : Instrucțiunea planificată spre execuție nu poate fi executată într-unul din ciclurile de ceas ale BA deoarece datele necesare execuției sale nu sunt încă disponibile.
- HOT SWAPPING presupune:
  - replasarea unui HDD cât timp sistemul este în stare de funcționare,
  - **replasarea unei componente hardware cât timp sistemul este în stare de funcționare (corect, curs)**
  - replasarea unei memorii cat timp sistemul este in stare de functionare(spune alta grila)(+1)

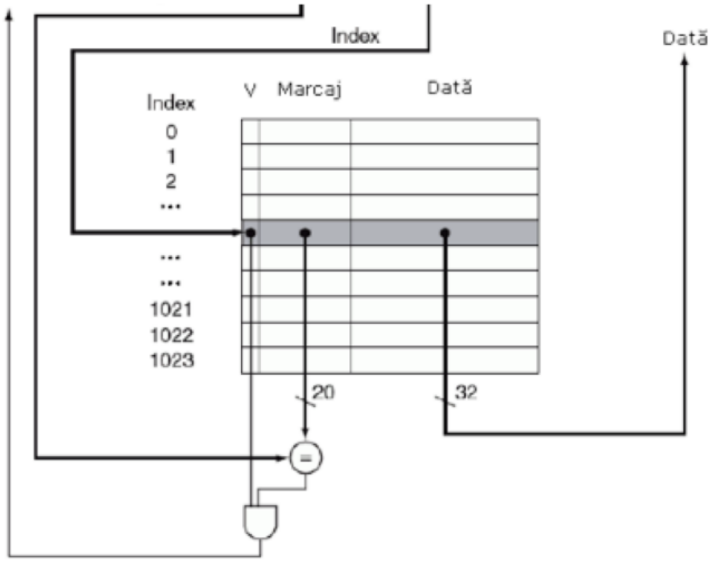
- În cazul procesorului prezentat mai jos, ce se întâmplă dacă în faza de ID prima instrucțiune este un LOAD:



-
    - nu se întâmplă nimic
    - nici un răspuns corect
    - eliminăm cumva instrucțiunea înainte de a o executa în BA
    - eliminarea instrucțiunii dar în faza de IF
    - **comutarea execuției pe BA-ul B**



Create PDF in your applications with the Pdfcrowd HTML to PDF API

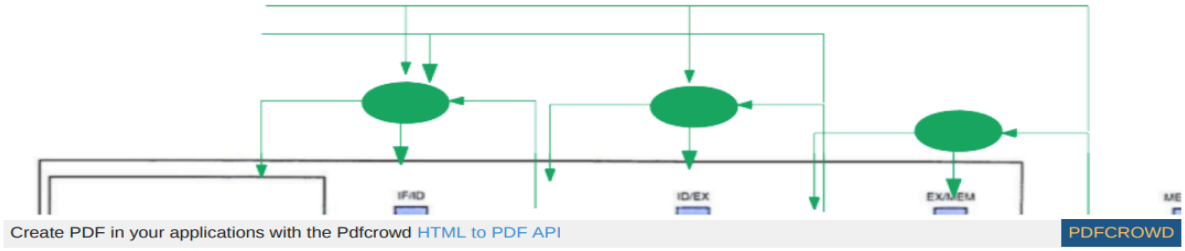


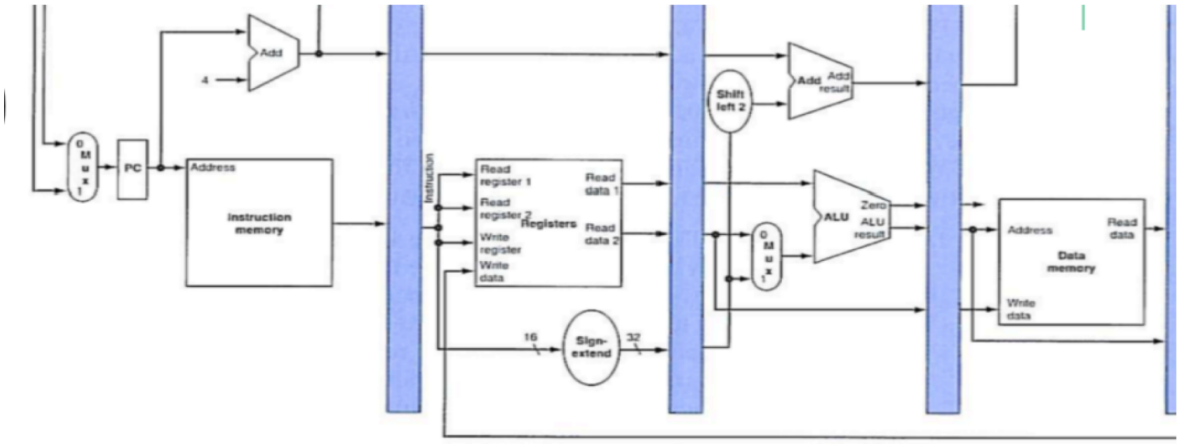
-
- Accesul la memoria cache se soldeaza cu un succes si cuvantul este trimis la procesor daca:
    - marcajul și cei 20 de biți superiori ai adresei corespund
    - bitul de validare este setat
- Marcajul din memoria cache este comparat cu porțiunea superioara a adresei pentru:
    - a determina dacă locația din memoria cache este ocupată
    - a determina numărul de biți necesar pentru index
    - a determina dacă locația din memoria cache este liberă
    - a determina dacă locația din memoria cache este diferită sau nu de adresa cerută
    - a determina numărul de biți necesar pentru byte-ul de offset
- Fiecare stagiu trebuie să : posede resurse hardware care sunt necesare execuției task-ului
- Hazardul de date dacă există se poate soluționa : planificare, stall, bypass, speculare
- Se consideră următorul program:
  DADDU R1, R2, R3
  BEQZ R4, L
  DSUBU R1, R5, R6 L: …..

OR R7, R1, R8
Instrucțiunea OR prezintă dependență:

- **de date pentru DADDU și DSUBU (corect, curs)**

- Se consideră următoarea implementare a unui procesor în BA cu reliefarea controlului (cu culoarea verde). Este această implementare optimă ?

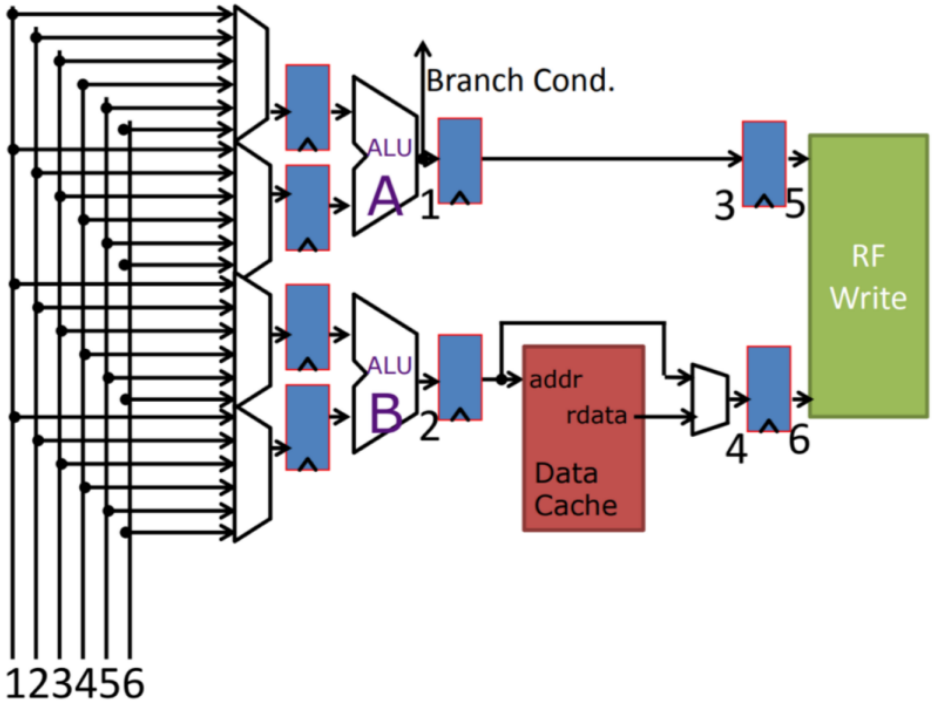




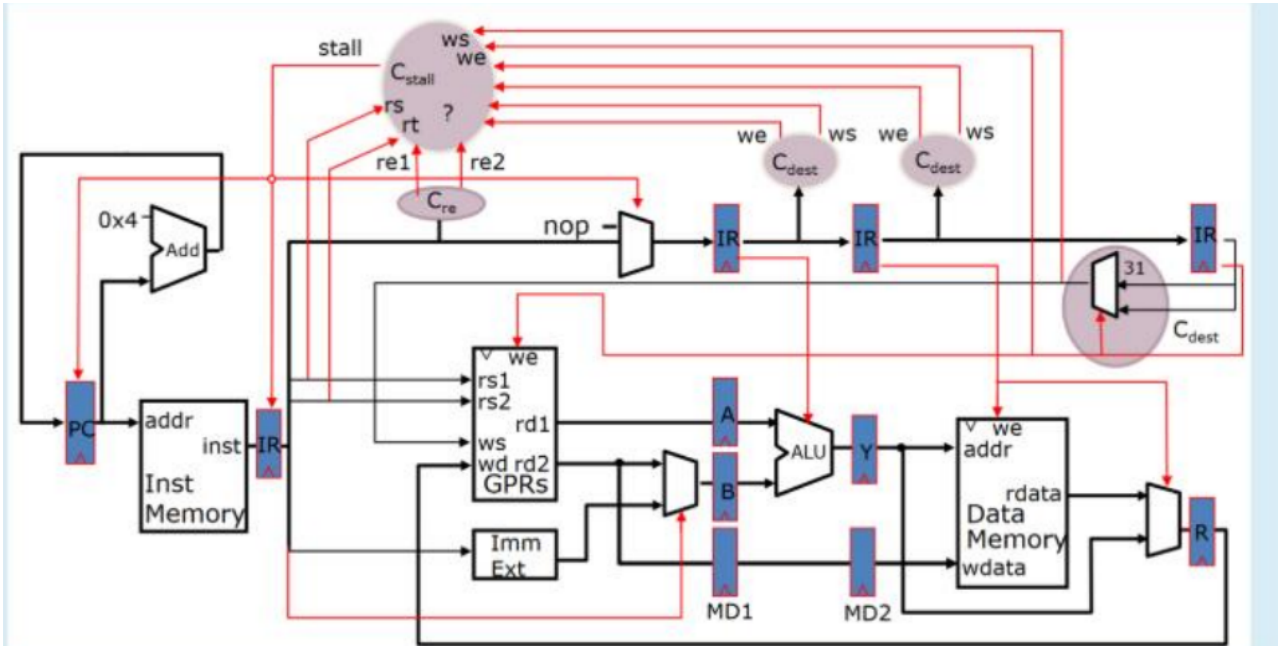
-
  - Implementarea este optimă deoarece putem introduce stall în oricare ciclu de ceas al benzii de asamblare
  - Implementarea este optimă deoarece stagiul
  - oferă informații către toate stagiile anterioare
  - **Implementarea nu este optimă**
  - **Implementarea nu este optima deoarece poate aparea deadlock**
  - Implementarea nu este optimă deoarece nu putem introduce stall în oricare ciclu de ceas al benzii de asamblare
  - Implementarea este optimă
- Punctul de comitere se poate pune doar în stagiul de BA:
  - în care este imposibil să mai apară vreo excepție
  - **în care este posibil să mai apară un hazard structural**
  - **în care este posibil să mai apară un hazard de date**
  - **în care este posibil să mai apară un hazard de control**
  - **în care este posibil să mai apară vreo excepție**
- Tratarea unei întreruperi presupune cel puțin execuția următorilor pași:
  - conținutul instrucțiunii curente se salvează într-un registru special EPC, (+1)
  - Utilizarea instrucțiunii RFE (o instrucțiune de salt indirect) pentru resume la cod (+2),
  - stergerea continutului registrului EPC pentru a permite intreruperi in intreruperi,
  - citirea unui registru status care ne indica cauza intreruperii, cel putin un pas este gresit
  - salvarea continutului registrului EPC pentru a permite intreruperi in intreruperi,
  - scrierea unui registru status care ne indica cauza intreruperii
- Execuția instrucțiunilor nu prezintă hazard și totuși în t2 există un NOP. Dacă am elimina NOP-ul există posibilitatea apariției unui hazard și dacă da ce tip are ? de date dacă avem LOAD

	t0	t1	t2	t3	t4	t5	t6
r1 = (r0)+10	IF_1	ID_1	EX_1	MEM_1	WB_1		
r3 = (r2)+17		IF_2	nop	ID_2	EX_2	MEM_2	WB_2

- În cazul unui procesor superscalar IO implementarea tehnicii de bypass presupune introducerea unor multiplexoare ca în figura de mai jos: Un mare dezavantaj al acestei implementări este creșterea dimensiunii multiplexoarelor odată cu creșterea dimensiunii BA-ului. Pentru eliminarea acestui dezavantaj putem:
  - să mărim dimensiunea fișierului de registre
  - să mărim dimensiunea memoriei
  - să folosim clustere de procesoare superscalare (+3)
  - să eliminăm stagiul ID
  - să eliminăm stagiul W
  - sa folosim procesoare out-of-order
  - putem elimina acest dezavantaj doar daca folosim procesoare OOO
  - acest dezavantaj nu constituie un impediment in conditiile tehnologice actuale(spune o grila)



- Avand in vedere implementarea de mai jos, de ce MUX-ul din WB este incercuit?

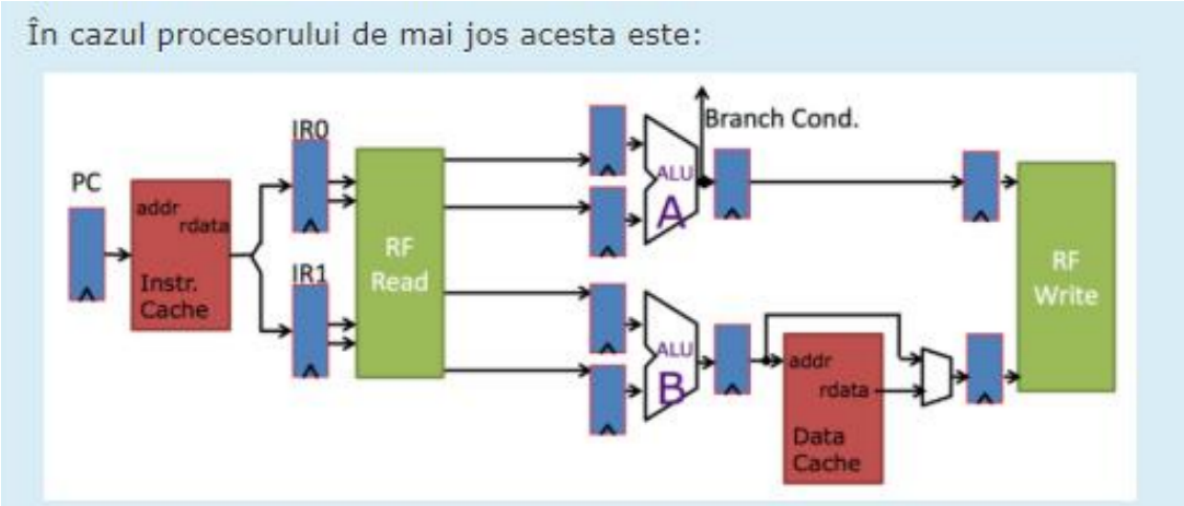


- Soluția finală ignorând instrucțiunile de jump și branch. (curs)
- Acest multiplexor poate fi văzut ca un registru către o destinație implicită
- Identificatorul de registru se calculează pe baza lui IR
- MUX-ul este foarte necesar deoarece instrucțiunile JUMP și LINK doar scriu nu și citesc
- nici un răspuns corect
- MUX-ul este foarte necesar deoarece instrucțiunile JUMP și LINK doar citesc nu și scriu
-

- Presupunem că instrucțiunea i precede instrucțiunea j. O antidependență între i și j apare când: instrucțiunea j scrie un registru sau o locație de memorie citită de instrucțiunea i
  instructiunea j sterge continutul unui registru citit de instrucțiunea i(spune alta grila)
- instrucțiunea i scrie un registru sau o locație de memorie citită de instrucțiunea j(spune alta grila)

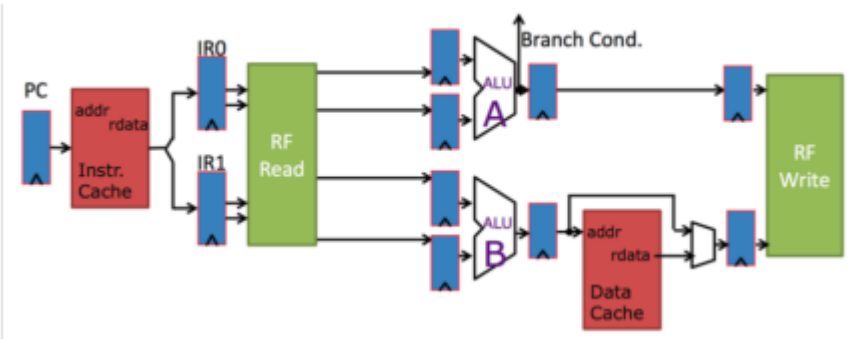
- Cum rezolvăm situația în care avem întreruperi simultane multiple în diferite stagii de pipe în cazul unui procesor superscalar IO ? -> instrucțiunea cea mai veche care a generat o excepție va face KILL pentru toate instrucțiunile din spate aflate în banda de asamblare

În cazul procesorului de mai jos acesta este:



- superscalar pe 2 cai IO

- Unul din MISS-urile posibile într-o memorie cache este cel compulsory(obligatorii) - prima referință a unui bloc. Acest tip de MISS apare și în cazul:
  - **unei memorii cache cu mapare directă**
- Memoria se prezintă ca: o ierarhie cu următoarele nivele: memorie cache, memoria principală, HD, CD/DVD etc.
- Definiție - Hazardul de control - Este necesar să luăm o decizie bazată pe rezultatul unei instrucțiuni cât timp alte instrucțiuni sunt în așteptare : definiția este incorectă - instrucțiunile trebuie să fie în execuție
- Când PTE-ul corespunzător unei pagini virtuale conține bitul de validitate setat pe : 0 avem page fault, PTE-ul corespunzător unei pagini virtuale are mai mult de 2 biți
- Între stagii trebuie să existe : registre
- O bandă de asamblare cu mai multe stagii este : de preferat, de evitat, de evitat deoarece cresc hazardele structurale
- În cazul procesorului de mai jos: banda de asamblare B poate executa operații pe întregi și accese la memorie (+2), Banda de asamblare A poate executa operații pe întregi și salturi (+2) , nu putem executa accese la memorie în BA-ul A (+1), nu putem executa salturi în BA-ul B (+1)

- 

- În cazul lui RAID 1: întotdeauna vom avea 2 copii ale informației, când o dată este scrisă pe un disk, automat datele sunt scrise pe un alt disk redundant (corect, curs)
- Banda de asamblare trebuie : golită și starea procesului salvată pentru a putea fi reluată (curs)
  sa functioneze in continuare neexistand pos de a salva starea procesorului(spune alta grila)
- Se consideră execuția programului de mai jos în cazul unui procesor superscalar IO care implementează tehnica de bypass.

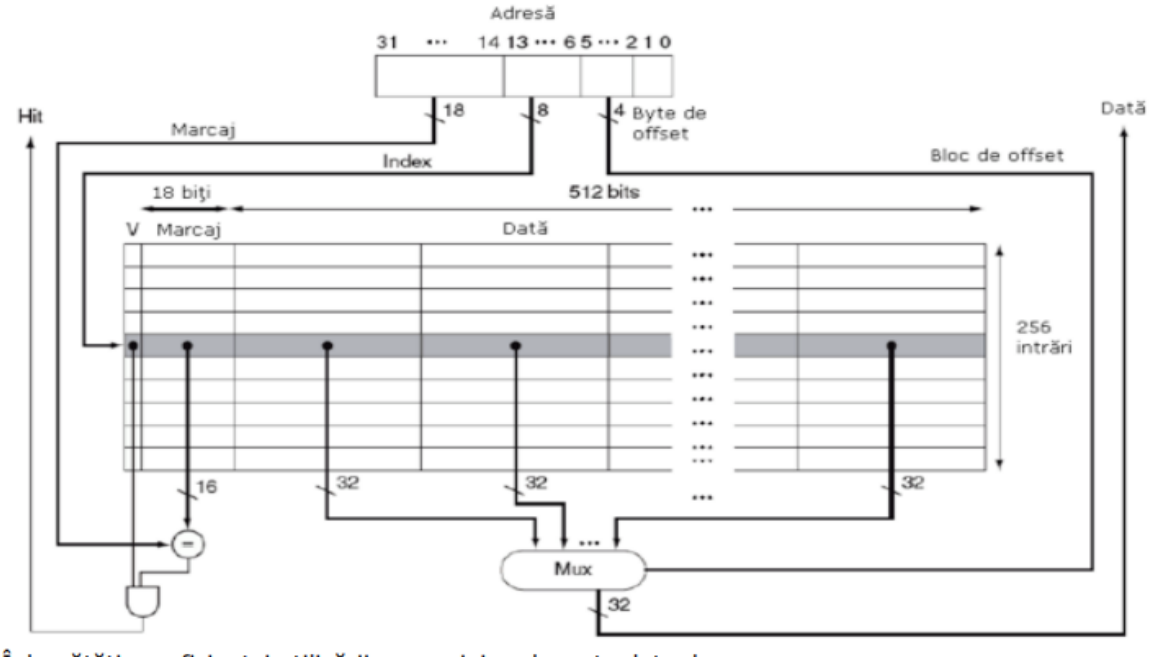
BEQZ	F	D	I	Ex_A	W				
I1	F	D	I	Ex_B	-				
I2		F	D	I	-	-			
I3		F	D	I	-	-			
I4			F	D	-	-	-		
I5			F	D	-	-	-		
I6				F	-	-	-	-	
I7				F	-	-	-	-	
I8					F	D	I	Ex_A	W
I9					F	D	I	Ex_B	W

  - 7 și vom folosi predictori pentru a determina dacă instrucțiunea se va executa sau nu
  - 5 și vom folosi predictori pentru a determina dacă instrucțiunea se va executa sau nu
- Tehnica de SPECULARE ne permite un impact minim asupra
  - dependenței de control cât timp fluxul de date se păstrează (+1)
  - cazurilor în care avem antidependență (am gasit in alta grila)

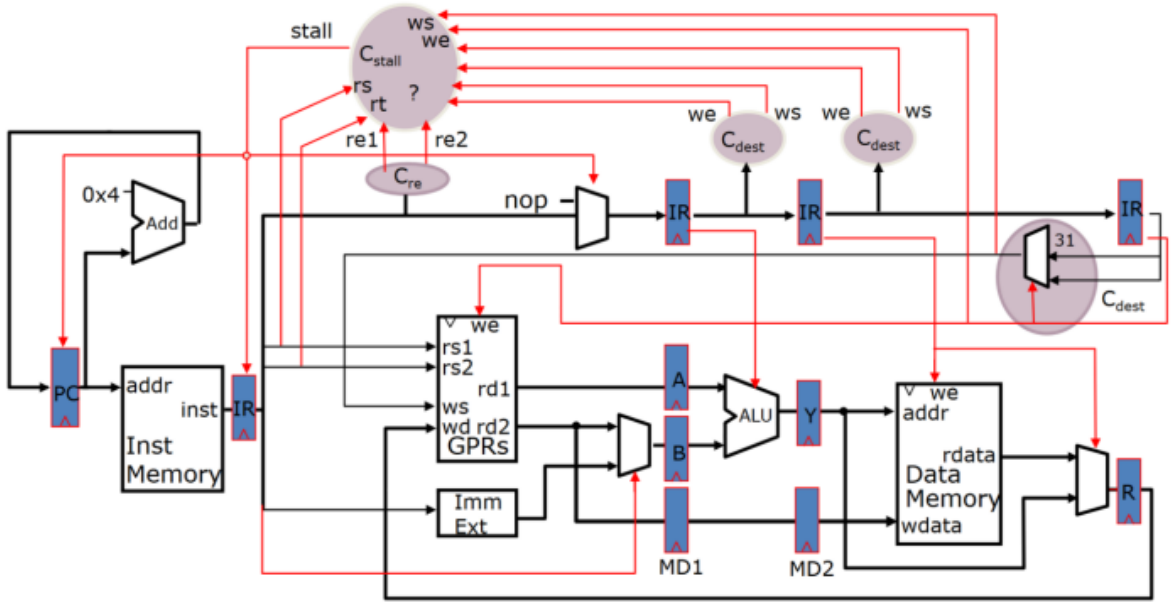
- Folosirea tehnicii denumita SPECULARE presupune:
  - ignorarea exceptiei aparuta atunci cand se executa o instructiune branch

- Figura de mai jos prezintă o memorie cache ce conține 64KB de date, cu blocuri de 4 cuvinte (16 octeți) fiecare.

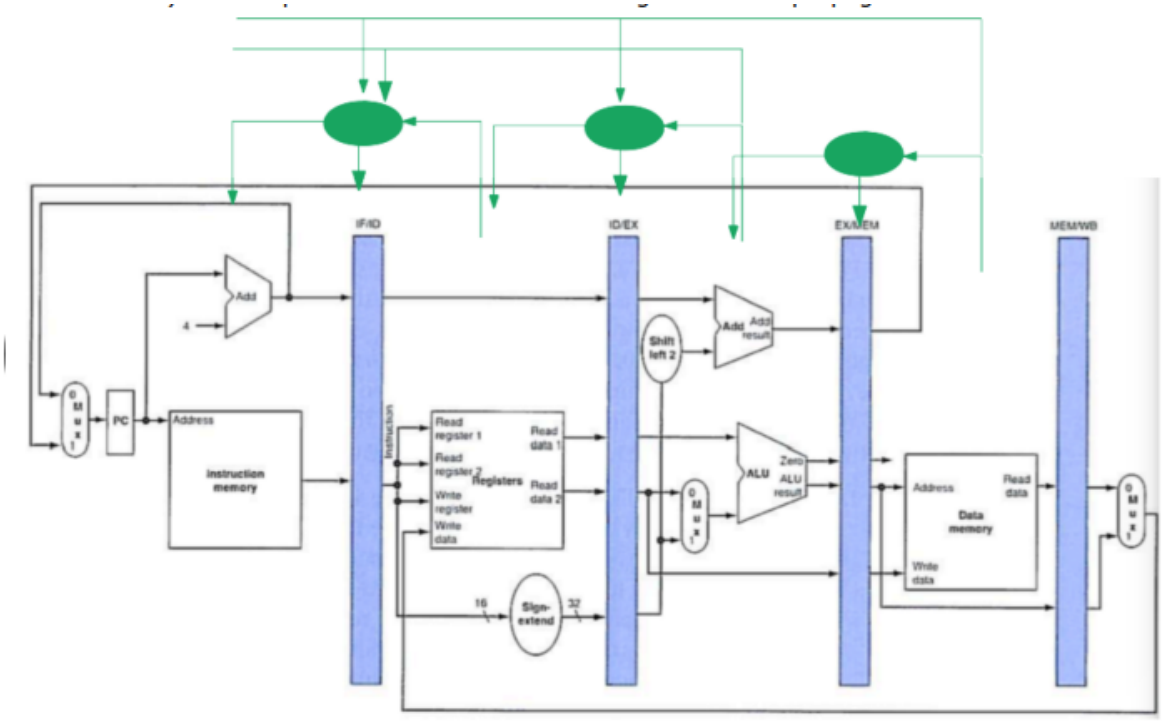
octeți) fiecare.



-
  - folosirea în comun a marcajelor dar nu și a biților de validitate pentru mai multe cuvinte
  - folosirea în comun a marcajelor dar nu și a biților de validitate pentru cuv care va fi selectat(spune alta grila)
- Având în vedere implementarea de mai jos, când se termină execuția instrucțiunii LOAD ?

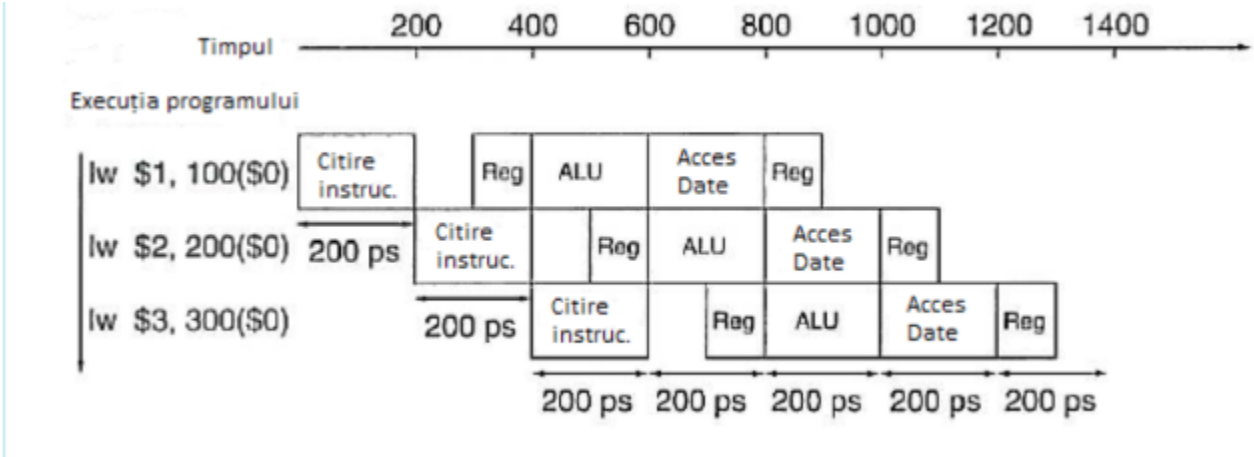


-
  - rezultatul instrucțiunii este în registrul R
- Notiunea de MISS presupune:
  - nici un raspuns corect
  - neregasirea datelor pe nivelul superior(curs)
- O metodă de îmbunătățire a performanței memoriilor cache presupune reducerea ratei de MISS dacă:
  - se crește asociativitatea memoriei cache
- Se considera urmatoarea implementare a unui procesor in BA cu reliefarea controlului (cu culoarea verde). Este important ca feedback-ul din stagiul “i” sa se propage catre:



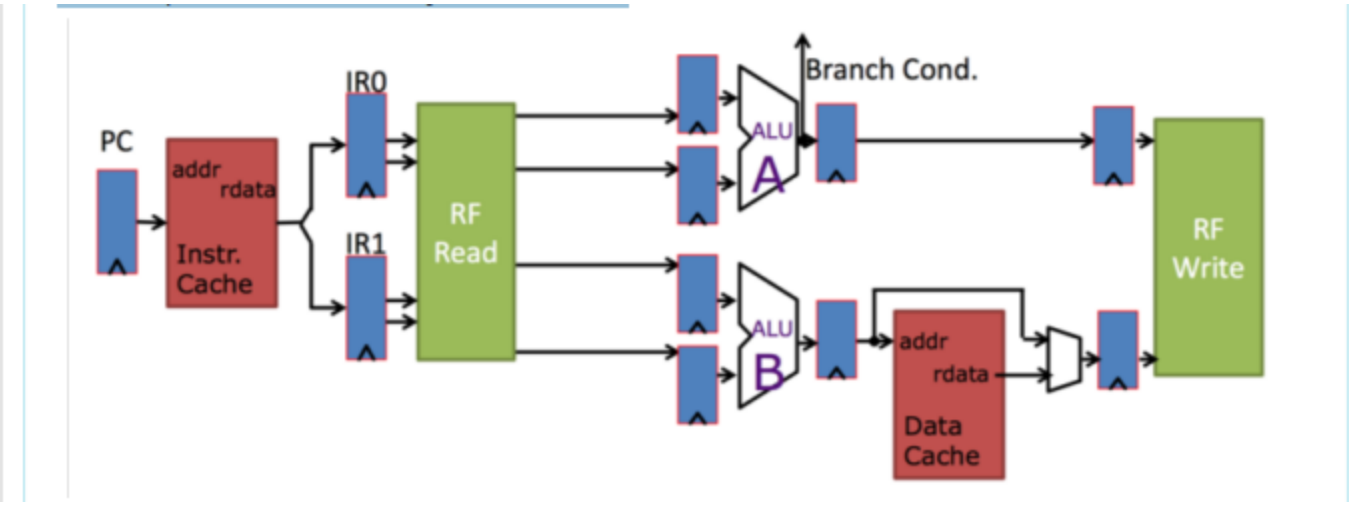
-
- către stagiile 1 ... i-1

- O instrucțiune j prezintă o dependență de date față de instrucțiunea i dacă
  - Instrucțiunea j prezintă dependențe de date față de instrucțiunea k, iar instrucțiunea k prezintă dependențe de date față de instrucțiunea i
  - Instrucțiunea i produce un rezultat care poate fi utilizat de instrucțiunea j
- Definitie - Hazard structural - o instriunce planificata:
  - nu poate fi executata in propriul ciclu de ceas deoarece HW ul nu suporta combinatia de instructiuni
- Avem o memorie cache având 4 blocuri de câte 1 cuvânt. Memoria cache este cu asociativitate pe 2 căi și pentru replasare se folosește algoritmul LRU. Să se găsească numărul de MISS-uri pentru următoarea secvență de adrese de bloc: 0,8, 0, 6, 8
  - 4
- Se consideră următorul program executat într-o bandă de asamblare. În cazul în care există un hazard structural să se specifice ciclul de ceas în care apare.



  - Există un hazard structural în ciclul 4

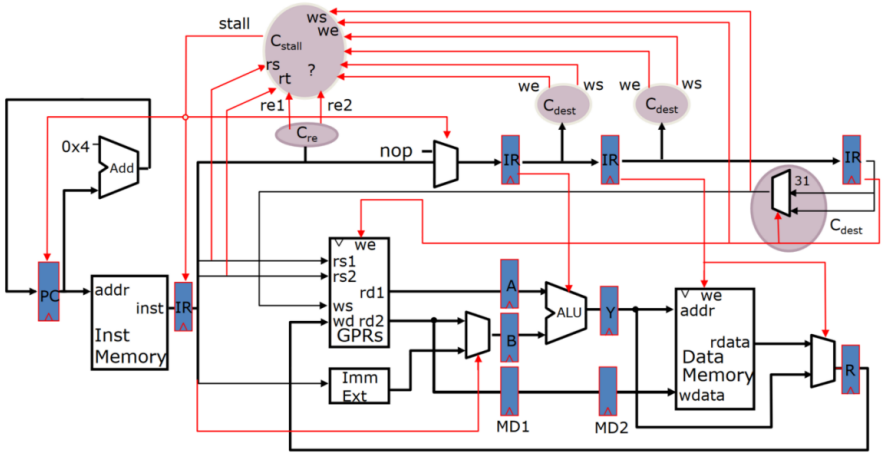
- În cazul procesorului de mai jos acesta este:



  - superscalar pe 4 căi IO

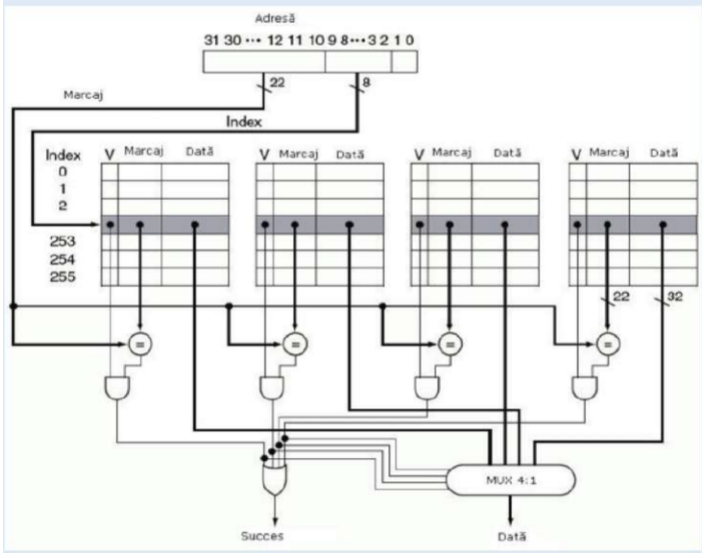
- RAID 1 este ales când:
  - **toleranța la defecte este un punct critic (corect, curs)**
  - **in cazul lui RAID 1 nr de hard disk-uri utilizat este dublu fata de RAID 0 (corect, curs)**

- În cazul implementării de mai jos, semnalul RE este necesar deoarece:



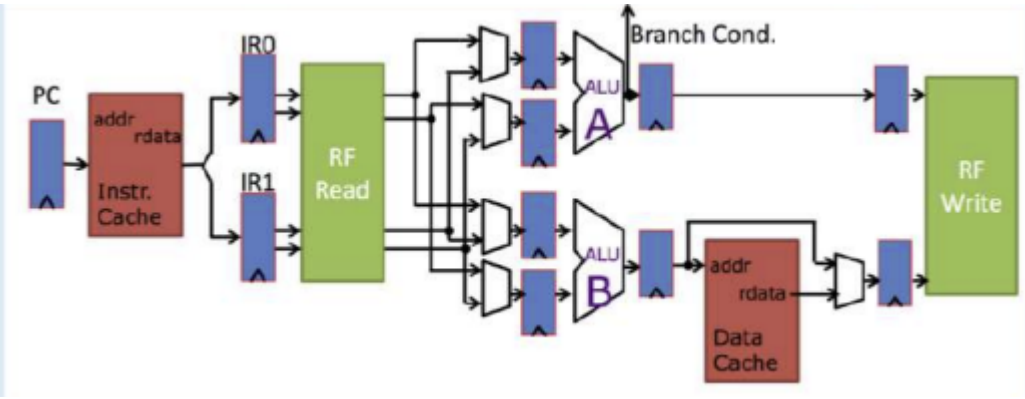
  - acest semnal este necesar deoarece nu orice instrucțiune citește un registru
  - exista instructiuni in MIPS care citesc doar un singur operand sursa(spune alta grila)

- Se considera memoria cache prezentată în figura de mai jos.



Fiecare bloc include un marcaj de adresa ce indica
    - adresa 2 * Marcaj a blocului

- Se considera procesorul din figura de mai jos:

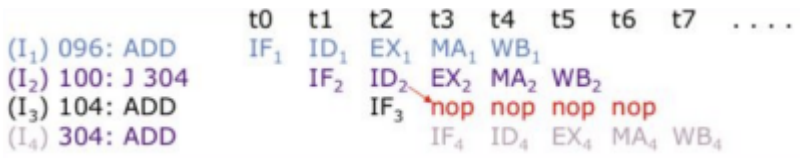


În cazul execuției programul de mai jos este posibilă apariția unui hazard și dacă da de ce?

Instrucțiunea	T0	T1	T2	T3	T4	T5
ADDIU	IF	ID	EX_A	W		
LW	IF	ID	EX_B	W		
LW		IF	ID	EX_B	W	
ADDIU		IF	ID	EX_A	W	
LW			IF	ID	EX_A	W
LW			IF	ID	EX_B	W

    - 1. este posibila aparitia until hazard deoarece nu se pot executa 2 instructiuni ADDIU si LW simultan
    - 2. este posibilă apariția unui hazard deoarece nu se pot executa 2 instrucțiuni de ADDIU în același moment de timp
    - 3. este posibilă apariția unui hazard deoarece nu se pot executa 2 instrucțiuni de LOAD în același moment de timp
    - da, un hazard structural (de la alta intrebare)

- Se considera execuția programului de mai jos în banda de asamblare. Se presupune ca speculam valoarea PC+4, instrucțiunea 2 nu face STALL deoarece avem speculații dar de ce instrucțiunea 3 introduce doar fetch?

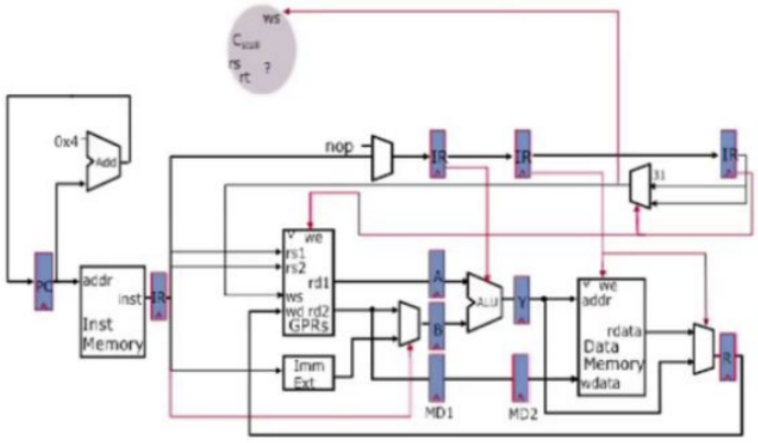


    - **a. Conform enuntului există speculație PC+4, ca atare, după instrucțiunea 2 trebuie să înceapă instrucțiunea 3. Speculația fiind greșită, instrucțiunea 3 trebuie eliminată din banda de asamblare.**

- b. Conform enuntului există speculație PC+4, ca atare, după instrucțiunea 2 trebuie să înceapă instrucţiunea 3. Cu toate acestea, execuția instrucţiunii 3 ar conduce la un hazard structural, ca atare va trebui eliminată din banda de asamblare
- Conform enuntului există speculație atare, după instrucțiunea 2 trebuie să înceapă instrucțiunea 3. Cu toate acestea, execuția instrucţiunii 3 ar conduce la un hazard de control, ca atare va trebui eliminată din banda de asamblare
- d. Programul nu poate fi executat în bandă de asamblare deoarece prezintă instrucţiuni ce nu pot fi executate în bandă de asamblare
- e. Conform enuntului există speculație PC.4, ca atare, după instrucţiunea 2 trebuie să înceapă instrucţiunea 3. Cu toate acestea, execuția instrucțiunii 3 ar conduce la un hazard de date, ca atare va trebui eliminată din banda de asamblare

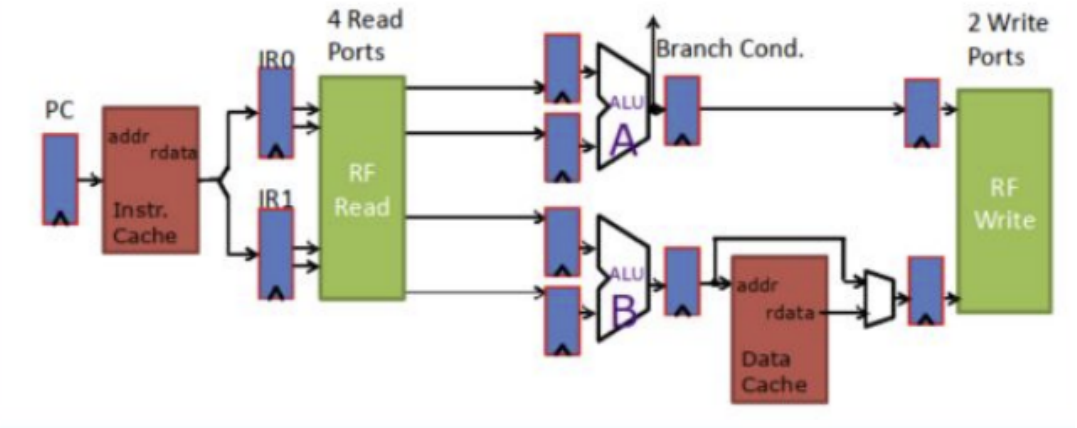
- Când poate apărea o anti-dependență ?
  - a. O anti-dependență nu este posibil să apară
  - **b. O anti-dependență poate apărea atunci când instrucțiunile nu sunt executate în ordine**
  - **c. O anti-dependență poate apărea atunci când mai multe instruciuni sunt executate în același moment de timp**
  - d. O anti-dependență poate apărea atunci avem un hazard structural
  - e. O anti-dependenţă poate apărea doar în faza de IF

- Se considera următoarea implementare a unui procesor pentru determinarea semnalului C_STALL. Ce trebuie verificat în acest caz? (Fără răspunsuri)



  - a. Se verifica idenficatorul registrului
  - b. Practic se verifica câmpul RS din cadrul unei instrucţiuni MIPS
  - c. Se verifica valoarea registrului
  - **d. Practic se verifica câmpul RD din cadrul unei instrucţiuni MIPS**
  - e. Practic se verifica câmpul immediate16 din cadrul unei instrucțiuni MIPS
  - f. Practic se verifica campul OPCODE din cadrul unei instrucţiuni MIPS
  - g. Practic se verIfica câmpul immediate26 din cadrul unei instrucțiuni MIPS
  - h. Practic se verifica câmpul RT din cadrul unei instrucțiuni MIPS

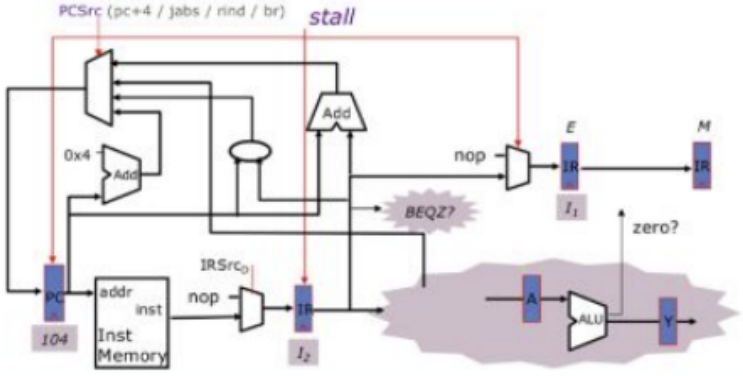
- În cazul unui procesor superscalar 10 cu 2 căi ca în figura de mai jos sunt planificate spre execuţie 2 instrucțiuni. 0 instructiune LW 1n registrul IRO şi o instrucţiune ADD în registrul IR1. Ce mai trebuie adăugat ca să poată fi posibilă execuția celor 2 instrucțiuni



- a. O logica suplimentară care să permita duplicarea operandului/operanzilor uneia sau mai multor instrucțiuni
- **b. Folosirea a două decodificatoare**
- **c. O logică suplimentară care să permită switch între instrucțiuni**
- **d. Duplicarea logicii de control**

- Executia urmatorului program in banda de asamblare presupune eliminarea instructiunilor 104 si 108. Pentru aceasta este nevoie:

I1 – 096 ADD
I2 – 100 ***BEQZ*** r1 + 200
I3 – 104 ADD
108 .....
I4 – 304 ADD



- **a. Introducerea unui MUX suplimentar**
- b. Introducerea de noi registre
- **c. Introducerea de NOP-uri**
- d.Introducerea unei memorii de instrucțiuni
- **e. Schimbată condiția pentru IRSrc**
- f. Introducerea unei memorii de date

- Se considera următoarea execuție a unui program în banda de asamblare. De ce este nevoie de introducerea NOP-urdor in executia programului cu toate ca programul nu prezintă hazard-uri?

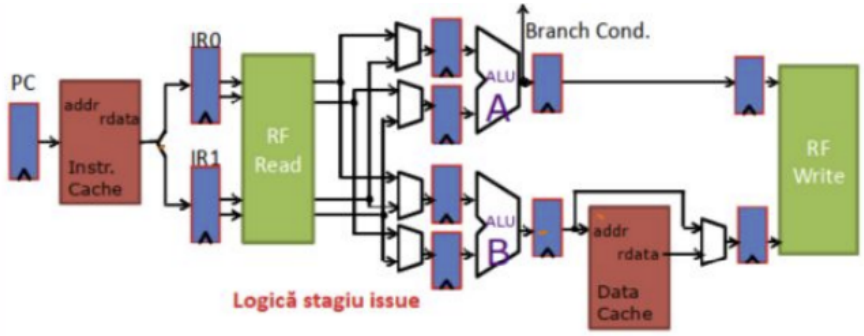
	t0	t1	t2	t3	t4	t5	t6	t7	. . . .
(I_1) r1 ← (r0) + 10	IF_1	ID_1	EX_1	MA_1	WB_1				
(I_2) r3 ← (r2) + 17		IF_2	IF_2	ID_2	EX_2	MA_2	WB_2		
(I_3)				IF_3	IF_3	ID_3	EX_3	MA_3	WB_3
(I_4)						IF_4	IF_4	ID_4	EX_4 MA_4 WB_4

- **a. Se evită hazardul de control deoarece trebuie determinată instrucțiunea 2**
- b. Programul nu prezinta hazarduri şi ca atare nu trebuiesc puse NOP-uri
- c. Se ffinta hazardul de date deoarece trebuie determinată instrucțiunea 2

- d. Nu trebuie puse NOP-uri
- e. Se evita hazardul structural deoarece trebuie determinată instrucțiunea 2

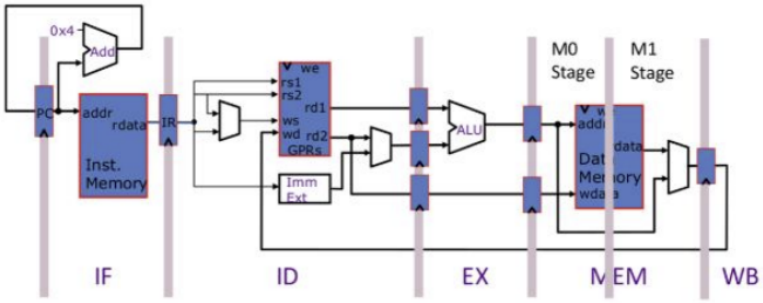
- O dependentă de ieșire cand poate apărea?
  - a. O dependenţa de ieşire poate apărea doar în faza de IF
  - b. O dependența de ieșire poate apărea doar în faza de ID
  - c. O dependenţa de iesire nu erte considerata un hazard
  - d. O dependenţa de ieşire poate apărea în cazul execuţiei a 5 sau mai multe operate O
  - **e. O dependență de ieşire poate apărea în carvl execuției a 2 operații care necesită toți ciclii de ceas**

- În cazul unui procesor superscalar IO cu 2 cai ce contine faza de FETCH? (Fără răspunsuri)



  - **a. în faza de FETCH vom avea PC şi PC4+4**
  - b. în faza de FETCH vom avea decodificarea instructii
  - c. în faza de FETCH vom avea determinarea instrucţii
  - **d. în faza de FETCH vom avea doba instrucţiuni simultan**
  - e. in faza de FETCH vom avea eliminarea instrucțiunilor JAL și JALR

- Se considera următorul procesor şi execuţia următorului program ADD, LW, LW, ADD. ExisM hatard structural pe primele 2 instrucţiuni la M0 si M1? Dar pe ultimele doua instrucțiuni la M0 şi M1?

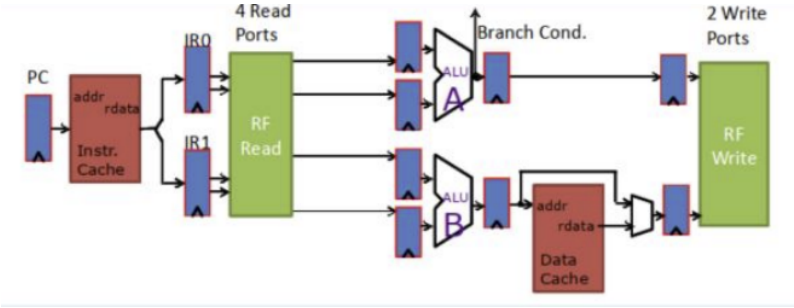


  - **a. Nu există hazard structural pe ultimele 2 instrucțiuni la MO şi M1**
  - b. Există hazard structural pe ultimele 2 instrucțiuni la MO şi M1
  - c. Ar fi hazard structural clacă instrucțiunile ar fi identice
  - d. Există hazard structural pe primele 2 instrucțiuni la MO şi M1
  - **e. Nu există hazard structural pe primele 2 instrucțiuni la MO şi M1**

- În cazul unei benzi de asamblare ideale:
  - -> Ca toate obiectele să treacă prin aceleași stagii (curs)
  - -> Între oricare 2 stagii nu este permisă partajarea resurselor (curs)
  - -> Întârzierea de propagare prin toate stagiile este aceeași (curs)
  - -> Tranziția de intrare în banda de asamblare nu este afectată de tranziția între alte stagii (curs)
  - a. Putem partaja resume fără restricții între stagii
  - b. Ordinea stagiilor poate fi modificată

- c. Putem partaja resurse ou restricții între stagii
- **d. Nu putem partaja resurse între stagii**
- **e. Ordinea stagiilor nu poate fi modificată**

- În cazul unui superscalar IO pe 2 cai registrul RF are nevoie de 4 porturi de intrare, așa cum se poate observa în figura de mai jos. De ce?

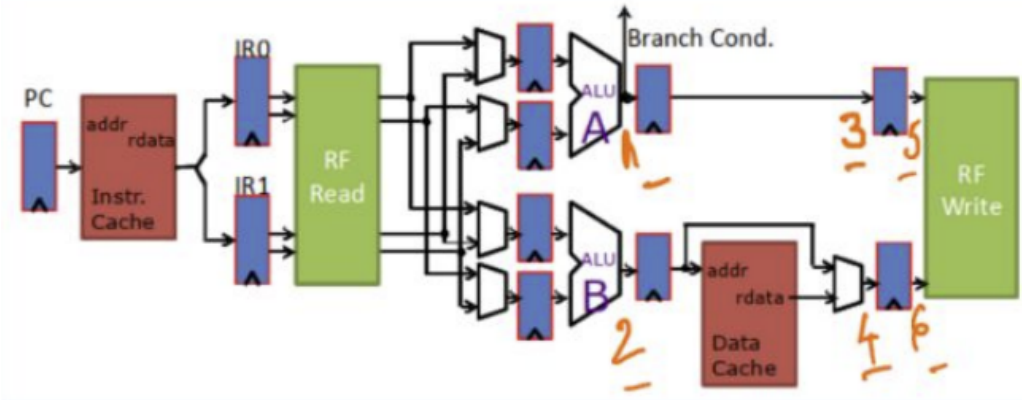


- a. Deoarece într-un astfel de procesor citirea se poate selectiv
- b. Deoarece conținutul registrelor IR nu pot fi simultan oferite la ieşire ci doar la ciclii de ceas consecutivi
- c. Deoarece un astfel de procesor poate executa doar operații cu întregi şi salturi
- d. Deoarece registrul de scriere RF are 2 porturi iar 4 este putere a lui 2
- **e. Deoarece numărul de instrucțiuni ce poate fi executat este dublu**

- ISA este un layer abstract
  - **a. menit a se schimba / modifica nu foarte des**
  - b. menit a se schimba / modifica foarte des
  - c. noțiune neprezentata la curs
  - **d. oferit software-ului**
  - e. oferit hardware-ului

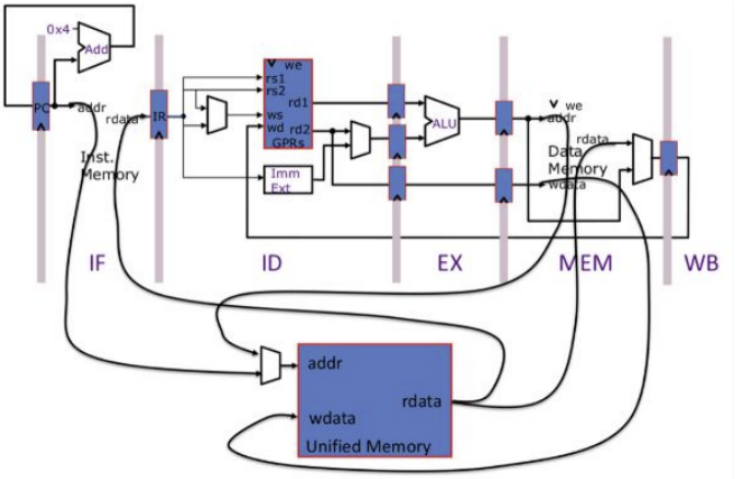
- O **intrerupere** sau o excepţie este denumită **precisa** prin definitie daca: Oprește programul curent la instrucțiunea I(i), terminând complet execuția tuturor instrucțiunilor până la I(i-1) – întrerupere precisă (curs)
  - a. apare doar în cadrul unui procesor OOO
  - **b. terminarea execuţiei unei instrucțiuni se face înainte de a începe execuția următoarei instrucțiuni**
  - c. execuția unei instrucțiuni se face în paralel cu execuţia următoarei instrucțiuni
  - **d. starea procesorului salvat corespunde modelului secvenţial de execuţie a programului**
  - e. apariția unei întreruperi imprecise nu este posibilă

- În cadrul unui procesorului de mai jos, a fost stabilit la curs cd putem introduce logica pentru bypass în 6 puncte conform figurii de mai jos. Logica selectarii acestor puncte crește odată cu mărirea dimensiunilor benzilor de asamblare. Cum eliminăm acest inconvenient ?



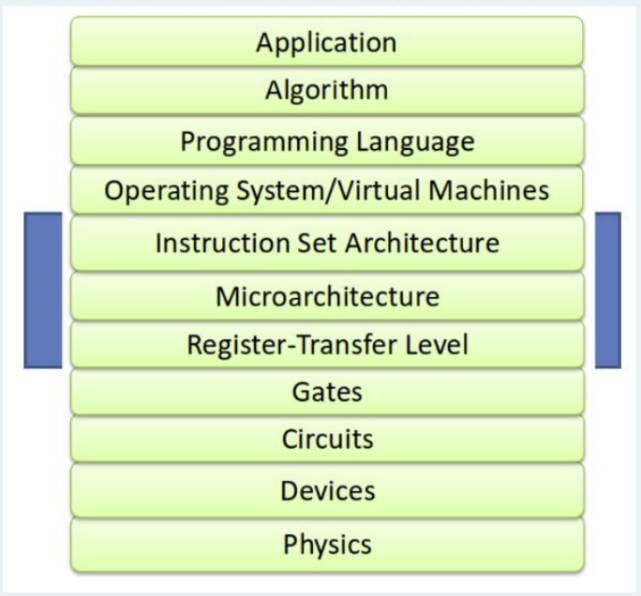
- **a. Avem în vedere o execuție a instrucțiunilor într-o maniera OOO**
- **b. Nu aplicam full bypass**
- **c. Eliminarea unor anumite resurse hardware**
- **d. Implementăm clustere superscalare**
- **e. Facem bypass doar la anumite locații**

- Se consideră un procesor implementat conform figurii de mai jos şi execuția următoarelor instrucțiuni LW, ADD, ADD, ADD. În execuția programului trebuie introdus un stall? Dacă da, unde ?



- **a. Trebuie introdus stall**
- b. Dacă avem stall atunci stall trebuie făcut pe a doua instrucţiune ADD
- c. Dacă avem stall atunci stall trebuie făcut pe prima instrucțiune ADD
- **d. Dacă avem stall atunci stall trebuie făcut pe ultima instrucțiune ADD**
- e. Nu contează introducerea de stall deoarece este un hazard structural si el nu poate fi evitat
- f. Nu trebuie introdus stall

- Se consideră straturile de abstractizare din figura de mai jos. Microarhitectura se referă la:

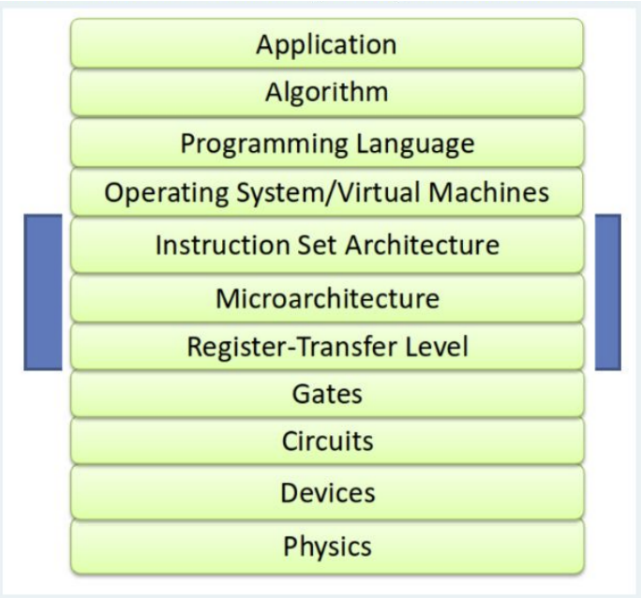


  - a. modalitatea de construire a unui chip cât mai general care să acopere toate necesitățile unui utilizator
  - b. organizarea sistemelor de calcul strict orientată pe consumul de putere
  - c. noţiunea nu a fost prezentată la curs
  - d. organizarea sistemelor de calcul
  - **e. modalitatea de construire a unui chip specific**

- În cazul în care dorim să facem bypass între clustere superscalare vom avea nevoie de: (Fără răspunsuri)
  - a. 3 extra cicluri de ceas
  - b. 2 extra cicluri de ceas
  - c. nu putem face această operație deoarece se va genera un hazard structural
  - d. 4 extra cicluri de ceas
  - **e. un extra ciclu de ceas**

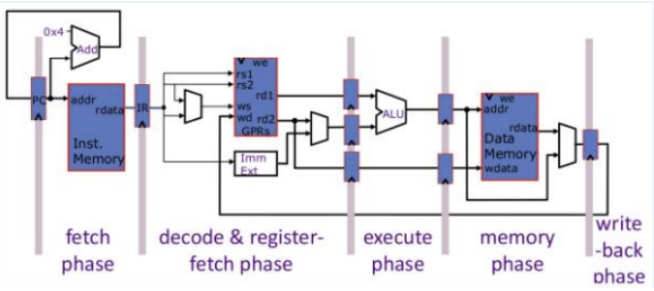
- La început fiecare procesor avea (până în 1960):
  - **a. propriul său set de instrucţiuni**
  - b. un sistem de memorie cache organizat pe nivele pentru obţinerea unei performanțe mărite
  - c. un sistem de memorie cache organizat pe minim două nivele pentru obţinerea unei performanțe mărite
  - **d. propriul său sistem de I/O și storage secundar**
  - **e. propriile sale asambloare, compilatoare şi librării**

- Conform figurii de mai jos există un layer denumit "Register-Transfer Level". Acest nivel permite descrierea unui circuit/dispozitiv electronic prin:



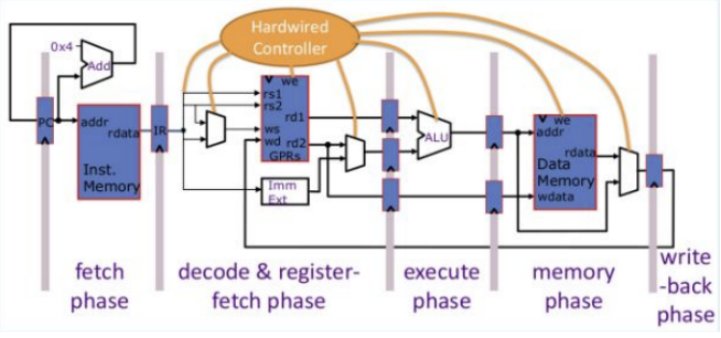
- **a. intermediul limbajului VHDL**
- b. prin intermediul limbajului C++
- **c. intermediul limbajului Verilog**
- d. prin intermediul limbajului Python

- Se consideră calea de date MIPS prevăzută cu bandă de asamblare. Pentru fiecare stagiu al benzii de asamblare se consideră câte un timp de execuție astfel: t_IM, t_RF, t_ALU, t_DM, t_RW. Frecvența ceasului care va controla banda de asamblare cel mai probabil va fi oferită de:



- a. t_RW
- b. t_ALU
- **c. t_DM**
- d. t_IM
- e. t_RF

- În cazul căii de date a unui procesor MIPS cu bandă de asamblare una din posibilitățile de implementare ale controlului se prezintă în figura de mai jos. Această implementare conduce la următoarele concluzii:



- **a. nu este o implementare bună deoarece unitatea de control nu este proiectată într-o manieră de tip bandă de asamblare**
- b. nu este necesară introducerea de registre pentru a fi capabili să executăm o instrucțiune în mai mulți ciclii de ceas
- **c. instrucțiunile nu pot fi executate în bandă de asamblare?**
- d. fără anumite modificări CPI-ul va creşte - va trebui să adăugăm resurse hardware
- **e. este necesară introducerea de registre pentru a fi capabili să executăm o instrucţiune în mai mulți ciclii de ceas**

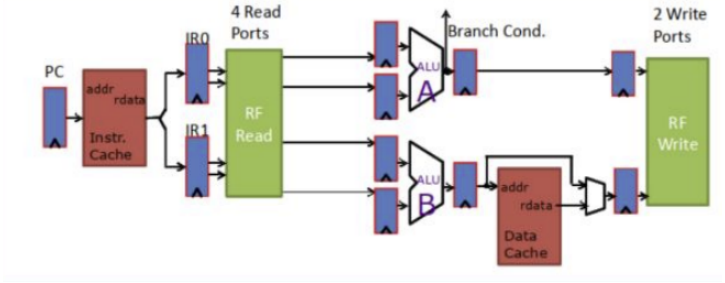
- Primul pas în determinarea semnalului C_STALL este determinarea semnalului C_DEST aşa cum se prezintă în figura de mai jos. De ce este nevoie de ws şi de we? Dar de CASE?



		source(s)	destination
ALU	rd ← (rs) func (rt)	rs, rt	rd
ALUI	rt ← (rs) op immediate	rs	rt
LW	rt ← M [(rs) + immediate]	rs	rt
SW	M [(rs) + immediate] ← (rt)	rs, rt	
BZ	*cond* (rs)		
	true: PC ← (PC) + immediate	rs	
	false: PC ← (PC) + 4	rs	
J	PC ← (PC) + immediate		
JAL	r31 ← (PC), PC ← (PC) + immediate		31
JR	PC ← (rs)	rs	
JALR	r31 ← (PC), PC ← (rs)	rs	31

  - a. CASE este folosit pentru a identifica tipul instrucțiunii. ws și we sunt necesare deoarece câte o dată se scrie o destinaţie
  - b. CASE este folosit pentru a identifica tipul instrucţiunii. ws este în plus, se poate și fără el. we este semnalul care ne spune dacă este nevoie să scriem sa nu.
  - **c. CASE este folosit pentru a identifica tipul instrucțiunii. ws și we sunt necesare deoarece câte o dată se scrie o destinație sau nu**
  - d. CASE este folosit pentru a identifica tipul instrucțiunii. ws și we sunt necesare deoarece câte o dată se citește o destinaţie sau nu e. CASE este folosit pentru a identifica tipul instrucțiunii. ws și we sunt necesare deoarece câte o dată se citeşte o destinaţie

- În cazul unui procesor superscalar IO cu 2 căi ca în figura de mai jos sunt planificate spre execuţie 2 instrucțiuni. O instrucțiune LW în registrul IRO şi o instrucţiune ADD în registrul IR1. Se pot ele executa ?

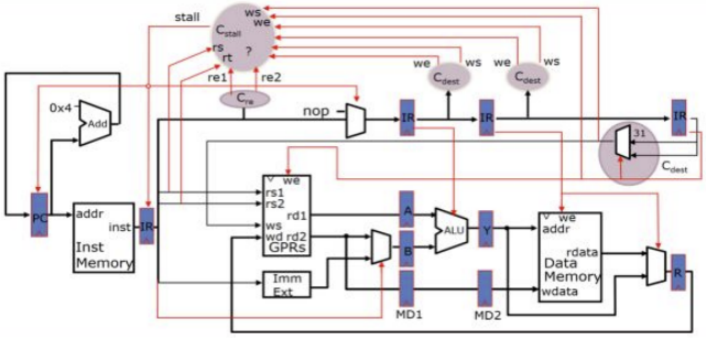


  - a. Nu deoarece registrul IRO are 4 ieşiri iar operaţia LW are un singur operand
  - b. Da, deoarece procesorul este prevăzut cu 2 căi de execuție
  - **c. Nu**
  - d. Da

- Misiunea unui arhitect de calculatoare este aceea de a lua decizii cât mai subtile de proiectare pentru a îndeplini anumite cerințe precum:
  - a. consum de putere cât mai mare pentru a face calculatorul să ruleze cât mai rapid
  - **b. o durată de viață cât mai mare a arhitecturi**

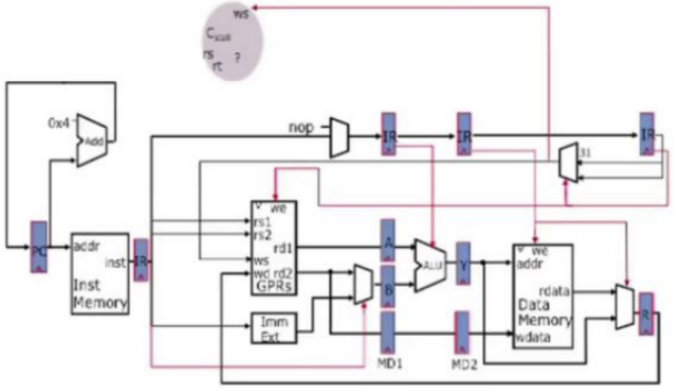
- c. consum de putere cât mai mic pentru a face calculatorul să ruleze cât mai rapid
- **d. uşurinţă în programare**
- e. o durată de viaţă cât mai mică a arhitecturi pentru a permite conceptelor noi să fie imediat exploatate

- Se consideră implementarea unui procesor ca în figura de mai jos. Când nu trebuie verificată condiţia de STALL şi de ce?



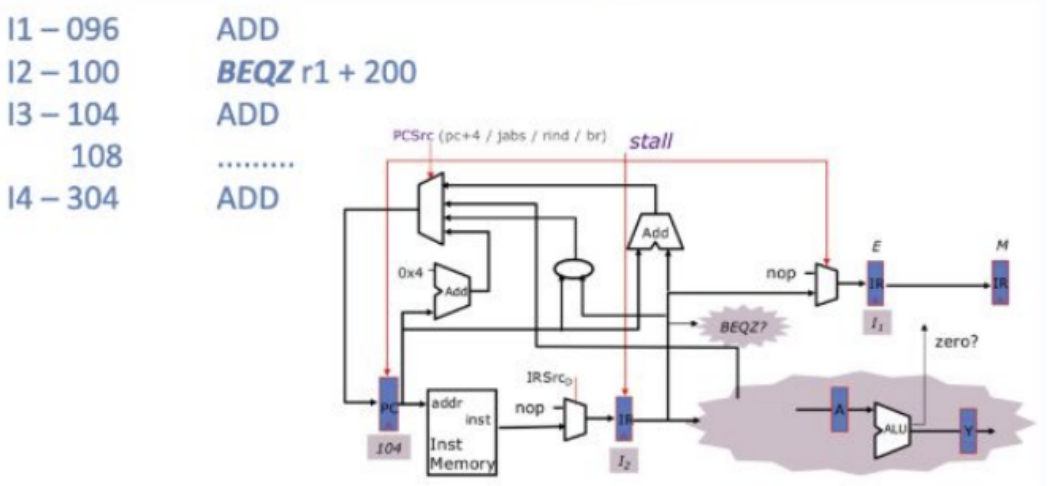
- **a. Condiția de STALL nu trebuie verificată în cazul instrucțiunilor în care operandul 2 este inclus în instrucţiune**
- **b. Condiţia STALL nu trebuie verificată în cazul instrucțiunilor care nu citesc un registru**
- c. În cazul instrucţiunii SW deoarece nu scrie nici un registru
- d. Condiția de STALL trebuie verificată întotdeauna
- e. În cazul instrucțiunii LW deoarece se scrie un registru

- Se consideră implementarea procesorului de mai jos. În execuția unui program trebuie introdus STALL atunci când:



- a. RT al instrucţiunii aflate în stagiul EX nu este egal cu OPCODE al unei instrucțiuni necomise
- b. RT al instrucţiunii aflate în stagiul IF nu este egal cu OPCODE al unei instrucțiuni necomise
- c. RS al instrucţiunii aflate în stagiul ID nu este egal cu RD al unei instrucțiuni necomise
- d. RT al instrucțiunii aflate în stagiul EX egal cu OPCODE al unei instrucțiuni necomise
- e. immediate16 al instrucțiunii aflate în stagiul IF nu este egal cu immediate26 al unei instrucțiuni necomise
- f. RT al instrucţiunii aflate în stagiul IF egal cu OPCODE al unei instrucțiuni necomise
- **g. RS al instrucţiunii aflate în stagiul ID egal cu RD al unei instrucțiuni necomise**
- h. immediate16 al instrucțiunii aflate în stagiul IF egal cu immediate26 al unei instrucțiuni necomise

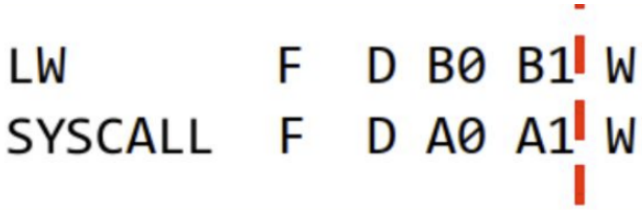
- Pentru execuția următorului program în bandă de asamblare se presupune că instrucțiunea 3 are nevoie de STALL și în același moment de timp trebuie să facem KILL. Este important ca instrucțiunea din stagiul DECODE să fie invalidă ?



  - a. Subiect netratat la curs (sarumana nea decebal)
  - b. Nu, deoarece STALL-ul trebuie să aibă prioritate minimă
  - c. Da, deoarece STALL-ul trebuie să aibă prioritate maximă
  - d. Nu
  - e. Nu are nici o importanță invaliditatea instrucţiunii din stagiul de DECODE
  - **f. Da**

- Modul de execuţie al unei întreruperi presupune:
  - a. Execuția unei întreruperi presupune obligatoriu existenţa unei benzi de asamblare
  - b. Redirecţionarea controlului fluxului de instrucțiuni se va face pentru o perioadă limitată de timp
  - **c. Redirecționa controlul fluxului de instrucțiuni într-o altă zonă**
  - **d. La expirarea intervalului de timp necesar executării unei întreruperi se revine înapoi**
  - e. Execuția unei întreruperi presupune obligatoriu un procesor superscalar IO

- Se consideră execuția a două instrucțiuni ca în figura de mai jos. În cazul în care instrucțiunile sunt executate altfel decât sunt prezentate în figura de mai jos ce trebuie făcut ?



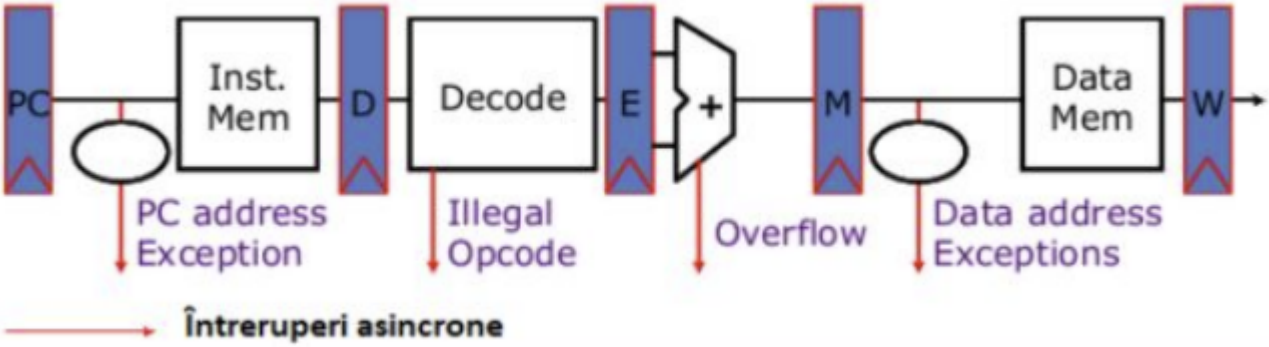
  - a. eliminăm instrucțiunea LW
  - b. ignorarea instrucțiunii SYSCALL
  - **c. ignorarea rezultatului instrucţiunii LW**
  - d. Nu trebuie făcut nimic
  - e. eliminăm instrucţiunea SYSCALL

- Se consideră execuția unui program conform figurii de mai jos. După cum se poate observa din execuția programului, instrucţiunea BEQZ este luată abia în etapa A0 ceea ce implică ca următoarele 7 instrucțiuni să fie KILL. Acest lucru conduce la un impact foarte mare asupra CPI-ului. Ce soluții avem de îmbunătăţire ? (Fără răspunsuri)

```
BEQZ  F  D  I  A0 A1 W
OpA   F  D  I  B0 -  -
OpB      F  D  I  -  -  -
OpC      F  D  I  -  -  -
OpD         F  D  -  -  -  -
OpE         F  D  -  -  -  -
OpF            F  -  -  -  -  -
OpG            F  -  -  -  -  -
OpH               F  D  I  A0 A1 W
OpI               F  D  I  B0 B1 W
```

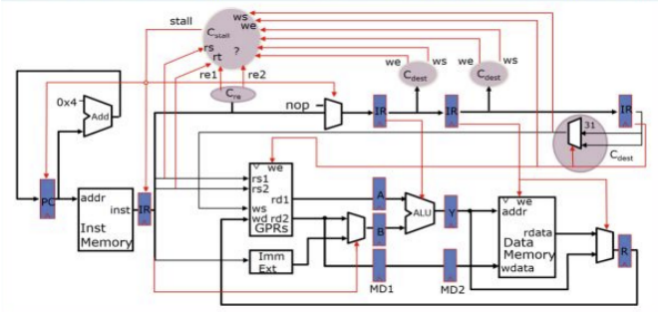
  - a. Nu avem nici o soluție viabilă pentru acest tip de procesor
  - b. mărirea numărului de stagii de pipe care preced stagiul A0 pentru obţinerea unei predicţii cât mai bune
  - c. mărirea numărului de stagii de pipe care preced stagiul AO
  - **d. Implementarea unui predictor de branch-uri cât mai bun**
  - e. micşorarea numărului de stagii de pipe care preced stagiul AO

- Conform figurii de mai jos care este cea mai veche întrerupere din banda de asamblare ?



  - a. Nici una deoarece nu există un sistem de prioritizare ca să ne dăm seama
  - b. Overflow
  - c. Illegal opcode
  - **d. Data address exception**
  - e. PC address exception

- Se consideră implementarea unui procesor ca în figura de mai jos. De ce este MUX-ul din stagiul WB încercuit ?



  - a. Avem nevoie de IR deoarece salturile şi legăturile doar citesc şi din acest motiv este nevoie de IR

- **b. Destinaţia nu este codată în câmpul RD al instrucţiunii aşa că IR este hardcodat cu 31**
- **c. Deoarece poate fi văzut ca un registru JAL către o destinaţie implicită**
- **d. Avem nevoie de IR deoarece salturile și legăturile doar scriu și din acest motiv este nevoie de IR**

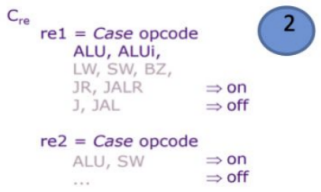
- Se consideră programul de mai jos executat în bandă de asamblare. Când se poate face eliminarea instrucțiunilor 104 şi 108 ?

```
I1 – 096      ADD
I2 – 100      BEQZ r1 + 200
I3 – 104      ADD
      108     .....
I4 – 304      ADD
```

  - a. După faza de MEM b. După faza de IF
  - c. După faza de WB
  - d. După faza de ID
  - **e. După faza de EX**

- În cadrul determinării semnalului de control C_STALL etapa a doua presupune introducerea a două ecuații: re1 și re2. De ce?

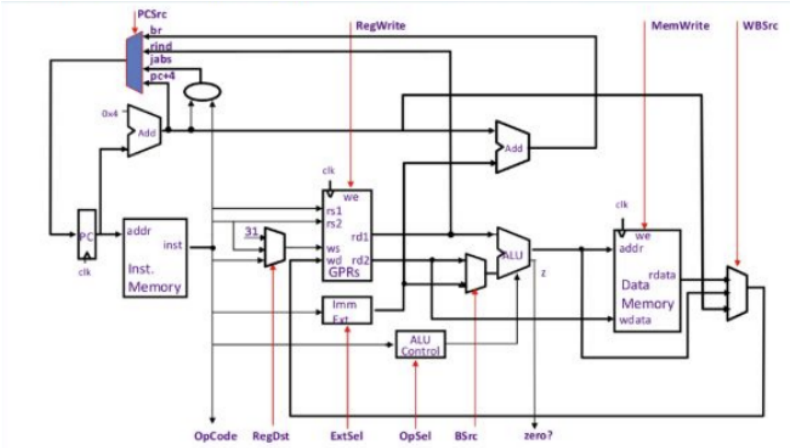
		source(s)	*destination*
ALU	rd ← (rs) func (rt)	rs, rt	rd
ALUI	rt ← (rs) op immediate	rs	rt
LW	rt ← M [(rs) + immediate]	rs	rt
SW	M [(rs) + immediate] ← (rt)	rs, rt	
BZ	*cond* (rs)		
	true: PC ← (PC) + immediate	rs	
	false: PC ← (PC) + 4	rs	
J	PC ← (PC) + immediate		
JAL	r31 ← (PC), PC ← (PC) + immediate		31
JR	PC ← (rs)	rs	
JALR	r31 ← (PC), PC ← (rs)	rs	31



  - a. Cele 2 ecuații reprezintă valorile realocate din cadrul unor registre folosite doar de instrucţiunile JALR și JAL
  - **b. Cele 2 ecuații reprezintă valorile realocate din cadrul unor registre folosite de anumite instrucţiuni**
  - c. Instrucțiunile JALR şi JAL scriu întotdeauna și apoi nimic nu mai scrie fişierul de registre. ca atare re1 este folosit pentru citirea primului operand cât timp re2 este folosit pentru citirea celui de al doilea operand
  - d. Avem nevoie obligatoriu doar de re1. re2 poate că se referă la operandul 2 dar cum se poate vedea şi în figură el va trebui să acopere instrucţiunea SW care nu are un al doilea operand. Ar trebui eliminat pentru creşterea performanței benzii de asamblare

- În cadrul figurii de mai jos, CP-ul este pus la finalul benzii de asamblare. De ce nu este el pus la finalul stagiului EX ?
  - **a. El se pune obligatoriu la finalul benzii de asamblare întotdeauna ?**
  - b. Putem stabili dacă folosim sau nu CP exact ca în cazul procesoarelor 000
  - c. El trebuie pus cât mai devreme în banda de asamblare deoarece este bine ca excepțiile să stea cât mai puțin în banda de asamblare
  - d. Ar fi mai bine să fie pus în faza de EX sau chiar în faza de ID
  - **e. El trebuie pus acolo unde nu mai pot fi luate excepţii în pipe**

- Execuția unei instrucțiuni în cazul procesorului de mai jos fără bandă de asamblare:



  - a. presupune necesitatea unui ciclu de ceas mic
  - **b. presupune necesitatea unui ciclu de ceas mare**
  - c. Figura nu prezintă semnale de control, deci nu se poate numi un procesor, ca atare nu poate executa instrucțiuni
  - **d. presupune trecerea prin toate elementele de la stânga la dreapta**
  - e. presupune trecerea prin toate elementele dar nu contează ordinea

- Se presupune execuția următorului program şi maparea instrucțiunilor în memoria cache de instrucțiuni conform figurii de mai jos. Totodată se presupune că nu avem JUMP. Așa cum se poate observa în figura, memoria poate conține 4 instrucțiuni pe o linie iar într-un ciclu de ceas se citesc 2 instrucţiuni odată (inclusiv pentru ciclul 4). Ce tip de memorie cache de instrucțiuni este necesară pentru a fi fezabil acest tip de citire?

```
Cyc Addr Instr
0   0x000 OpA
0   0x004 OpB
1   0x008 OpC
1   0x00C J 0x100
…
2   0x100 OpD
2   0x104 J 0x204
…
3   0x204 OpE
3   0x208 J 0x30C
…
4   0x30C OpF
4   0x310 OpG
5   0x314 OpH
```

0x000	0	0	1	1
…				
0x100	2	2		
…				
0x200		3	3	
…				
0x300				4
0x310	4	5		

  - a. O memorie cache de instrucțiuni single port cu o capacitate putere a lui 2
  - b. O memorie cache de instrucțiuni single port
  - **c. O memorie cache de instrucțiuni dual port**
  - d. O memorie cache de instrucțiuni cu cel puțin 4 porturi
  - e. O memorie cache de instrucțiuni cu cel mult 4 porturi

- Este important sa avem compilatoare cat mai inteligente deoarece:

- a. eliminam mai ușor instrucțiunile care nu sunt definite în ISA
- b. eliminam mai ușor instrucțiunile care sunt interdependente
- **c. astfel eliminăm mai ușor instrucțiunile redundante**
- **d. eliminam mai ușor instrucțiunile care nu ne sunt utile**
- e. eliminam mai ușor instrucțiunile mai lungi

- Apariția unei întreruperi precise presupune ca:
  - a. toate instrucțiunile anterioare ei sa fie kill la următorul ciclu de ceas și toate instructiunile dupa ea sa poate fi comise la următorul ciclu de ceas
  - **b. toate instrucțiunile anterioare ei sa fie comise și toate instrucțiunile după ea să poată fi kill**
  - c. toate instructiunile anterioare ei sa fie comise la urmatorul ciclu de ceas si toate instructiunile dupa ea sa poata fi kill
  - d. toate instructiunile anterioare ei sa fie kill si toate instructiunile dupa ea sa poata fi comise la urmatorul ciclu de ceas
  - e. toate instructiunile anterioare si dupa vor fi kill

- Când procesorul decide să proceseze o întrerupere execută:

  1. Oprește programul curent la instrucțiunea I(i), terminând complet execuția tuturor instrucțiunilor până la I(i-1) – *întrerupere precisă.*
  2. Salvează PC-ul instrucțiunii I(i) într-un registru special EPC
  3. Dezactivează întreruperile și transferă controlul către o întrerupere desemnată de controller-ul de întreruperi care rulează în kernel mode

  Ce se intampla in cazul in care apare o noua intrerupere in timpul tratarii unei intreruperi?
  -> a. o solutie ar fi sa stergem EPC-ul si apoi reluam intreruperile in interiorul intreruperii
  -> b. o solutie ar fi sa memoram PC+4 si apoi reluam intreruperile in interiorul intreruperii
  -> **c. o solutie ar fi sa memoram EPC-ul in memorie si apoi reluam intreruperile in interiorul intreruperii**
  -> d. o solutie ar fi sa memoram complet aceasta posibilitate

- Se considera urmatoarea executie a unui program in banda de asamblare. Asa cum se poate observa in executia programului au fost introduse niste NOP-uri. De ce?



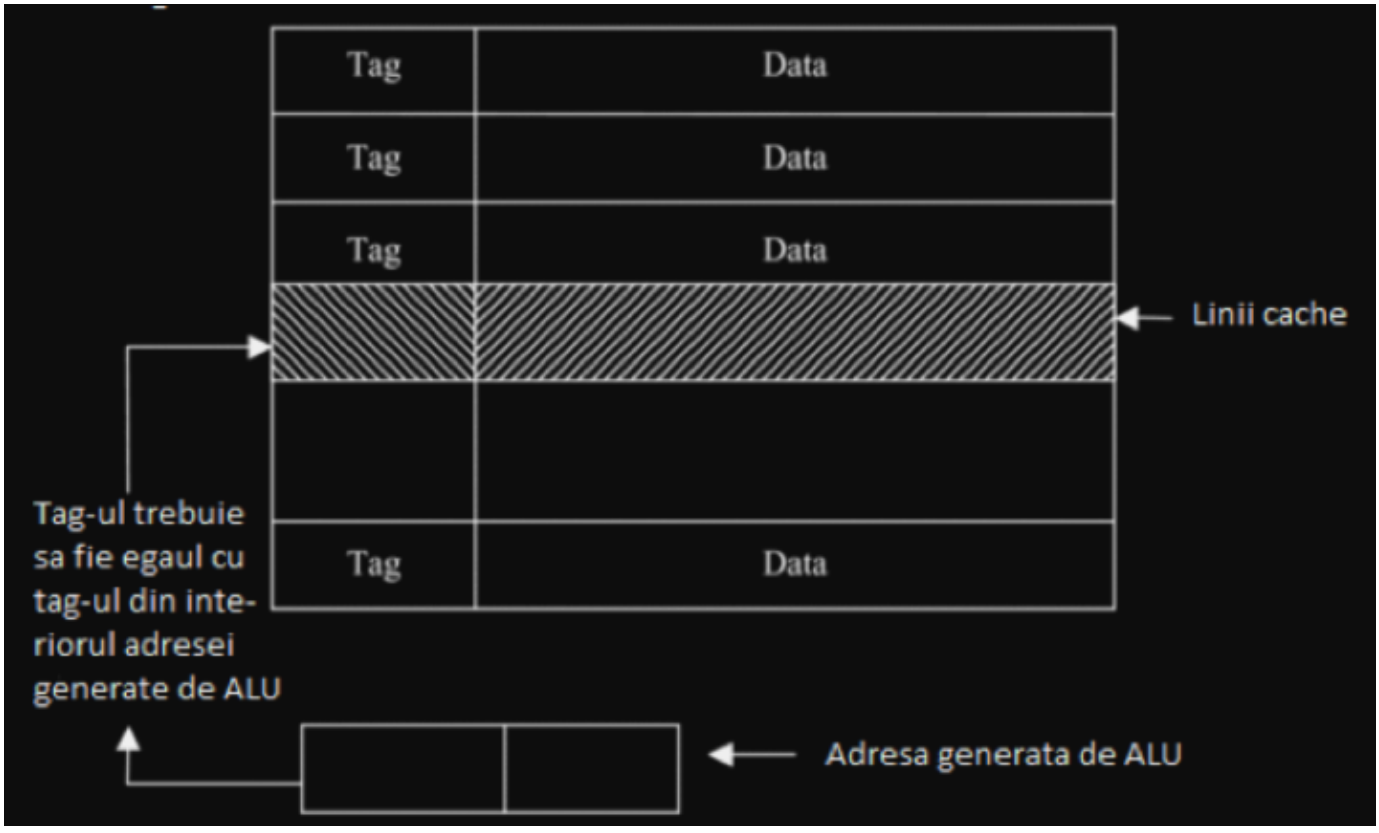
  - **a. fara aceste NOP-uri in cazul in care prima instructiune ar fi JUMP atunci am avea hazard de control**
  - b. fara aceste NOP-uri in cazul in care prima instructiune ar fi BRANCH atunci am avea hazard de date
  - c. fara aceste NOP-uri in cazul in care prima instructiune ar fi BRANCH atunci am avea hazard de control
  - d. fara aceste NOP-uri in cazul in care prima instructiune ar fi JUMP atunci am avea hazard structural

- e. fara aceste NOP-uri in cazul in care prima instructiune ar fi JUMP atunci am avea hazard de date
- f. fara aceste NOP-uri in cazul in care prima instructiune ar fi BRANCH atunci am avea hazard structural

- Clusterelor superscalare presupun legarea in paralel a mai multor procesoare superscalare. Modalitatea de functionare a acestor clustere presupune:
  - a. nu exista o astfel de posibilitate pentru procesoarele superscalare
  - **b. putem face switch intre pipe_1 si pipe_2 iar in interioarul fiecarui pipe mai avem 2 pipe-uri intre care putem face de asemenea switch**
  - c. nu putem face switch intre pipe_1 si pipe_2 iar in interiorul fiecarui pipe mai avem 2 pipe-uri intre care putem face de asemenea switch
  - d. putem face switch intre pipe_1 si pipe_2 iar in interioarul fiecarui pipe mai avem 2 pipe-uri intre care nu mai putem face switch

- Orice adresa de memorie generata de ALU poate fi descompusa in:
  - **tag (corect, curs)**
  - **index (corect, curs)**
  - **deplasament / offset (corect, curs)**

- Se consideră memoria cache din figură.



În cazul unei memorii cache complet asociativă (datele se pun în orice locație de memorie cache) câmpul TAG:
Select one or more:
1. va trebui să aibă dimensiunea 2*(număr biți adresă)
2. nici un răspuns corect
3. va trebui să memoreze o parte din adresă
4. va trebui să aibă dimensiunea 2^(număr biți adresă)
5. **va trebui să memoreze întreaga adresă (☑)**

- Latență se definește ca:
    - **timpul pentru un singur ceas**
    - media timpurilor pentru mai multe ceasuri
    - timpul pentru mai multe ceasuri
    - nici un raspun corect

Nr. crt.	Intrebare	Variante de raspuns	Raspunsuri corecte
1	**În cazul sistemelor embedded pentru determinarea performantei ne intereseaza:**	a. Niciun raspuns corect b. Cazul sistemelor embedded datorita resurselor hardware sarace nu ne intereseaza performanta lor. c. Numarul de task-uri ce poate fi procesat într-o secunda d. Durata fiecarui task	C, D
2	**Principiul localizarii temporale spune ca:**	a. Daca se face referire la un anumit obiect, este posibil ca acesta sa fie referit din nou cât de curând b. Nicio asertiune nu este adevarata c. Daca se face referire la un anumit obiect, este imposibil ca acesta sa fie referit din nou cât de curând d .Daca se face referire la un anumit obiect, este imposibil ca acesta sa mai fie referit vreodata	A
3	**Se da figura de mai jos care reprezinta executia în pipeline a programului prezentat în figura. Care dintre urmatoarele asertiuni sunt adevarate ?**	a. Rezultatul corect este -10 b. Rezultatul corect este -20 c. Rezultatul corect este 20	B
4	**Ierarhiile de memorie folosesc avantajul localizarii spatiale:**	a. prin duplicarea blocurilor continând cuvinte învecinate din memorie în nivelurile superioare ale ierarhiei b. prin mutarea blocurilor continând cuvinte învecinate din memorie în nivelurile superioare ale ierarhiei c. prin mutarea blocurilor continând cuvinte învecinate din memorie în nivelurile inferioare ale ierarhiei	B
5	**Care dintre urmatoarele asertiuni sunt adevarate ?**	a. Un program nu acceseaza toata sectiunea sa de cod cu aceeasi probabilitate b. Un program acceseaza toata sectiunea sa de date cu aceeasi probabilitate c. Un program acceseaza toata sectiunea sa de cod cu aceeasi probabilitate d. Nici un raspuns nu este corect e. Un program nu acceseaza toata sectiunea sa de date cu aceeasi probabilitate	A, E

6	**Pe baza pozei de mai sus sa se determine câte ore sunt necesare pentru terminarea tuturor task-urilor. Daca luam în considerare introducerea pipeline-ului în câte ore s-ar termina toate task-urile ?**	a. În cazul clasic fara pipeline este nevoie de 8 ore. În cazul cu pipeline este nevoie de 3 ore b. Nici un raspuns corect c. În cazul clasic fara pipeline este nevoie de 10 ore. În cazul cu pipeline este nevoie de 2,5 ore d. În cazul clasic fara pipeline este nevoie de 8 ore. În cazul cu pipeline este nevoie de 3,5 ore e. În cazul clasic fara pipeline este nevoie de 8 ore. În cazul cu pipeline este nevoie de 4 ore	D
7	**Procesorul MIPS dispune de urmatoarele moduri de adresare:**	a. Adresare a registrelor b. Cel putin un raspuns este gresit c. Adresare pseudodirecta d. Adresare relativa la PC e. Adresare imediata f. Adresare prin baza sau deplasare g. Toate raspunsurile sunt gresite	A, C, D, E, F
8	**Sa se specifice care dintre asertiunile de mai jos sunt false:**	a. Fractiunea din accesele la memorie ce nu au gasit datele în nivelul superior de memorie se numeste rata de succes b. Neregasirea datelor pe nivelul superior se numeste HIT c. Regasirea informatiei necesare procesorului într-un bloc de memorie superior se numeste MISS d. Timpul de succes, reprezinta timpul necesar accesului la un nivel superior al ierarhiei de memorie, ce include si timpul necesar determinarii tipului de acces e. Penalizarea de esec, reprezinta timpul necesar înlocuirii blocului din nivelul superior cu blocul corespunzator din nivelul inferior, incluzând si timpul trimiterii acestui bloc catre procesor. f. Fractiunea din accesele la memorie care nu au gasit datele în	A, B, C
9	**Din ce este format limbajul unui calculator ?**	a. Din vocabular b. Din cuvinte c. Nici unul din raspunsurile de mai sus nu este corect d. Din cuvinte si vocabular	Varianta 1: D (drive ACS) Varianta 2: **A, B, D (Ovidiu, Vlad)**

10	**În figura de mai jos se prezinta cu albastru liniile de control pentru implementarea pipeline a procesorului MIPS. Care dintre urmatoarele asertiuni sunt adevarate pentru faza de extragere a instructiunii?**	a. semnalele de control pentru citirea memoriei de instructiuni si pentru scrierea PC-ului sunt activate permanent b. nicio asertiune nu este adevarata c. o singura asertiune este adevarata d. nu avem ceva special de controlat în acesta etapa pipeline	Varianta 1: A (drive ACS) Varianta 2: C (drive ACS) Varianta 3: C, D (drive ACS) **Varianta 4: A, C (Ovidiu, Roxana, Andreea, Andrei, Vlad**
11	**În figura de mai jos se prezinta cu albastru liniile de control pentru implementarea pipeline a procesorului MIPS. Care dintre urmatoarele asertiuni sunt adevarate pentru faza de citire a fisierului de registre?**	a. nicio asertiune nu este adevarata b. nu avem ceva special de controlat în acesta etapa pipeline c. în fiecare ciclu de ceas se repeta acelasi lucru d. o singura asertiune este adevarata	~~Varianta 1: C (Drive ACS)~~ ~~Varianta 2: C, D (Drive ACS)~~ **Varianta 3: B, D** (**Roxana, Ovidiu, Andrei, Stefan, Vlad**)

12	**Figura de mai jos prezinta protocolul HANDSHAKING. Protocolul prezentat este corect ?**	a. Când memoria vede semnalul ReadReq activat, ea citeste adresa de la magistrala de date si activeaza Ack pentru a indica ca a vazut activarea semnalului ReadReq b. Dispozitivele I/O vad semnalul Ack activat si elibereaza ReadReq si liniile de date c. Memoria observa ca semnalul ReadReq este 0 logic si inactiveaza semnalul Ack pentru cerere acceptat? d. Acest pas începe când memoria are datele ready. Ea plaseaza datele de la ReadReq pe liniile de date si activeaza semnalul DataRdy e. Dispozitivele I/O observa semnalul DataRdy activ, citesc datele si semnalel de la magistrala si activeaza semnalul Ack f. Memoria observa semnalul Ack activ, inactiveaza semnalul DataRdy si elibereaza liniile de date g. Dispozitivele I/O observa semanlul DataRdy inactiv, inactiveaza	**Varianta 1: A, B, C, D, E, F, G** (**Roxana, Andreea**) ~~Varianta 2: A, B, D, E, F, G, H (Drive ACS)~~ ~~Varianta 3: B (drive ACS)~~
13	**Figura afisata prezinta comparativ urmatoarele variante de RAID:**	a. Figura din stânga prezinta RAID 5 iar cea din dreapta RAID 4 b. Figura din stânga prezinta RAID5 iar cea din dreapta RAID6 c. Figura din stânga prezinta RAID 4 iar cea din dreapta RAID 5 d. Nici un raspuns corect	C
14	**Faptul ca un cuvant in MIPS incepe de la o adresa multiplu de 4 reprezinta:**	a. o restrictie de copiere b. o restrictie de aliniere c. nici un raspuns nu este corect d. o restrictie ciudata facuta de proiectanti	B
15	**Unul dintre principiile de proiectare al unui calculator spune ca :** **"Simplitatea favorizeaza uniformitatea".** **Care este acest principiu ?**	a. Principiul 3 b. Principiul 1 c. Nu exista un asemenea principiu d. Principiul 2 (mai mic mai rapid)	B
16	**Figura de mai jos prezinta cum se realizeaza faza de decodificare a instructiuni de încarcare. Care din pasii de mai jos sunt necesari pentru aceasta operatie ?**	a. Valorile furnizate de EI/DI, împreuna cu adresa PC incrementata se regasesc în DI/EX b. registrul EI/DI furnizeaza câmpul imediat de 16 biti caruia i se extinde semnul pâna la 32 de biti c. Registrul EI/DI furnizeaza numele registrelor pentru citirea celor doua registre d. Valorile furnizate de EI/DI, împreuna cu adresa PC se regasesc în MEM/WB e. registrul EI/DI furnizeaza doar câmpul imediat de 16 biti f. Registrul EI/DI furnizeaza numerele registrelor pentru citirea celor doua registre	**A, B, F** **(confirmare Roxana, Vlad**)

17	**Care dintre urmatoarele asertiuni sunt adevarate ?**	a. Instructiunile pot fi reprezentate ca numere b. Pastrarea instructiunilor intr-un calculator se fac ca o serie de semnale electronice inalte sau joase c. Instructiunile nu pot fi pastrate ca numere d. Pastrare instructiunilor intr-un calculator se fac ca o serie de semnale electronice medii	A, B
18	**Formatul instructiunii STORE presupune existenta urmatoarelor campuri:**	a. offset-ul pentru selectarea elementului din tablou b. registrul de memorat c. registrul de baza d. nume operatie e. cel putin un raspuns gresit	A, B, C, D
19	**Figura de mai jos prezinta modalitatea de rescriere a fisierului de registre în cazul instructiunii de încarcare. Ce defectiune remarcati?**	a. Instructiunea din registrul pipeline EI/DI ofera numarul registrului pentru scriere, desi acesta instructiune are loc mult timp dupa instructiunea de încarcare b. Instructiunea din registrul pipeline EI/DI ofera numarul registrului pentru scriere, desi acesta instructiune are loc mult timp înaintea instructiunea de încarcare c. Instructiunea din registrul pipeline EI/DI ofera numele registrului pentru scriere, desi acesta instructiune are loc mult timp dupa instructiunea de încarcare	~~Varianta 1: C (Drive ACS)~~ **Varianta 2: B (Ovidiu, Roxana, Vlad)**
20	**Specificati cum poate fi masurata latimea de banda dintre doua dispozitive de I/O:**	a. Viteza cu care datele se transfera de la discul magnetic la procesor b. Câte operatii de I/O pot fi efectuate în unitatea de timp c. Nici o afirmatie nu este corect? d. Numarul de ciclii de ceas necesari unei operatii de I/O e. Numarul de operatii de I/O ce pot fi efectuate într-o unitate de timp	B, E
21	**Figura de mai jos prezinta modalitatea de realizarea a accesului la memorie pentru instructiunea de memorare. Pentru realizarea acestui acces care din pasii de mai jos trebuie parcursi ?**	a. Scrie memoria de date folosind adresa din registrul pipeline IF/ID b. Citeste memoria de date folosind adresa din registrul pipeline EX/MEM c. Sterge apoi data din registrul pipeline MEM/RS d. încarca data în registrul pipeline MEM/RS	**Varianta 1: B, D (Ovidiu, Roxana)** ~~Varianta 2: A (Drive ACS)~~

22	**Prin hazard pipeline se întelege:**	a. Imposibilitatea executiei instructiunii urmatoare b. Evenimentul ca instructiunea urmatoare sa nu fie executata în urmatorul ciclu de ceas c. Evenimentul ca instructiunea urmatoare sa fie executata în urmatorul ciclu de ceas d. Evenimentul ca instructiunea urmatoare sa fie executata în urmatorii ciclii de ceas	Varianta 1: B Varianta 2: A, B (Bianca)
23	**Care dintre urmatoarele principii este folosit la implementarea memoriei unui calculator sub forma unei ierarhii de memorie ?**	a. O combinatie dintre mai multe principii. b. Principiul localizarii spatiale c. Nicio asertiune nu este valida d. Principiul localizarii în timp e. Principiul localizarii temporale	~~Varianta 1: E (Drive ACS)~~ ~~Varianta 2: D, E (Bianca, Corina)~~ Varianta 3: **B, D, E (Ovidiu, Andrei)**

24	**Care dintre urmatoarele asertiuni este/sunt false ?**	a. O ierarhie de memorii poate fi alcatuita din mai multe niveluri, însa datele la un moment dat sunt copiate între diferite niveluri ale ierarhiei de memorie b. O ierarhie de memorii poate fi alcatuita din mai multe niveluri, însa datele la un moment dat sunt copiate doar între doua niveluri adiacente c. O ierarhie de memorii este întodeauna alcatuita din 2 nivele, însa datele la un moment dat sunt copiate între diferite niveluri ale ierarhiei de memorie	A, C, D
25	**Ce se întâmpla într-un sistem care are implementat RAID 5 daca se întrerupe tensiunea de alimentare între urmatoarele actiuni: actualizarea datei dintr-un bloc; actualizarea blocul unde se mentine data de verificare.** **În cazul acestui scenariu din cele 2 operatii planificate spre executie, numai una dintre ele s-a încheiat cu succes. Ce solutie se poate alege pentru a evita astfel de incidente?**	a. Nu se întâmpla nimic Trebuie avuta în vedere achizitionarea unui UPS. b. Sistemul nu mai poate fi repornit. Trebuie achizitionat un hard disk nou si evident un UPS. c. Nu mai avem paritate între date si blocul de verificar De urgenta trebuie achizitionat un UPS d. Nici un raspuns corect e. Nu mai avem paritate între date si blocul de verificar O solutie ar fi ca scrierea în cele doua blocuri sa se faca simultan.	E
26	**add $s0, $t0, $t1** **sub $t2, $s0, $t3**	a. Se genereaza hazard. Vom avea hazard de date b. Se genereaza hazard. Vom avea hazard pipeline c. Se genereaza hazard. Vom avea hazard structural d. Nu se genereaza hazard	**A, B** **(Bianca, Andrei, Tudor)**
27	**Intsructiunea LOAD este folosita pentru:**	a. deplasarea datelor intre 2 echipamente periferice b. sincronizarea datelor intre 2 registre c. deplasarea datelor de pe magistrala intr-un registru d. deplasarea datelor din memorie in registru	**D (Ovidiu, Tudor)**
28	**Numarul total de biti dintr-o memorie cache cu corespondenta directa este dat de formula urmatoare:**	a. 2^n*(dimensiunea blocului de memorie + dimensiune marcaj - dimensiunea câmpului de validare) b. 2*n*(dimensiunea blocului de memorie + dimensiune marcaj - dimensiunea câmpului de validare) c. 2^n * (dimensiunea blocului de memorie + dimensiune marcaj + dimensiunea câmpului de validare) d. 2*n*(dimensiunea paginii de memorie + dimensiune marcaj + dimensiunea câmpului de validare) e. 2*n^(dimensiunea blocului de memorie + dimensiune marcaj + dimensiunea câmpului de validare)	C

29	**Structura de date ideala pentru transferul în memorie al registrelor este:**	a. lista dublu inlantuita b. coada c. stiva d. lista simplu inlantuita	C
30	**Care dintre urmatoarele afirmatii sunt adevarate ?**	a. Pipeline-ul mareste frecventa la care sunt incepute/terminate instructiunile b. Nici un raspuns corect c. Pipeline-ul mareste numarul de executii simultane pentru instructiuni d. Pipeline-ul nu reduce timpul necesar executiei unei instructiuni.	**A, C, D** **(aproba toti Johnii)**
31	**Precizati care dintre afirmatiile de mai jos sunt adevarate:**	a. Tastatura este un dispozitiv de intrare utilizat de catre om cu o rata a datelor de 0,0001 Mb/s b. Imprimanta laser este un dispozitiv de iesire utilizat de catre om cu rata a datelor de 3.200 Mb/s c. Disk-ul optic este un echipament de tip storage, utilizat de catre masina cu o rata a datelor de 80.000 Mb/s d. Disk-ul magnetic este un echipament de storage utilizat de catre masina cu o rata a datelor de 240.000-2560.000 Mb/s e. Nici o afirmatie nu este corecta f. Afirmatiile b si d sunt incorecte	Varianta 1: A , posibil si F (Roxana) Varianta 2: A, B, C, D (Tudor) Varianta 3: A, B, D (Andrei)
32	**Figura afisata reprezinta:**	a. Nici un raspuns corect b. Mecanismele de citire pentru RAID3 si RAID4 c. Figura din dreapta reprezinta mecanismul de scriere pentru RAID3 d. Figura din stanga reprezinta mecanismul de scriere pentru RAID4	**C, D**
33	**Care dintre urmatoarele asertiuni sunt adevarate ?** **OBS: duplicat**	a. Nici un raspuns nu este corect b. Un program nu acceseaza toata sectiunea sa de cod cu aceeasi probabilitate c. Un program nu acceseaza toata sectiunea sa de date cu aceeasi probabilitate d. Un program acceseaza toata sectiunea sa de cod cu aceeasi probabilitate	

34	**Principiul localizarii spatiale spune ca:**	a. Daca se face referire la un anumit obiect din memorie, obiectele ale caror adrese sunt invecinate cu acesta, tind sa fie adresate cat de curand b. Daca se face referire la un anumit obiect din memorie, obiectele ale caror adrese sunt invecinate cu acesta, tind sa nu fie adresate cat de curand c .Daca se face referire la un anumit obiect, este posibil ca acesta sa fie referit din nou cat de curand d .Toate raspunsurile sunt corecte e .Daca se face referire la un anumit obiect, este posibil ca acesta sa nu fie referit din nou	A
35	**Se stie ca o metoda de evitare a hazardurilor de date este avansare in care din cazurile de mai jos, nu se poate aplica aceasta metoda ?**	a. O instructiune incearca sa citeasca un registru, imediat dupa o instructiune de incarcare care scrie in acelasi registru b. Metoda se poate aplica in orice situatie c .O instructiune incearca sa citeasca un registru, imediat dupa o instructiune de incarcare care scrie intr-un alt registru	**A** **(Andrei, Tudor)**
36	**Notiunea LITTLE END presupune:**	a. Posibilitatea calculatoarelor de a folosi adresele celui mai din stanga semicuvant ca adresa a cuvantului b. Posibilitatea calculatoarelor de a folosi adresele celui mai din dreapta octet ca adresa a cuvantului c. Posibilitatea calculatoarelor de a folosi adresele celui mai din stanga octet ca adresa a cuvantului d .Posibilitatea calculatoarelor de a folosi adresele celui mai din dreapta semicuvant ca adresa a cuvantului	Varianta 1: B Varianta 2: B, D (Andrei)

37	**Se da urmatorul program MIPS:** **lw $t1, 0($t0)** **lw $t2, 4($t0)** **add $t3, $t1, $t2** **sw $t3, 12($t0)** **lw $t4, 8($t1)** **add $t5, $t1, $t4** **sw $t5, 16($t0)** **In acest cod apare hazard? Daca da, unde?**	a. Apare hazard la prima instructiune sw b. Nici un raspuns corect c. Apare hazard la ultima instructiune sw d. Apare hazard la a doua instructiune lw	~~Varianta 1: B~~ Varianta 2: **A, C(Andrei, Roxana, Tudor, Ovidiu, Vlad)**
38	**In conditii ideale, castigul de viteza pentru pipeline este egal cu:**	a. Nu avem niciun castig si trebuie renuntat la concept b. numarul etajelor din pipe / 2 c. numarul etajelor din pipe d. 1 / numarul etajelor din pipe e. Nu exista niciun raspuns corect	C
39	**Primul principiu de proiectare al unui calculator spune ca:**	a. Simplitatea nu conduce la uniformitate b. Uniformitatea conduce la simplicitate c. Uniformitatea este favorizata de simplitate d. Simplitatea favorizeaza uniformitatea	C, D
40	**Cum se determina corespondenta dintre adresele si locatiile memoriei cache ?**	a. (Adresa blocului) + (numarul de blocuri din memoria cache) b. (Adresa paginii) modulo (numarul de pagini din memoria cache) c. (Adresa paginii) / (numarul de pagini din memoria cache) d. Nici o formula nu este corecta e. ((Adresa blocului) modulo (numarul de blocuri din memoria cache)) + 1	D

41	**Bus transaction inseamna :**	a. O secventa de operatii care includ numai cereri b. Niciun raspuns corect c. O stare de mare impedanta in care se poate gasi o magistrala d. O secventa de operatii care include cereri si raspunsuri e. O secventa de operatii care includ numai raspunsuri	~~Varianta 1: D (Drive ACS)~~ **Varianta 2: B (Roxana)**
42	**Accesarea unei date in cazul discurilor magnetice se realizeaza astfel:**	a. pozitionarea unui anumit cap de citire pe o anumita pista, restul capetelor de citire se plaseaza pe alte piste si apoi capul de citire se pozitioneaza pe un anumit sector b. pozitionarea capetelor de citire asupra sectorului dorit c. pozitionarea capetelor de citire asupra pistei dorite d. Niciun raspuns corect	Varianta 1: B, C Varianta 2: D
43	**Memoriile actuale fiind foarte mari apare necesitatea ca adresa de baza a unui tablou sa fie introdusa intr-un registru deoarece :**	a. niciun raspuns corect b. nu exista o asemenea necesitate c. dimensiunea rezervata zonei de offset este prea mica d. nu este loc suficient in zona rezervata pt offset	C, D

44	**Specificati care din urmatoarele afirmatii sunt adevarate:**	a. Programele prezinta localizare temporala b. Programele prezinta localizare spatiala c. Programele prezinta localizare temporala cat si localizare spatiala d. Toate asertiunile sunt adevarate	~~Varianta 1: C (Drive ACS)~~ Varianta 2: **A, B, C, D (Ovidiu)**
45	**Organizarea dispozitivelor de I/O se poate face avand in vedere urmatoarele caracteristici:**	a) Comportarea, partenerul si rata datelor b) Niciun raspuns corect c) Comportarea, partenerul si valoarea minima de transfer a datelor d) Partenerul, comportarea si valoarea maxima de transfer a datelor	A, D
46	**Conceptul de BIG END presupune:**	a. Posibilitatea calculatoarelor de a folosi adresele celui mai din stanga octet ca adresa a cuvantului b. Posibilitatea calculatoarelor de a folosi adresele celui mai din dreapta octet ca adresa a cuvantului c. Posibilitatea calculatoarelor de a folosi adresele celui mai din dreapta semicuvant ca adresa a cuvantului d. Posibilitatea calculatoarelor de a folosi adresele celui mai din stanga semicuvant ca adresa a cuvantului	~~Varianta 1: A~~ Varianta 2: A, D
47	**In MIPS intructiunile:**	a. au acelasi lungime b. acelasi reprezentare c. reprez. diferita d. lungime diferita	A, C
48	**La Cache, ce fac index si marcaj (tag)**	a) indexul memoriei cache, folosit la selectarea blocului de memorie cache b) câmpul marcajului, folosit la compararea cu valoarea din câmpul marcaj al memoriei cache c) altceva d) altceva	A, B

49	**Cum putem rezolva in caz de hazard?**	a) Bubble b) Forward c) Stalling d) nop	Varianta 1: A, B, C (**Tudor**) Varianta 2: A, B, C, D (**Andrei, Ovidiu**)
50	**Cand instructiunile se executa in paralel si au dependente, atunci:**	a) Rezultatele pot fi partial gresite b) Rezultatele pot fi toate gresite c) Rezultatele nu pot fi gresite d) Niciuna	A, B
51	**viteza de transfer pe HDD depinde de:**	a) Pozitionarea capetelor pe piste dorite b) ..... pe sectoare dorite c) RPM a lui HDD d) marimea block-ului	A, B, C (Ovidiu)
52	**Se considera microprocesorul MIPS. Numiti cele 5 stagii de pipeline.** **ATENTIE - ordinea este importanta.**	a.EX, EX, DI, DI, WB b.MEM, IF, EX, DI, WB c.EX, IF, DI, WB, MEM d.IF, DI, EX, MEM, WB e.IF, EX, DI, WB, MEM	D
53	**Se da figura de mai jos care reprezinta executia in pipeline a programului prezentat în partea din stânga a figurii. Care dintre urmatoarele asertiuni sunt adevarate?**	a. Instructiunile care obtin rezultatul corect sunt and si or. b. Nici o asertiune nu este adevarata. c. Instructiunile care obtin rezultatul corect sunt add si sw.	
54	**De ce depinde perioada de ceas la MIPS cu BA (banda de asamblare):**	a) Durata stagiului cel mai scurt b) Durata stagiului cel mai lung c) bla bla d) bla bla	B
55	**Ce prezinta CPU Mips?**	a. Procesul MIPS prezintă o arhitectură de tip LOAD-STORE b. Procesul MIPS prezintă o arhitectură de tip LOAD-LOAD c. Procesul MIPS prezintă o arhitectură de tip ADD-LOAD d. Procesul MIPS prezintă o arhitectură de tip STORE-STORE	A

56	**In cadrul figurii de mai jos sunt prezentate semnalele de control pentru procesorul MIPS varianta pipeline. Pentru executia unei instructiuni de tipul LW valoarea semnalelor de control este:**	a. Niciun raspuns corect b. RegDst = 0, ALUOp1 = 0, ALUOp0 = 0, ALUSRC = 1, Branch = 0, MemRead = 1, MemWrite = 0, RegWrite = 1, MemToReg = 0 c. RegDst = 1, ALUOp1 = 1, ALUOp0 = 1, ALUSRC = 1, Branch = 1, MemRead = 1, MemWrite = 1, RegWrite = 1, MemToReg = 1 d. RegDst = 1, ALUOp1 = 0, ALUOp0 = 0, ALUSRC = 1, Branch = 0, MemRead = 1, MemWrite = 0, RegWrite = 1, MemToReg = 1	A
57	**In cadrul figurii de mai jos (nu era figura pusa pe drive) se prezinta decodificarea instructiunii si citirea fisierului de registre. Figura face referire la:**	a. Cele 3 valori sunt memorate in registrul pipe ID/EX cat timp se face incrementarea PC-ului b. Cele 3 valori sunt memorate in registrul pipe ID/EX c. partea din registrul pipe IF/ID	Varianta 1: A, B (Drive ACS) Varianta 2: B, C, D, E (Drive ACS)
58	**Penalizarea de esec presupune:**	a. timpul necesar inlocuirii blocului din nivelul superior cu blocul corespunzator din nivelul inferior b. in penalizarea de esec se include si timpul trimiterii acestui bloc catre procesare c. penalizarea de esec se refera la banda de asamblare d. penalizarea de esec nu se refera la blocurile de memorie, ci se refera la adresele obiectelor din memorie e. in penalizarea de esec nu se include si timpul trimiterii acestui bloc catre procesor	A, B
59	**Se considera o memorie cache cu 4 blocuri de cate un cuvant organizata sub forma de memorie cache cu asociativitate partiala (cu 2 cai). Sa se determine numarul de esecuri daca se presupune urmatoarea secventa de adrese bloc: 0,8,0,6,8. Se va considera algoritmul LRU pentru inlocuirea datelor in memoria cache**	a. numarul de esecuri = 4 b. numarul de esecuri = 3 c. numarul de esecuri = 2 d. numarul de esecuri = 1	A
60	**Se considera urmatorul program din figura de mai jos. In ce ciclu de ceas este disponibila valoarea lui $2?**	a. in ciclul 4 de ceas b. in ciclul 5 de ceas c. in ciclul 3 de ceas	B
61	**Pentru reducerea volumului de memorie ocupat de tabelele de pagini putem folosi:**	a. un registru care mentine un bit de validitate b. un registru de limitare c. doua portiuni expandabile (stiva si heap-ul)	B, C

62	**In cadrul figurii de mai jos sunt prezentate semnalele de control pentru procesorul MIPS varianta pipeline. Pentru executia unei instructiuni de tipul BEQ valoarea semnalelor de control este:**	a. Niciun raspuns corect b. RegDst = 0, ALUOp1 = 0, ALUOp0 = 0, ALUSRC = 1, Branch = 0, MemRead = 1, MemWrite = 0, RegWrite = 1, MemToReg = 0 c. RegDst = 1, ALUOp1 = 1, ALUOp0 = 1, ALUSRC = 1, Branch = 1, MemRead = 1, MemWrite = 1, RegWrite = 1, MemToReg = 1 d. RegDst = 1, ALUOp1 = 0, ALUOp0 = 0, ALUSRC = 1, Branch = 0, MemRead = 1, MemWrite = 0, RegWrite = 1, MemToReg = 1	A
63	**Pentru reducerea volumului de memorie ocupat de tabelele de pagini putem folosi:**	a. paginarea tabelelor de pagini b. o functie de cautare pentru adresa virtuala c. o functie de sortare pentru adresa virtuala d. mai multe niveluri de tabele de pagini	A, B, D
64	**Consideram memoria cache din figura de mai jos, care contine 1024 de cuvinte si dimensiunea unui bloc este de un cuvant. Cati biti vor fi folositi pentru comparatia cu marcajul?**	a. 12 biti de date b. niciun raspuns corect c. 18 biti de date d. 20 de biti de date e. 10 biti de date	D
65	**In cadrul figurii de mai jos sunt prezentate semnalele de control pentru procesorul MIPS varianta pipeline. Pentru executia unei instructiuni pasului de INSTRUCTION DECODE a unei instructiuni MIPS, trebuie activate urmatoarele semnale de control:**	a. niciunul b. ALUOp si ALUSRC c. RegDs, MemWrite si MemRead d. toate semnalele de control	**A(Ovidiu, Tudor)**
66	**Prin HIT se intelege:**	a. Regasirea informatiei necesare procesorului intr-un bloc de memorie superior b. Regasirea informatiei necesare procesorului c. Regasirea informatiei necesare procesorului pe un mediu de stocare extern d. neregasirea informatiei necesare	A

67	**In cadrul figurii de mai jos, liniile punctate pot fi inlocuite cu registre pentru mentinerea datelor. Acest lucru este necesar, deoarece:**	a. portiuni ale caii de control sa fie comune pe durata executiei instructiunii b. portiuni ale caii de date sa fie comune pe durata executiei instructiunii c. evitarea aparitiei hazardelor structurale d. evitarea aparitiei hazardelor de date	D
68	**Tehnica de forwarding este valabila doar daca:**	a. Avem cel putin 2 afirmatii corecte b. stagiul de destinatie este in acelasi ciclu de ceas ca si stagiul sursa c. stagiul de destinatie este mai devreme in timp decat stagiul sursa d. stagiul de destinatie este mai tarziu in timp decat stagiul sursa	~~Varianta 1: A, B, D (Drive ACS)~~ Varianta 2: **D (Ovidiu, Andrei)**
69	**Tehnica de pipeline:**	a. creste viteza de procesare in fiecare stagiu b. nu creste viteza de procesare in fiecare stagiu c. creste viteza de procesare in fiecare stagiu foarte mult	B
70	**Daca dimensiunea blocului devine egala cu o fractiune semnificativa din capacitatea totala a memoriei cache, atunci:**	a. rata de esec a instructiunilor nu se modifica b. rata de esec a instructiunilor creste c. niciun raspuns corect d. rata de esec a instructiunilor scade	D
71	**Cresterea gradului de asociativitate in cadrul unei memorii cache conduce la:**	a. cresterea timpului de hit b. timpul de hit nu e influentat c. reducerea ratei de esec d. cresterea ratei de esec e. scaderea timpului de hit	A, C
72	**Ierarhiile de memorie folosesc avantajul localizarii temporale, deoarece:**	a. datele accesate recent trebuie salvate pe un spatiu de stocare extern pentru reutilizare b. datele accesate recent se pierd deoarece ele nu vor mai fi accesate c. datele accesate recent se afla cat mai aproape de procesor d. datele accesate recent se afla in memoria cache	~~Varianta 1: C (Drive ACS)~~ **Varianta 2: C, D (Andreea D, Stefan, Ovidiu)**

73	**In figura de mai jos se prezinta o memorie cache care contine 16K blocuri de 1 cuvant. se presupune ca avem memorii cache diferite pentru date si pentru instructiuni. Citirea unui cuvant presupune:**	a. adresa vine doar de la UAL b. adresa vine doar de la PC c. in caz de hit cuvantul solicitat este disponibil pe liniile de date d. cel putin 3 raspunsuri sunt gresite e. in caz de miss, adresa este trimisa in memoria principala f. adresa este generata de pc sau de ual g, se trimite adresa la memoria cache corespunzatoare	C, E, F, G
74	**Ce tip de hazard exista in situatia: In acelasi ciclu de ceas o instructiune acceseaza date din memorie cat timp instructiunea urmatoare citeste o instructiune din aceeasi memorie**	a. hazard de control b. hazard de date c. hazard structural	C
75	**Cine prezinta o localizare spatiala mai buna?**	a. instructiunile prezinta localizare spatiala mai buna b. atat instructiunile cat si datele c. datele d. nici instructiunile, nici datele	Varianta 1: C Varianta 2: B (Ovidiu)
76	**O metoda de reducere a efectului esecurilor la memoria cache este folosirea tehnicii de stationare la utilizare. Aceasta tehnica se aplica:**	a. in cazul esecurilor de date b. in cazul aparitiei unui defect fizic al memoriei cache c. ambele cazuri d. in cazul esecului de instructiuni	A si D
77	**Hazardele structurale apar atunci cand:**	a. o instructiune planificata la executie se poate executa datorita faptului ca hardware-ul suporta combinatia de instructiuni care sunt setare spre executie b. aceste hazarde apar in cazul in care se adauga un hardware nou la sistem c. o instructiune planificata la executie nu se poate executa datorita faptului ca hardware-ul nu suporta combinatia de instructiuni care sunt setate spre executie	C
78	**In cazul unei memorii cache, pentru a determina daca o data este valida sau nu se foloseste:**	a. nu se foloseste nimic b. un bloc de validitate c. un cuvant de validitate d. un bit de validitate	D
79	**Presupunem ca urmatorii octeti de adrese sunt ceruti de catre un program: 16, 24, 20 si niciuna din aceste adrese nu se gaseste in memoria cache. daca folosim o memorie cache cu blocuri de date de 1 cuvant, cate esecuri vom avea?**	a. 2 b. 1 c. 3 d. niciunul	C

80	**In cadrul figurii de mai jos sunt prezentate semnalele de control pentru procesorul MIPS varianta pipeline. Pentru executia unei instructiuni pasul de "Memoria de Access" a unei instructiuni MIPS, trebuie activate urmatoarele semnale de control:**	a. Branch, MemRead si MemWrite b. Branch, ALUOp si ALUSrc c. Branch, MemRead si MemToReg d. Branch, MemWrite si ALUSrc	A
81	**Adresa unui tabel de pagina in cazul unei memorii virtuale este:**	a. memorata intr-un registru ce indica adresa de final a tabelului b. niciun raspuns corect c. memorata intr-un registru ce indica adresa de mijloc a tabelului d. nememorabila, se calculeaza intotdeauna	B
82	**In cadrul figurii de mai jos sunt prezentate semnalele de control pentru procesorul MIPS varianta pipeline. Pentru executia unei instructiuni pasul de "Execution/Address Calculation" a unei instructiuni MIPS, trebuie activate urmatoarele semnale de control:**	a. MemWrite, MemToReg, ALUSrc b. MemWrite, ALUOp, ALUSrc c. niciun raspuns corect d. MemRead, ALUOp, ALUSrc	C
83	**Figura de mai jos prezinta executia sau calcularea adresei. Aceasta executie presupune:**	a. instructiunea LOAD citeste extensia de semn imediata de la registr	A, B, D, E(Andrei)
84	**Ce se intampla cu rata de esec a instructiunilor in cazul unei memorii cache daca modificam dimensiunea blocului?**	a. rata de esec nu este influentata de cresterea dimensiunii blocului b. rata de esec creste exponential cu rata de cresterea a dimensiunii blocului c. rata de esec scade proportional cu rata de cresterea a dimensiunii blocului d. rata de esec creste proportional cu rata de cresterea a dimensiunii blocului	C

85	**Se considera urmatorul program din figura de mai jos. Apare un hazard in cadrul programului? Ce tehnica de evitare a hazardului putem folosi?**	a. apare un hazard de date si putem folosi tehnica de forwarding urmata de un stall al benzii de asamblare b. apare un hazard de date si putem folosi forwarding c. nu apare niciun hazard de date deoarece nu ne putem intoarce in banda de asamblare d. apare un hazard si putem folosi tehnica stall	Varianta 1: A, B, D(Andrei) Varianta 2: B (Drive ACS)
86	**Presupunem o memorie cache cu blocuri de 4Kb si adrese de 32 biti. Sa se gaseasca numarul total de seturi si de biti de marcaj pentru memoria cache cu corespondenta directa.**	a. 88Kb b. 80Kb c. 128 biti d. 84 biti	B
87	**Presupunem o memorie cache de blocuri de 4Kb si adrese de 32 de biti. Sa se gaseasca numarul total de seturi si de biti de marcaj pentru memoria cache cu asociativitate partiala cu 4 cai**	a. 128 biti b. 84 Kb c. 88 Kb. d. 80Kb	C
88	**Page fault-urile in cazul unei memorii virtuale pot fi tratate:**	a. niciun raspuns corect b. numai hardware c. doar software	C
89	**utilizand adresa din PC si plasarea ei in registrul pipe IF/ID. Adresa PC este incrementata cu 4 si apoi scrisa inapoi in PC pentru a fi gata de citire in urmatorul ciclu de ceas. Adresa incrementata este salvata in registrul IF/ID pentru a fi utilizata mai tarziu. Sunt corecte etapele descrise?**	a. Nu sunt corecte. Se incrementeaza valoarea lui PC cu 4, dar nu trebuie suprascris PC b. Etapele prezentate sunt corecte. c. Nu sunt corecte. Figura nu trateaza cazul citirii unei instructiuni. d. Nu sunt corecte. Procesul poate intui instruciunea care urmeaza.	**B (Ovidiu, Roxana)**
90	**Rata de esec se defineste ca:**	a. fractiunea din accesele in memorie care nu au gasit datele in nivelul superior de memorie b. fractiunea din accesele la memorie care au gasit datele in nivelul superior de memorie c. 1 - rata de succes d. fractiunea din accesele la memorie care au gasit datele pe un mediu de stocare extern	A, C

91	**O memorie cache cu corespondenta directa presupune ca:**	a. fiecare bit sa poata fi memorat in orice locatie a memoriei cache b. fiecare cuvant sa poata fi memorat in orice locatie a memoriei cache c. fiecare cuvant sa poata fi memorat doar intr-o anumita locatie a memoriei cache d. fiecare bit sa poata fi memorat doar intr-o anumita locatie a memoriei cache	C
92	**In cadrul figurii de mai jos sunt prezentate semnalele de control pentru procesorul MIPS varianta pipeline. Pentru executia instructiunii de tipul SW, valoarea semnalelor este:**	a. Niciun raspuns corect b. RegDst = 0, ALUOp1 = 0, ALUOp0 = 0, ALUSRC = 1, Branch = 0, MemRead = 1, MemWrite = 0, RegWrite = 1, MemToReg = 0 c. RegDst = 1, ALUOp1 = 0, ALUOp0 = 0, ALUSRC = 1, Branch = 0, MemRead = 1, MemWrite = 0, RegWrite = 1, MemToReg = 1 d. RegDst = 1, ALUOp1 = 1, ALUOp0 = 1, ALUSRC = 1, Branch = 1, MemRead = 1, MemWrite = 0, RegWrite = 1, MemToReg = 1	A
93	**In cazul unei memorii cache cu organizare de tip set-asociativa, cresterea asociativitatii cu un factor de doi conduce la:**	a. cresterea dimensiunii marcajului cu 1 bit. b. Cel putin un raspuns este gresit. c. dublarea numarului blocurilor din set. d. descresterea dimensiunii indexului cu 1 bit. e. injumatatirea numarului de seturi	~~Varianta 1: A, B, D, E (Drive ACS)~~ Varianta 2: A,C,D, E (Roxana, Ovidiu)
94	**Mecanismul de realocare se aplica la:**	a. memorie virtuala b. memorie cache c. memorie principala d. nu exista un asemenea mecanism	A
95	**In cazul unei memorii cache, pentru a determina daca o data este valida sau nu se foloseste**	a. nu se foloseste nimic b. un bloc de validare c. un cuvant de validare d. un bit de validare	D

96	**Hazardul de control presupune:**	a. Luarea unei decizii bazata pe rezultatul unei instructiuni cat timp alte instructiuni sunt oprite din executie b. Reproiectarea intregului circuit c. Luarea unei decizii bazata pe rezultatul unei instructiuni cat timp alte instructiuni sunt excluse de la executie d. Luarea unei decizii bazata pe rezultatul unei instructiuni cat timp alte instructiuni sunt in executie e. Nu exista un astfel de hazard	D
97	**Controller-ul de disk (in cazul unui Hdd) are rol de:**	a. Control al transferului dintre disk si memorie b. Control al discului c. Control al transferului dintre disk si memoria cache d. Control al transferului dintre level 1 al memoriei cache si level 2 al memoriei cache	A si B
98	**Se da figura de mai jos care reprezinta executia in pipeline a programului prezentat in partea stanga a figurii. care dintre urmatoarele asertiuni sunt adevarate?**	a. se obtin rezultate incorecte deoarece ele sunt dependente de o valoare care va fi scrisa mai tarziu b. se obtin rezultate incorecte deoarece ele sunt independente de o valoare care va fi scrisa mai tarziu c. nu se obtin rezultate incorecte	A
99	**In cazul unui HDD, la timpul de acces la disk se:**	a. scade timpul necesat operarii controller-ului de disk b. adauga dublul timpului necesar operarii controller-ului de disk c. ignora timpul necesar operarii controller-ului de disk d. adauga timpul necesar operarii controller-ului de disk	D
100	**Hazardele in pipeline pot fi:**	a. Nu exista hazarde in BA b. Hazarde de control c. Hazarde structurale d. Hazarde de date e. Nu putem avea decat hazarde structurale	B, C, D
101	**Se considera programul din figura de mai jos. In acest caz este luata in considerare predictia?**	a. Da b. Nu c. Predictia nu se poate aplica in niciun caz in BA d. Predictia ne ajuta la cresterea performantei BA	A sau **B** (**Ovidiu**)

102	**Se considera urmatorul program. Executia acestui program in BA va genera hazarduri?** **lw $t1, 0($t0);** **lw $t2, 4($t0);** **lw $t4, 8($t0);** **add $t3, $t1, $t2;** **sw $t3, 12($t0);** **add $t5, $t1, $4;** **sw $t5, 16($t0);**	a. Executia programului va genera un hazard structural. b. Executia programului nu va genera hazarduri. c. Executia programul va genera un hazard de date. d. Executia programului va genera un hazard de control.	C(**Andrei**, **Ovidiu, Tudor, Vlad**) sau A, C (**Andreea, Roxana, John**)
103	**Solutionarea unui hazard de date se poate face prin**	a. Hazardul de date nu poate fi solutionat b. Forwarding c. Bypassing d. Nu exista hazard de date e Solutionarea hazardului de date se poate face doar prin intermediul unui compilator	B, C
104	**Din formatul instructiunilor MIPS rezulta:**	a. Posibilitatea aparitiei simultane a citirii si a accesarii unui operand in memoria de date b. Posibilitatea aparitiei simultane a citirii si a scrierii rezultatului intr-un registru c. Posibilitatea aparitiei simultane a citirii si a executiei d. Posibilitatea aparitiei simultane a citirii si a decodificarii	Toate = A, B, C, D
105	**Se considera urmatoarele valori:** **Acces la memorie 200ps** **Operatie ALU 200ps** **Citirea unui registru sau scrierea lui 100ps** **Timpul total de executie pentru instructiunea OR va fi:**	a. 800 ps b. 500 ps c. 600 ps d. 700 ps e. 100 ps	C
106	**Se da figura de mai jos care reprezinta executia in pipeline a programului prezentat in partea stanga a figurii. care dintre urmatoarele asertiuni sunt adevarate?**	a. Instructiunile care obtin rezultatul corect sunt and si or b. Nici o asertiune nu este adevarata. c. Instructiunile care obtin rezultatul corect sunt add si sw	C

107	**In cazul paralelismului de instructiuni, daca doua instructiuni sunt dependente:**	a. trebuiesc executate in paralel b. trebuiesc executate in ordine c. ele nu pot fi paralele d. trebuiesc executate secvential e. ele pot fi paralele	B, C, D
108	**Se da figura de mai jos care reprezinta executia în pipeline a programului prezentat în figura. Care dintre urmatoarele asertiuni sunt adevarate ?**	a.Rezultatul **incorect** este 10 b.Rezultatul **incorect** este 20 c.Rezultatul **incorect** este -10 (Exista si intrebarea cu corect- vezi linia 7)	A
109	**Se considera urmatoarele valori:** **Acces la memorie 200ps** **Operatie ALU 200ps** **Citirea unui registru sau scrierea lui 100ps** **Timpul total de executie pentru instructiunea SUB va fi:**	a. 500 ps b. 800 ps c. 700 ps d. 100 ps e. 600 ps	E
110	**Notiunea de hazard structural presupune:**	a. o instructiune planificata nu poate fi executata in propiul ciclu de ceas deoarece hardware-ul nu suporta combinatia de instructiuni. b. aparitia hazardului structural conduce la o imbunatatire a performantei BA. c. o instructiune planificata nu poate fi executata in propiul ciclu de ceas deoarece software-ul nu suporta combinatia de instructiuni. d. aparitia hazardului structural conduce la reproiectarea circuitului.	A sau **A, D (Andreea D, Vio, John, Stefi, Ovidiu, Tudor)**

111	**Se considera urmatorul program. Executia acestui program in BA va genera cel putin un hazard?** **lw $t1, 0($t0);** **lw $t2, 4($t0);** **add $t3, $t1, $t2;** **sw $t3, 12($t0);** **lw $t4, 8($t0);** **add $t5, $t1, $t4;** **sw $t5, 16($t0);**	a. Executia programului va genera cel putin un hazard care poate fi rezolvat prin tehnica BYPASSING. b. Executia programului va genera cel putin un hazard care poate fi rezolvat prin introducerea de semnale noi de control. c. Executia programului va genera cel putin un hazard care poate fi rezolvat prin tehinca de FORWARDING. d. Executia programului nu va genera niciun hazard.	A, C (Corina)
112	**Procesorul MIPS prezinta o arhitectura de tipul:**	a. load-load b. load-store c. add-load d. store-store	B
113	**Se considera urmatorul program. Executia acestul program in BA va genera hazarduri?** **lw $t1, 0($t0);** **lw $t2, 4($t0);** **add $t3, $t1, $t2;** **sw $t3, 12($t0);** **lw $t4, 8($t0);**	a. Executia programului nu va genera hazarduri b. Executia programului va genera un hazard de control c. Executia programului va genera un hazard structural d. Amandoua instructiunile add prezinta hazard deoarece prezinta o dependenta fata de instructiunea precedenta LW.	C, dar e si D incercuit sau D (Ovidiu, Tudor)
114	**Se considera figura de mai jos:**	a. In caz de LOAD sau STORE adresa addr = Rj + depl se determina (depl este marit la 32 de biti in stagiul ID) instructiunea este decodificata si tipul ei este identificat b. In cazul unei instructiuni aritmetico-logice, ALU realizeaza operatia aritmetica sau logica c. In caz de branch, PC-ul va indica valoarea corecta pentru	A, B, C
115	**Se considera urmatoarea secventa de program care se va executa in BA. In ipoteza in care exista un hazard de date, acesta poate fi eliminat?** **add $s0, $t0, $t1** **sub $t2, $s0, $t3**	a. Exista un tip de hazard, dar care nu poate fi eliminat b. Exista hazard. Acesta poate fi eliminat prin forwarding in ciclul 4 de ceas. c. Exista hazard. Acesta poate fi eliminat prin forwarding in ciclul 3 de ceas d. Nu exista hazard	C
116	**Daca 2 instructiuni sunt paralele, atunci:**	a. pot fi executate simultan intr-o BA fara a cauza stall-uri; b. pot fi executate simultan intr-o BA cauzand stall-uri; c. nu pot fi executate simultan intr-o BA; d. se vor executa obligatoriu secvential; e. pot fi executate simultan.	A, E

117	**Se considera memoria cache din figura de mai jos. Pentru localizarea unui bloc:**	a. Comparatoarele determina care dintre elementele setului selectat corespunde marcajului b. Iesirile comparatoarelor sunt folosite pentru selectarea datelor dintr-unul din cele 4 blocuri ale setului indexat folosind un multiplexor cu semnal de selectare decodificat c. Marcajele tuturor blocurilor din set trebuie parcurse in paralel d. Fiecare bloc include un marcaj de adresa ce indica adresa blocului. Acest marcaj este folosit la comparatie e. Valoarea campului index este folosita pentru selectarea setului continand adresa de interes	A, B (dubios), C, D, E
118	**Principiul localizarii:**	a. stabileste faptul ca programele acceseaza o portiune relativ foarte mare a spatiului lor de adrese la orice moment de timp. b. stabileste faptul ca programele acceseaza o portiune relativ mare a spatiului lor de adrese la orice moment de timp. c. stabileste faptul ca programele acceseaza o portiune relativ redusa a spatiului lor de adrese la orice moment de timp. d. stabileste faptul ca programele pot fi rulate sau nu.	C
119	**Se considera exemplul de mai jos. Unde poate aparea un hazard structural?**	a. Hazardul structural ar aparea in ciclul 4 de ceas b. Hazardul structural ar aparea in ciclul 2 de ceas c Hazardul structural ar aparea in ciclul 3 de ceas d. Hazardul structural ar aparea in ciclul 5 de ceas	**A si D (Ovidiu, Roxana, John)**
120	**In cazul executiei instructiunilor unui program:**	a. fluxul de date nu trebuie pastrat b. fluxul de date trebuie pastrat c. pastrarea dependentei de control este necesara pentru prevenirea modificarii ilegale a fluxului de control d. fluxul de date nu are nicio importanta in executia unui program	B si C
121	**RAID 1 este:**	a. Nu este tolerant la defecte. b. Torelant la defecte. c. Din punct de vedere al utlizatorului exista doar un singur disk. d. Cea mai ieftina solutie RAID.	B, C

122	**Comparand RAID 3 si RAID 4 putem spune ca:**	a. Sunt optimizate scrierile pentru fisiere de dimensiune mare b. Numarul de accese la disk este mare la RAID 3 c. Numarul de disk-uri ocupate este mic d. Numarul de accese la disk este redus e. Sunt optimizate scrierile pentru fisiere de dimensiune mica f. Numarul de disk-uri ocupate este mare	C, D, E
123	**Existenta unui page fault presupune:**	a. PTE-ul (Tabel pagina program) corespunzator unei pagini virtuale contine bitul de validitate setat pe 0 b. PTE-ul (Tabel pagina program) corespunzator unei pagini virtuale contine bitul de validitate setat pe x c. PTE-ul (Tabel pagina program) corespunzator unei pagini virtuale contine bitul de validitate setat pe 1 d. PTE-ul (Tabel pagina program) corespunzator unei pagini virtuale contine bitul de validitate setat pe z	A
124	**Intre stagiile unui BA trebuie sa existe:**	a. divizoare de timp. b. numaratoare. c. sumatoare. d. registre. e. comparatoare.	D
125	**In cazul unui hazard de date de tipul RAW:**	a. instructiunile care prezinta un astfel de hazard sunt ignorate la executie. b. ordinea programului nu trebuie mentinuta. c. ordinea programului trebuie mentinuta. d. RAW nu exista in cazul instructiunilor.	C
126	**Instructiunile MIPS au:** **OBS: duplicat**	a. Acelasi format b. Formate diferite c. Aceeasi lungime d. Lungime diferita	B, C
127	**Programele sunt compilate si link-editate ca si cum ar folosi intregul spatiu disponibil. Deci, adresele generate de CPU sunt adrese virtuale care vor fi:**	a. vor fi translatate in adrese reale sau fizice cand memora cache va fi referita. b. translatate in adrese reale sau fizice cand memoria principala va fi referita. c. adresele generate de CPU sunt adrese fizice. d. ignorate.	B

128	**Secventa de scriere pe un harddisk este urmatoarea:** **numar sector** **gap** **gap** **numarul urmatorului sector**	a. Scrierea pe un harddisk se face dupa o secventa mult mai complexa b. Afirmatia este incorecta c. Concept neprezentat la curs d. Afirmatia este corecta	A, B (sunt dubioase raspunsurile, nu stim cum se defineste “mult”) sau B(Ovidiu)
129	**Rata de succes se defineste ca:**	a. fractiunea din accesele la memorie ce au gasit datele doar pe CD/DVD. b. fractiunea din accesele la memorie ce au gasit datele in nivelul superior de memorie. c. fractiunea din accesele la memorie ce au gasit datele doar pe harddisk. d. fractiunea din accesele la memorie ca au gasit datele in nivelul urmator de memorie.	B
130	**Se considera urmatoarea secventa de program care se va executa in BA. In care ciclu de ceas va fi disponibil rezultatul adunarii?** **add $s0, $t0, $t1** **sub $t2, $s0, $t3**	a. In ciclul 8 de ceas b. In ciclul 5 de ceas c. In ciclul 6 de ceas d. In ciclul 7 de ceas	B sau A, B, C, D
131	**Se considera urmatoarele valori:** **Access la memorie 200ps** **Operatia ALU 200ps** **Citirea unui registru sau scrirea lui 100ps** **Tipul total de executie pentru instructiunea AND va fi**	a. 800ps b. 500ps c. 600ps d. 700ps e. 100ps	C
132	**In cazul unei BA in MIPS, stagiile de executie ale unei insructiuni sunt cele de mai jos. In cazul intructiunii STORE ce se modifica?** **IF - citirea urmatoarei instructiuni oferite de PC** **ID - decodarea instructiunii** **IE - executia instructiunii** **MEM - memorarea rezultatului si incrementarea**	a. Se modfica pasul 2. b. Se modifica pasul 1. c. Se modifica pasul 3. d. Nu se modifica niciun pas. e. Se modifica pasul 4.	C

133	**Un esec la accesarea memorie virtuale se numeste**	a. MISS b. Nu putem avea esec la accesarea memoriei virtuale c. HIT d. Page fault	D
134	**Dependentele din cadrul locatiilor de memorie sunt:**	a. Dificil de determinat deoarece doua adrese pot referi aceeasi locatie de memorie, dar continutul este diferit b. usor de identificat deoarece cunoastem numele registrelor c. dificil de determinat deoarece doua adrese nu pot referi aceeasi locatie de memorie d. dificil de determinat deoarece doua adrese pot referi aceeasi locatie de memorie, dar continutul este identic	A
135	**In cazul unui harddisk, timpul de transfer este o functie de:**	a. Viteza de rotatie. b. Dimensiunea memoriei cache. c. Densitatea inregistrarilor unei piste. d. Dimensiunea sectorului.	A, C, D
136	**Pasii de executie a unei instructiuni in BA pentru MIPS sunt:** **Pas 1 - Citire instructiune din memorie** **Pas 2 - Citirea registrelor cat timp se decodifica instructiunea** **Pas 3 - Executia operatiei sau calcularea unei adrese** **Pas 4 - Accesarea unui operand in memoria de date** **Pas 5 - Scrierea rezultatului intr-un registru**	a. Pasii prezentati sunt incorecti b. Pasii prezentati sunt corecti c. Ordinea pasilor este corecta d. Pentru executia unei instructiuni in BA pentru MIPS sunt necesari doar doi pasi e. Ordinea pasilor este incorecta f. Instructiunile MIPS nu se pot executa in BA	B,C
137	**Maparea full asociativa existenta intr-o memorie virtuala presupune:**	a. Se folosesc doar adrese fizice b. Translatarea adreselor virtuale in adrese fizice c. Memoria virtuala nu are un astfel de mecanism d. Translatarea adreselor fizice in adrese virtuale	B
138	**Se considera exemplul de mai jos:** **Loop: L.D F0, 0(R1)** **//F0 este un sir de elem** **ADD.D F4, F0, F2** **//adaugare scalar in F2** **S.D F4, 0(R1)** **//memorare rezultat** **DADDUI R1, R1, #-8** **//decrementare pointer de 8 bytes** **BNE R1, R2, LOOP** **//salt R1 != R2**	a. R1 prezinta dependente intre DADDIU si BNE. b. F4 prezinta dependente intre ADD.D si S.D. c. Ordinea trebuie pastrata pentru o execuie corecta a programului. d. F0 prezinta dependente intre L.D si ADD.D.	toate

139	**Detectarea dependentelor in cazul registrelor este usoara deoarece:**	a. Numele registrelor sunt situate aleatoriu in cadrul instructiunilor b. Numele registrelor sunt fixe in cadrul instructiunilor c. Numele registrelor nu sunt prezente in cadrul instructiunilor d. Numele registrelor nu au nicio importanta in detectarea dependentelor	B
140	**Se considera figura de mai jos. Semnificatia stagiului EX este:**	a. In caz de LOAD sau STORE adresa addr = Rj + depl se determina (depl este marit la 32 de biti in stagiul ID). Instructiunea este decodificata si tipul ei este identificat b. in cazul unei instructiuni aritmetico-logice, ALU realizeaza operatia aritmetica sau logica c. in caz de branch, PC-ul va indica valoarea corecta pentru urmatoarea instructiune	B sau B, C(Andrei)
141	**Se cunoaste de la curs ca operanzii in memorie apar doar intr-o instructiune. Aceasta restrictie presupune:**	a. Utilizarea stagiului de EX pentru calculul adresei de memorie b. Utilizarea stagiului de IF pentru calculul adresei de memorie c. Utilizarea stagiului de EX pentru calculul adresei de memorie si apoi accesarea memoriei in urmatorul stagiu d. Accesarea memoriei in urmatorul stagiu e. Nu exista asemenea restrictii f. Utilizarea stagiului de MEM pentru calculul adresei de memorie	C (aici e posibil sa fie si A + D, cu toate ca formularea e defectuoasa) sau A, C, D (Ovidiu)
142	**La timpul de acces la disk se include si:**	a. timpul necesar operarii controller-ului de disk. b. dublul timpului necesar operarii memoriei cache. c. (timpul necesar operarii memoriei cache) modulo 2. d. timpul necesar operarii memoriei cache.	A
143	**Memoria virtuala:**	a. Permite folosirea eficienta a memoriei de catre mai multe programe. b. Nu permite folosirea sigura a memoriei de catre mai multe programe. c. Nu perminte folosirea eficienta a memoriei de catre mai multe programe. d. Permite folosirea in comun a memoriei de catre mai multe programe. e. Nu permite folosirea in comun a memoriei de catre mai multe programe. f. Permite folosirea sigura a memoriei de catre mai multe programe.	A, D, F

144	**Se considera urmatorul program. Daca exista hazard specificati tipul sau.** **i1: add r1, r2, r3** **i2: lw r4, 0(r1)** **i3: sw 12(r1), r4**	a. Exista hazard si este de tipul RAW b. Nu exista hazard c. Exista hazard si este de tipul WAW d. Exista hazard si este de tipul WAR	A
145	**O instructiune j prezinta o dependenta de date fata de instructiunea i daca:** **Instructiunea i produce un rezultat care poate fi utilizat de instructiunea j**	a. Enuntul este partial corect. b. Nu exist dependente de date intre instructiuni. c. Enuntul este corect. d. Enuntul este incorect.	A sau **C (Gabriela)**
146	**Protocolul HANDSHAKING prezentat in figura de mai jos presupune urmatorii pasi:**	a. Pasii sunt prezentati incorect dar ordinea lor este corecta b. Pasii sunt prezentati corect dar odinea lor este incorecta c. Pasii sunt prezentati incorect si ordinea lor este incorecta d. Pasii sunt prezentati corect si ordinea lor este corecta	B
147	**Fiecare stagiu al unei BA trebuie:**	a. Sa posede cel putin un comparator b. Sa posede resurse hardware necesare executiei task-ului	B
148	**In cazul unui sistem de memorie virtuala, adresa unui tabel este memorata:**	a. Intr-un page table register b. Intr-un sumator c. Intr-un registru ce indica adresa de inceput a tabelului d. Intr-un comparator	A, C
149	**RAID 4**	a. Este similar cu RAID 3 b. Este similar cu RAID 0 c. Este similar cu RAID 1 d. Paritatea este memorata ca blocuri si asociata cu un set de blocuri de date	A, D
150	**In cazul unui RAID 1:**	a. Intotdeauna vom avea doua copii ale informatiei b. Cea mai ieftina solutie RAID c. Intotdeauna vom avea 8 copii ale informatiei d. Intotdeauna vom avea 4 copii ale informatiei	A
151	**RAID 5 permite:**	a. Executia in paralel a anumitor scrieri mici. b. Salvarea paritatii pe un singur disk. c. Executia secventiala a anumitor scrieri mici. d. Salvarea paritiatii pe mai multe disk-uri.	A, D

152	**Localizarea in spatiu presupune** **OBS: duplicat**	a. daca se face referire la un anumit obiect din memorie, obiectele ale caror adrese sunt invecinate cu acesta, tind sa nu mai fie adresate cat de curand b. referirea la un anumit obiect nu are nicio legatura cu principiul localizarii spatiale c. daca se face referire la un anumit obiect din memorie, obiectele ale caror adrese sunt invecinate cu acesta, tind sa fie adresate cat mai curand d. daca se face referire la un anumit obiect din memorie, obiectele ale caror adrese sunt invecinate cu acesta, nu vor mai fi accesate niciodata	C
153	**Se considera urmatorul program. Daca exista hazard specificati tipul sau.** **i1: mov 20(r3), 30(r4)** **i2: add r3, 1**	a. Exista hazard si este de tipul WAR b. Exista hazard si este de tipul WAW c. Exista hazard si este de tipul RAW d. Nu exista hazard	A
154	**In cazul unei dependente de iesire:**	a. ordinea instructiunilor trebuie pastrata b. conceptul nu a fost prezentat la curs c. ordinea instructiunilor nu trebuie pastrata d. dependenta de iesire nu se aplica instructiunilor	A (Ovidiu) sau C

155	**Secventa de scriere pe un harddisk este urmatoarea:** **numar sector** **gap** **informatia pentru sector + codul de corectie eroare** **gap** **numarul urmatorului sector**	a. Afirmatia este corecta. b. Concept neprezentat la curs. c. Afirmatia este incorecta. d. Scrierea pe un harddisk se face dupa o secventa mult mai complexa.	A
156	**In cazul unei memorii cache, indexul memoriei cache este folosit pentru:**	a. Selectarea unui bit din memoria cache b. Memoria cache nu prezinta index c. Selectarea blocului din memoria principala d. Selectarea blocului de memorie cache	D
157	**Regasirea unul bloc de date intr-o memorie cache cu asociativitate totala se face prin:**	a. Secventierea cautarii b. Examinarea primelor doua locatii de memorie cache c. Paralelizarea cautarii d. Examinarea tuturor locatiilor de memorie cache	C si D
158	**Intr-o memorie cache cu asociativitate partiala cresterea gradului de asociativitate:**	a. Creste rata de esec. b. Creste timpul de HIT. c. Reduce rata de esec. d. Scade timpul de HIT.	B, C
159	**Se da urmatorul program MIPS:** **lw $t1, 0($t0)** **lw $t2, 4($t0)** **add $t3, $t1, $t2** **sw $t3, 12($t0)** **lw $t4, 8($t0)** **add $t5, $t1, $t4** **sw $t5, 16($t0)** **In acest cod apare hazard? Daca da, unde?** **OBS: duplicat**	a. Executia programului nu va genera hazarduri b. Executia programului va genera un hazard de control c. Executia programului va genera un hazard structural d. Amandoua instructiunile ADD prezinta hazard deoarece prezinta dependenta fata de instructiunea pecedenta LW	D
160	**In cazul lui RAID 1** **OBS: duplicat**	a. Intotdeauna vom avea 2 copii ale informatiei b. cea mai ieftine solutie RAID 1 c. intotdeauna vom avea 8 copii ale informatiei d. intotdeauna vom avea 4 copii ale informatiei	A

161	**Tehnicile folosite de solutionare ale unui hazard in BA**	a. Nop b. Bubble c. Bypass d. Forwarding	A, B, C, D
162	**Perioada de ceas pentru BA se determina astfel:**	a.Diferenta dintre timpii de procesare necesare fiecarui stagiu b. maximul dintre timpii de procesare necesare fiecarui stagiu c. produsul dintre timpii de procesare necesare fiecarui stagiu d. Minimul dintre timpii de procesare necesare fiecarui stagiu	B
163	**Hazardele date la incarcare:**	a. Se solutioneaza folosind numai tehnica de forwarding b. Se solutioneaza cu bubble c. Se solutioneaza folosind numai tehnica de bypass d. Se solutioneaza prin introducerea de stall-uri e. Nu se solutioneaza	A, B, C, D sau B, D (Ovidiu, Tudor)
164	**Se considera ca rularea unui program in BA va genera un hazard de control. Eliminarea acestui hazard presupune:** **1. Se executa un Fetch** **2. In urmatorul ciclu de ceas trebuie sa avem o instructiune BEQ** **3. Dupa primirea instructiunii BEQ se incep stall-urile** **4. Stall-urile se termina in momentul in care se cunoaste adresa de ka care trebuie sa se execute urmatorul FETCH**	a. Solutia este incorecta b. Solutia rezolva un hazard structural c. Solutia rezolva un hazard de date d. Solutia este corecta	D
165	**Se da urmatorul program. Executia acestui program in BA va genera hazarduri?** **lw $t1, 0($t0)** **lw $t2, 4($t0)** **add $t3, $t1, $t2** **sw $t3, 12($t0)** **lw $t4, 8($t0)** **add $t5, $t1, $t4** **sw $t5, 16($t0)** **OBS: duplicat**	a. Executia programului nu va genera hazarduri b. Executia programului va genera un hazard de control c. Executia programului va genera un hazard structural d. Amandoua instructiunile ADD vor genera hazard deoarece prezinta o dependenta fata de instructiunea precedenta LW	C si D sau D (Ovidiu)

166	**O dependenta de iesire apare cand:**	a. instructiunile i si j nu scriu acelasi registru. b. instructiunile i si j scriu acelasi registru. c. instructiunile i si j nu scriu aceeasi locatie de memorie. d. instructiunile i si j scriu aceeasi locatie de memorie.	B, D
167	**Daca doua instructiuni sunt paralele, atunci**	a. Pot fi executate simultan intr-o BA cauzand stall-uri b. Pot fi executate simultan intr-o BA fara a cauza stall-uri c. Se vor executa obligatoriu secvential d. Nu pot fi executate simultan intr-o BA e. pot fi executate simultan	B, E
168	**O antidependenta intre i si j apare cand:**	a. instructiunea j nu scrie o locatie de memorie citita de instructiunea i. b. instructiunea j scrie o locatie de memorie cititia de instructiunea i. c. instructiunea j nu scrie un registru citit de instructiunea i. d. instructiunea j scrie un registru citit de instructiunea i.	B, D
169	**Cum determinam daca o data este valida sau nu?**	a. Nu este necesar sa determinam daca o data este valida sau nu b. Folosim un bloc de validare c. Folosim un bit de validare d. Folosim un cuvant de validare	C
170	**Se considera urmatoarele valori:** **Acces la memorie 200ps** **Operatie ALU 200ps** **Citirea unui registru sau scrierea lui 100ps** **Timpul total de executie pentru instructiunea LW va fi:**	a. 500 ps b. 800 ps c. 100 ps d. 700 ps e. 600 ps	B
171	**Se considera urmatoarele valori:** **Acces la memorie 200ps** **Operatie ALU 200ps** **Citirea unui registru sau scrierea lui 100ps** **Timpul total de executie pentru instructiunea ADD va fi:**	a. 600 ps b. 700 ps c. 500 ps d. 800 ps e. 100 ps	A

172	**Dezavantajul lui RAID 4 este faptul ca:**	a. paritatea disk-ului trebuie mentinuta pe mai multe disk-uri b. timpul de calcul al paritatii este prea mare c. nu se calculeaza paritatea disk-ului d. paritatea disk-ului trebuie recalculata la fiecare scriere	D
173	**Se considera urmatorul program. Daca exista hazard specificati tipul sau.** **i R7 = R12 + R15** **i+1 R8 = R7 - R12** **i+2 R15 = R8 + R7**	a. Exista hazard si este de tipul RAWb. Exista hazard si este de tipul \	A sau B sau A, B (Ovidiu)
174	**Pentru MIPS, operanzii in memorie apar doar in instructiunile:**	a. lwb. swc. subd. adde slt	B
175	**Controller-ul de disk (in cazul unui HDD) are rol de:** **OBS: duplicat**	a. Control al transferului dintre disk si memorie b. Control al discului c. Control al transferului dintre disk si memroia cache d. Control al transferului dintre level 1 al memoriei cache si level 2 al memoriei cache	A, B
176	**Se considera figura de mai jos. Semnificatia stagiului IF este:**	a. Valoarea lui PC se incrementeaza cu o constanta 8. b. Valoarea lui PC nu se incrementeaza. c. Instructiunea este citita din memorie(I-cache) de la adresa indicata de PC. Se presupune ca instructiunea nu este o instructiune de tip branch si de aceea incrementeaza PC catre urmatoarea instructiune. d. PC-ul nu are nicio contributie in acest stagiu.	C
177	**Un bloc de memorie virtuala este denumit:**	a. Pagina b. Cuvant c. Semicuvant d. Bloc	A

178	**In cazul MIPS, fiecare registru general are:**	a. 128 biti b. 32 biti c 4 biti d. 64 biti e. 8 biti	B
179	**MIPS poseda:**	a. O memorie cache de instructiuni. b. Program counter. c. O memorie cache de date. d. PC. e. Registre generale.	A, B, C, D, E
180	**RAID 2:**	a. foloseste doar detectare de erori. b. foloseste doar corectarea de erori. c. foloseste detectare si corectare erori. d. nu este posibila detectia si corectia de erori.	C
181	**Se considera urmatoarele valori:** **Acces la memorie 200ps** **Operatie ALU 200ps** **Citirea unui registru sau scrierea lui 100ps** **Timpul total de executie pentru instructiunea SW va fi:**	a. 700 ps b. 600 ps c. 100 ps d. 800 ps e. 500 ps	A
182	**Tehnicile folosite de solutionare ale unui hazard in BA sunt:**	a. predictie hardware dinamica b. bubble c. nop d. predictia	toate = A, B, C, D
183	**Se considera executia instructiunii lw conform figurii de mai jos. Executia acestui stagiu presupune:**	a. La acest moment se stie tipul instructiunii citite asa ca procesorul trebuie sa fie pregatit pentru un anumit tip de instructiune. b. La acest moment nu se stie tipul instructiunii citite asa ca procesorul trebuie sa fie pregatit pentru orice tip de instructiune. c. Salvarea valorii PC-ul este necesara pentru apartia instructiunii beq. d. Salvarea valorii PC-ul este necesara pentru urmatorul ciclu de ceas.	B, C
184	**In cazul paralelismului la nivel de instructiune, daca instructiunile sunt dependente intre ele, atunci:**	a. nu exista dependente intre instructiuni in acest caz b. ele pot fi partial suprapuse c. nu pot fi suprapuse d. pot fi suprapuse total	C

185	**Se considera figura de mai jos. Semnificatia stagiului MEM este:**	a. Daca instructiunea este un SUB - dar nu STORE - rezultatul operatiei (sau cel al unui LOAD) este memorat in registrul rezultat b. Daca instructiunea este un branch - dar nu STORE - rezultatul operatiei (sau cel al unui LOAD) este memorat in registrul rezultat. c. Daca instructiunea este un NOT - dar nu STORE - rezultatul operatiei (sau cel al unui LOAD) este memorat in registrul rezultat. d. Daca instructiunea este un ADD - dar nu STORE - rezultatul operatiei (sau cel al unui LOAD) este memorat in registrul rezultat.	B sau A, C, D (Ovidiu) sau A, B, C, D
186	**In cazul unei memorii cache cu asociativitate partiala**	a. Blocul de date poate fi amplasat doar in prima cale din interiorul setului determinat b. Blocul de date poate fi amplasat oriunde in interiorul setului determinat c. Blocul de date poate fi amplasat in orice set d. Nu exista o astfel de amplasare a memoriei cache	B
187	**Se dă figura de mai jos care reprezintă execuția în pipeline a programului prezentat în figura. Care dintre urmatoarele asertiuni sunt adevarate?**	a. Rezultatul incorect este 10. b. Rezultatul incorect este 20. c. Rezultatul incorect este -10.	A
188	**Memoria virtuala ofera mecanismul de:**	a. Relocare pe baza de blocuri de dimensiune egala cu dublul capacitatii memoriei cache b. Relocare pe baza de blocuri de dimensiune egala cu capacitatea harddiskului c. Relocare pe baza de blocuri cu dimensiune fixa d. Relocare pe baza de blocuri cu dimensiune variabila	C
189	**Presupunem o memorie cache de blocuri de 4Kb si adrese de 32 de biti. Sa se gaseasca numarul total de seturi si de biti de marcaj pentru memoria cache cu asociativitate partiala cu 2 cai**	a. 128 biti b. 84 Kb c. 80 Kb d. 88 Kb	B

190	**Figura de mai sus prezinta cum se realizeaza faza de executie a instructiunii de memorare. Care din pasii de mai jos sunt necesari pentru aceasta operatie ? (Fara figura)**	a. Aduna ce se citeste din registrul EI/DI, continutul lui $t2 b. Aduna ce se citeste din registrul EI/DI, continutul lui $t1 si câmpul imediat c. Aduna ce se citeste din registrul EI/DI, continutul lui $t2 si câmpul imediat d. Aduna ce se citeste din registrul EI/DI, continutul lui $t1 si câmpul imediat cu semn extins	D
191	**TLB sunt:**	a. buffere de translatare b. memorie cache care foloseste tehnica de write-back	A, B, D